

LEARNING TO THEORIZE IN MENTAL TERMS

MSc Thesis (*Afstudeerscriptie*)

written by

Gidon Kaminer

under the supervision of **Dr Fausto Carcassi**, and submitted to the
Examinations Board in partial fulfilment of the requirements for the degree of

MSc in Logic

at the *Universiteit van Amsterdam*.

Date of the public defence: **Members of the Thesis Committee:**

August 28th, 2026

Dr Fausto Carcassi (supervisor)

Dr Malvin Gatteringer (chair)

Dr Martha Lewis

Dr Giorgio Sbardolini



INSTITUTE FOR LOGIC, LANGUAGE AND COMPUTATION

Abstract

Contents

1	Theory of Mind and its Acquisition	5
1.1	The Nature and Scope of Theory of Mind	5
1.2	Experimental Evidence: Developmental Milestones	6
1.3	The Explanandum	6
1.4	Theoretical perspectives on ToM acquisition	7
1.4.1	Nativism and core knowledge	8
1.4.2	Empiricism, association, and cultural learning	8
1.4.3	Rational constructivism and the theory theory	9
2	Learning as Bayesian Program Synthesis	12
2.1	Bayesian inference: the logic of learning	12
2.2	Programs as the hypothesis space	14
2.3	Program induction and the search problem	16
2.4	Learning the language itself	17
2.4.1	The library as an overhypothesis	17
2.4.2	Bootstrapping and minimal nativism	19
2.5	What is claimed, and what is not	21
3	Bayesian Theory of Mind	22
3.1	Agent as planner	22
3.2	Inverse planning	23
3.3	Belief as an input	23
3.4	What is stipulated in BToM	24
3.5	What this thesis asks	25
4.	The Model	27
4.1.	The learning problem	27
4.2.	The task corpus	27
4.3.	The hypothesis space	28
4.3.1.	Programs as typed expression trees	28
4.3.2.	Type system	29
4.4.	The primitive library	30
4.4.1.	The target compound	31
4.5.	Inference	32
4.5.1.	Search	32
4.5.2.	Library revision	33
4.6.	The model does not encode a Bayesian theory of mind	35
4.6.1.	The decomposition into agent, goal and belief	35
4.6.2.	The separation of the believed world from the actual world	36
4.6.3.	The space of admissible divergences	37
4.6.4.	The space of goals	37
4.6.5.	Rationality, and the one thing we do grant	38

4.6.6.	Two confounds that are ours rather than BToM's	38
4.6.7.	Two structural differences worth naming	39
4.7.	What would count as success or failure	40
5	Results	42
5.1	The runs	42
5.2	What the loop finds	43
5.2.1	The combinator library: a two-stage bootstrap	43
5.2.2	The atomic control	45
5.2.3	The agency signature	47
5.3	The discovered term against BToM	47
5.4	The objective selected it	49
5.4.1	The corpus description length	49
5.4.2	What the library buys each family	50
5.4.3	No non-mental rival reproduces the scenes	51
5.5	Reachability, and the order in which things arrive	53
5.6	Generality and selectivity	54
5.6.1	The constructor spans the corpus	54
5.6.2	The symmetric complements go elsewhere	55
5.6.3	Attribution is selective	56
5.7	Scaffolding	58
5.8	The behavioural probe	59
5.9	The verdict	61
5.10	Objections	62
5.10.1	“fork and <code>sync_to_world</code> are just <code>believe</code> cut in half”	62
5.10.2	“But <code>fork</code> still smuggles in a representational capacity”	63
5.10.3	“The regress does not end”	63
5.10.4	“The corpus was built to produce this result”	64
5.11	What the results do not show	64
6	Further work	66
6.1	Comparison to LLMs	66
6.2	Polymorphism	66
6.3	Channel arity as a free parameter	66
6.4	persistent models and interaction	67
6.5	breaking down optimize	67
6.6	probabilistic primitives	67
6.7	vs. real world	68
6.8	richer belief content	68
6.9	recursive mindreading	68
6.10	POMDP	68
7.	Appendix	69

7.1. Full Corpus	70
7.2. The interpreter	72
7.3. The primitive catalogue	73
7.4. Program representation	75
7.4.1. The library object	76
7.5. Task generators and necessity filters	76
7.6. Cost-bounded enumeration	77
7.7. Compression with Stitch	78
7.7.1. Normalization before compression	79
7.7.2. No recognition model	79
Bibliography	81

Chapter 1

Theory of Mind and its Acquisition

1.1 The Nature and Scope of Theory of Mind

The ice cream peddler’s scratchy jingle got louder still as he brought his cart to a stop nearby. A child, hitherto engrossed in sandbox amusements, abandoned her playmates abruptly. She darted across the playground, hurtling at full speed towards a gaggle of parents. Absorbing the impact of a 20 kg cruise missile, the target continued his conversation unfazed by the girl now tugging at his jeans. Only when he deigned to look down at the child did he shriek and go pale. But the girl barely had time to react, because just then she heard her name shouted from the other side of the playground. She turned to its source and ran towards a similar-looking man whose outstretched arms held two ice cream cones.

You chuckle at the scene, observing from a nearby park bench. The girl, wanting ice cream, pestered a man she thought was her father. The man, undoubtedly accustomed to his own nagging kids, ignored the girl thinking it was his own child. Descriptions like these, in terms of desires, beliefs, and intentions, seem trivial and obvious. But there’s something miraculous about them. How are you able to peer into other peoples’ minds? Mental content is unobservable; all you see is a sequence of physical actions. And yet humans, even infants, are experts at inferring mental content rapidly and accurately.

Humans have the cognitive ability to impute mental states to others: to attribute a belief, a desire, or an intention to another person. This capacity is known as “theory of mind”, “commonsense psychology”, “naive psychology”, “folk psychology”, or the ability to “mindread”, to “mentalize” (Margolis et al., 2012). Among these we’ll primarily use “theory of mind”, henceforth ToM (Premack & Woodruff, 1978). We choose this term to emphasize that this capacity is a *theory* proper in that it enables behavioral prediction by positing states that are not directly observable (Gopnik & Wellman, 1992).

The mental states we attribute include perceptual states, bodily feelings, emotional states, and propositional attitudes such as beliefs, desires, hopes and intentions (Margolis et al., 2012). This thesis is concerned with the attribution of propositional attitudes, particularly belief. A propositional attitude relates an agent to some content (“she believes that x ”), so to attribute a belief to an agent is to attribute a *model* that may diverge from the world, and to predict behaviour from the model rather than from the world (Crane, n.d.; Perner, 1991). In this thesis we study how this capacity to attribute and predict off a divergent world model is acquired. How do humans, even at a young age, come to have it?

1.2 Experimental Evidence: Developmental Milestones

ToM doesn't arrive all at once, but in pieces over roughly four years and in a roughly fixed order (Beaudoin et al., 2020; Coşkun et al., 2020; Rakoczy, 2022; Wellman, 2018; Wellman & Liu, 2004). Long before children can say anything about minds, they parse motion as goal-directed. Woodward habituated 5- to 9-month-olds to a hand reaching for one of two objects and then swapped the objects' positions. Infants looked longer when the hand reached to the old location for a new object than when it reached to a new location for the old object (Woodward, 1998). So the infant encodes the goal object, not the trajectory.

Gergely et al. showed that, more than object-directed, the interpretation is even efficiency-sensitive: 12-month-olds expect an agent to take the most efficient means its situation affords, and are surprised when it does not (Gergely et al., 1995). Gergely and Csibra call this schema the *teleological stance* and distinguish it from the intentional stance (Csibra & Gergely, 2007; Gergely & Csibra, 2003). So before the first birthday, an infant can read bodies as taking efficient means to ends. This is *not* ToM. There's no attribution of mental states, it's just an extensional story.

In the second year, infants become sensitive to what an agent can (and can't) see, and begin to condition their expectations about an agent's goal based on the agent's perceptual access to the objects involved (Luo & Baillargeon, 2010). This is a perspective-taking ability, but restricted mostly to the epistemic case of perceptual access. ToM similarly requires perspective-taking, but pertaining to more complex mental states.

The jump to ToM involves a transition from reality-centered to belief-centered prediction. In the original *false-belief* task, Maxi puts his chocolate in one cupboard and leaves; his mother moves it to another; Maxi returns, and the child is asked where he will look (Wimmer & Perner, n.d.).¹ Though the observing child knows the chocolate is in the other cupboard, the child must predict Maxi's behavior according to *his* (false) belief, not according to the *child's* (true) belief. Children under about 3.5 years consistently fail, and (tellingly) their scores don't improve when the scenario is made more explicit. The age threshold is robust across task variations, countries and manipulations, aggregated over 178 studies (Wellman et al., 2001).²

1.3 The Explanandum

We first state some of the features of the ToM capacity, which a (good) theory of the acquisition of ToM must account for. Take for example someone trying to learn an intuitive physics by watching a ball's motion. The learner can simply extract regularities from what she sees, because a ball's position and velocity are recoverable from a sequence of images. A learner of intuitive psychology (ToM) doesn't have that luxury. What is available to the observer is a trajectory of bodies through space, but what has to be acquired is a vocabulary of attitudes that appear nowhere in the trajectory

¹The task is also commonly known as the Sally–Anne task (Baron-Cohen et al., 1985), with similar setup

²4 years as the threshold has been challenged by studies seemingly showing *implicit* false-belief ability around 1.5 years (Buttelmann et al., 2009; Onishi & Baillargeon, 2005; Southgate et al., 2007). But the findings have replicated badly (Barone et al., 2019; Dörrenberg et al., 2018; Kulke et al., 2018) and even granting them, they may be accounted for by early automatic tracking abilities (Apperly & Butterfill, 2009; Butterfill & Apperly, 2013). For the purposes of this thesis, we take the ToM scale (Wellman et al., 2001; Wellman & Liu, 2004) as our explanandum.

(Churchland, 1981; Sellars, 1956). So an acquisition procedure which works by finding structure in the observable variables can't yield ToM.

Another acquisition problem is that attributions are underdetermined by behaviour.³ Any behaviour is compatible with indefinitely many belief–desire pairs and no amount of further observation narrows the field to one, similarly to Quine on translation (Quine, 2013).⁴ Yet humans select a fitting belief–desire explanation quickly, confidently, and on very little data. An acquisition theory should explain what does the selecting.

The underdetermination problem presupposes a learner who already has the concepts and must choose among ascriptions built from them. But a learner who lacks the concept *believes* cannot make even a wrong attribution. So the acquisition problem isn't just curve-fitting or parameter estimation within a given theory. It requires constructing the theory's terms, positing unobservable entities to explain observable behavior. Fodor argued that no learning process could deliver a concept the learner did not in some sense already have (Fodor, 1992; 1998), and Carey's work on conceptual change tries to answer Fodor's challenge (Carey, 2011). An account of acquisition must explain how novel conceptual terms can come to exist.

Because *believes* takes any proposition as an argument, the set of available attributions is unbounded: one can entertain attributions never encountered in the data, including attributions of beliefs one takes to be false and beliefs about things that do not exist. The operator also embeds in itself, which enables second-order mentalizing (Perner & Wimmer, 1985). And the attributions come in systematic families: anyone who can represent that Sally believes the marble is in the basket can also represent that Anne believes it is in the box (Fodor & Pylyshyn, n.d.). The terms get their content from their role in a network of inferences rather than from links to observations one at a time. So a good acquisition theory must offer a representational medium in which mental content composes, embeds, and licenses inference.

Finally, we saw in Section 1.2 that goal attribution is in place before the first birthday while explicit false-belief prediction is not reliable until around four. A good account of ToM acquisition should explain this long delay. Also, the later competences look like assemblies of the earlier ones: belief attribution appears to be goal attribution plus subjective perceptual access. So in a good account, this ordering would be explained by the acquisition mechanism itself.

1.4 Theoretical perspectives on ToM acquisition

We now survey the standing accounts of how theory of mind is acquired, evaluating them according to the ToM features in Section 1.3. Nativism, empiricism, and constructivism are general positions in the debate about cognitive development, but we'll consider them specifically in the ToM context.⁵

³For example, an agent walking across the room to the cupboard can be explained as wanting the chocolate and believing it is there, or as wanting to check whether it has been moved and believing it is there, or as wanting to be seen approaching the cupboard, and so on without end.

⁴This is a special case of the fact that data does not select a hypothesis without a prior (Goodman, 1983).

⁵There's also simulationism, which denies that mindreading proceeds by a theory at all. On this account the attributor runs her own decision-making machinery offline on pretend inputs rather than applying general principles connecting states to behaviour (Goldman, 2006; Gordon, 1986).

1.4.1 Nativism and core knowledge

The nativist holds that ToM is a built-in system that matures over time, so it’s not acquired from data in any interesting sense. Nativist positions are mainly motivated by the poverty of stimulus argument (PoS): the data available to the child is too sparse and too degenerate to fix the target by general learning, so the constraints that do fix it must be innate and domain-specific (Berwick et al., 2011; Chomsky, 1975; 2014).

Leslie argues for a theory of mind module (ToMM): a natively endowed, domain-specific system for interpreting the behaviour of intentional agents, which introduces attitude concepts such as *believes* and deploys a “proprietary representational system” for ascribing them to agents (Leslie, n.d.; 1994). The dissociation found in autism is held as evidence that ToM is a dedicated system, since it can be selectively impaired (Baron-Cohen et al., 1985)⁶.

Fodor’s nativism argues that the child’s theory of mind is not learned, because there is no coherent story on which a concept like belief could be learned (Fodor, 1992). So concepts are not constructed, and the productivity and systematicity of ToM are properties of a representational format that was there from the start. There’s also the “core knowledge” nativist position, which posits a small stock of encapsulated systems that give the child a skeletal domain-specific ontology from which they elaborate to more complex concepts (Spelke, 2022; Spelke & Kinzler, 2007).

The almost three year developmental delay from goal attribution to false-belief prediction is a challenge for nativists, since they must explain why an innate capacity isn’t manifested until later in development. A ToMM account can appeal to maturation of the module, or argue that task performance demands mask underlying native competence (Leslie, 2005; Scholl & Leslie, 2001). But these concessions accomodate the developmental data by auxiliary hypotheses. What would be better is a theory that itself predicts staggered developmental results.

Nativism answers the hard requirement — how a learner comes to have terms for unobservables — by declining it.

1.4.2 Empiricism, association, and cultural learning

The empiricist holds that theory of mind is learned like anything else: through experience, from statistical regularities, by bottom-up association. In a connectionist implementation knowledge is distributed over the weights of a network, concepts are patterns of activation rather than symbols, and structure in behaviour is emergent rather than represented (McClelland, 2010; Rumelhart, 1986). Connectionist networks have even been shown to account for false belief without using symbolic representation (Berthiaume et al., 2013).

There’s also Heyes’ non-connectionist position which takes that ToM is a cognitive gadget, i.e. a culturally inherited skill assembled out of general-purpose learning mechanisms and transmitted through social interaction and talk about the mind (rather than a genetically inherited instinct) (Heyes, 2018; Heyes & Frith, 2014). Ruffman argues in a similar spirit that the developmental data can be accounted for without crediting children with belief concepts at all, by low-level and statistical means (Ruffman, 2014). In the linguistic domain, one empiricist approach derives the capacity to

⁶TODO though these experiments have also faced replication troubles ADD CITATIONS

represent a false complement from acquiring the syntax of sentential complementation (Milligan et al., 2007; Villiers & College, n.d.).

Empiricist accounts are well-suited to explaining how ToM emerges even though mental content isn't in the data. But these explanations often deny that the structure of ToM is acquired at all, instead they just show how the learner acquires useful associations from data: it learns a mapping. And then the empiricist can't account for systematicity, since there's no representational medium with constituent structure (Fodor & Pylyshyn, n.d.).

Against a purely associative empiricism, the observed developmental order (Section 1.2) is not obviously the order of frequency in experience. Nothing about the input makes goal-directed pursuit a hundredfold more frequent than the conjunction of pursuit with restricted perceptual access. And no empiricist implementation shows how a data frequency difference leads to a three year false belief delay.

Furthermore, the later competences appear to presuppose the earlier ones rather than to be acquired independently alongside them as a separate regularity learned from its own statistics. The cultural-learning empiricist could attribute the competency ordering to the ordering of the input (Heyes & Frith, 2014). But that's a hypothesis about the input rather than a mechanism, and also it predicts that the developmental sequence should vary between children more than it appears to (Wellman & Liu, 2004).

Empiricism takes seriously the fact that the content is not in the data, and then concludes that there is no content to acquire.

1.4.3 Rational constructivism and the theory theory

The constructivist holds that the child is analogous to a scientist (Downes, 1999; Gopnik, 2012). A child constructs intuitive theories of mind and world which she revises in light of evidence, by something like the process that drives scientific progress (Gopnik, 1997; Gopnik & Wellman, 1992; Wellman et al., 1990). So domain-general statistical machinery constructs domain-specific structured knowledge. Infants have been shown to be sensitive to sampling and to statistical evidence in ways that indicate they do inference to hidden structure (including psychological structure) (Wellman et al., 2016; Xu, 2019; Xu & Kushnir, 2013). And in general children's exploratory behaviour is well described as evidence-gathering aimed at resolving uncertainty about which theory is right (Schulz, 2012).

Rational constructivists specify that Bayesian inference is the formal mechanism of theory change (Gopnik & Wellman, 2012; Newcombe, 2011; Xu et al., 2012). Structured, abstract knowledge, embedded in a probabilistic framework, can be acquired from sparse and noisy data (Gopnik & Tenenbaum, 2007; Griffiths et al., 2010; 2011; Perfors et al., 2010; Ullman & Tenenbaum, n.d.; Xu, 2019). For example, the transition from failure to success in false-belief tasks is analyzed as an evidence-driven shift between two causal Bayesian models. The infant starts with a simpler model, but after encountering evidence inexplicable by that model switches to a more complex one which offers greater explanatory resources (Goodman et al., n.d.).

Theory formation generally has been modeled as probabilistic inference over systems of related concepts (Goodman et al., 2011; Kemp et al., 2010), and theory change has been modelled as stochastic search through a space of theories generated by a grammar (Ullman et al., n.d.).

Theories have terms; theory change introduces new ones; and the terms are answerable to evidence without being read off it.

Nativism posits domain-specific logical symbolic mechanisms, whereas empiricism posits domain-general probabilistic statistical mechanisms. Rational constructivism seeks a middle-way between various dichotomies in the field (Griffiths et al., 2024)

“Powerful abstractions can be learned surprisingly quickly, together with or prior to learning the more concrete knowledge they constrain. Structured symbolic representations need not be rigid, static, hard-wired, or brittle. Embedded in a probabilistic framework, they can grow dynamically and robustly in response to the sparse, noisy data of experience.” (Tenenbaum et al., 2011)

, by asking “how can domain-general mechanisms of learning and representation build domain-specific systems of knowledge? How can structured symbolic knowledge be acquired through statistical learning?” (Tenenbaum et al., 2011)

“How do our minds get so much from so little?” (Griffiths et al., 2024) poverty of stimulus

On this account the four-year transition is a theory change. The child’s earlier answers are systematic and wrong (rather than merely absent) because she has an earlier theory.

The delay. This is the position that says the most about the ordering, because it says that acquisition is *cumulative*: what is learned early becomes the material out of which what is learned later is composed. That is the bootstrapping picture (Carey, 2011; Piantadosi et al., 2012). The explanandum is then not merely that the capacity appears but that it appears in this order, with the later stages built from the earlier ones, and the account has the right shape to explain it. A prior over hypotheses is a way of selecting among attributions that the data underdetermines; a structured representational medium is one in which content composes, embeds, and executes; and a mechanism by which abstractions acquired early become the primitives from which later ones are built is exactly what a cumulative ordering requires. Whether such a mechanism actually delivers a mental vocabulary, rather than merely being able to express one, is the question the rest of this thesis puts to the test.

The theory theory comes closest. It has the right shape for every one of the five requirements: theories have terms, terms are constructed, construction is answerable to evidence, the medium is structured, and construction is cumulative so the order of acquisition is a consequence of what has already been built. But having the right shape is not the same as being a mechanism. The account says that children construct theories; it does not say what process does the constructing, nor by what standard one candidate theory is preferred to another, nor — and this is the sharp version — how a learner searches a space of possible theories that is infinite. The metaphor of the child as scientist is powerful, and it describes what happens rather than naming something that would make it happen.

That gap is what keeps the nativist in business. Absent a mechanism, the nativist can say that no constructive process exists, that what looks like construction is really the maturation of something

already in place, and that the appearance of theory change is a redescription of a developmental timetable. The strength of that reply is parasitic on the absence of a mechanism. It is not an argument that construction is impossible; it is an argument that nobody has exhibited one.

So the business of the next chapter is to supply one. If the theory theory needs a process that searches a space of structured hypotheses, prefers among those that fit the data on principled grounds, introduces new terms whose content is fixed by their role rather than by observation, and makes what is learnable at each stage depend on what has already been learned, then that is a specification, and it is a specification we can go looking for a formal system to meet. Chapter 2 argues that Bayesian program induction is such a system, and that DreamCoder is a concrete algorithm that implements it.

Chapter 2

Learning as Bayesian Program Synthesis

The mind takes as input a trajectory of bodies through space, and outputs the process that generated the trajectory. The output process is formulated in terms of mental states (beliefs, desires, intentions) that don't appear in the data. The trajectory underdetermines the process, since any behaviour is compatible with indefinitely many belief-desire pairs. So no amount of further observation eliminates all but one.

Stated this way the problem is an instance of a familiar one: given data, entertain hypotheses about the process that generated it, assign each a prior plausibility, and revise those plausibilities as new data comes in. There's already a known answer to the normative question of how such revision ought to proceed, which is Bayes' rule. So our computational-level claim (Marr, 2010) is that acquiring a theory of mind is the problem of finding the best explanation of observed behaviour, where the best explanation takes into account both its prior plausibility and how well it fits the data.

2.1 Bayesian inference: the logic of learning

Say a learner observes data d , a trajectory of bodies through space. A learner's task is to choose among possible explanations $\mathcal{H}$ a hypothesis h that best explains the trajectory. The task is to choose the most plausible hypothesis given the data, i.e. to find the hypothesis whose posterior probability $P(h|d)$ is highest. In calculating this quantity, the learner should weigh two things. First, the prior probability $P(h)$: how plausible was hypothesis h before any data d was observed at all? Second, the likelihood $P(d|h)$: how well does hypothesis h account for the data d , now that data is available? Bayes' rule specifies how to combine these considerations:

$$P(h|d) = \frac{P(d|h)P(h)}{\sum_{h' \in \mathcal{H}} P(d|h')P(h')}$$

Since the denominator is a normalising constant that doesn't depend on h , it can be ignored for the purpose of ranking hypotheses against each other. So we rewrite the equation as the proportion

$$P(h|d) \propto P(d|h) \cdot P(h)$$

i.e. the *posterior* probability $P(h|d)$ of a hypothesis h given data d is proportional to the *likelihood* $P(d|h)$ of the data d given hypothesis h times the *prior* probability $P(h)$ of hypothesis h before data d was observed.

The prior $P(h)$ expresses the a priori plausibility of a hypothesis, i.e. how likely the learner takes h to be, independent of the current observations. So the prior encodes the learner's background

knowledge. For example a hypothesis that is outright impossible has prior $P(h) = 0$ so it's excluded from $\mathcal{H}$ and no data observation can increase its posterior probability. The prior determines which explanations are more natural, so we can start to generalize from among the indefinitely many hypotheses consistent with the data (Frank et al., 2009; Xu & Tenenbaum, 2007).

Anything can be a prior, since it's just what encodes the learner's background knowledge, assumptions, and preferences. In this thesis the prior we'll focus on is the preference for simplicity. Hypotheses that are short, that reuse existing structure rather than stipulating new structure, will have higher priors than hypotheses that are long and idiosyncratic. This assumption is supported by accounts of human rule-based concept learning (Chater & Vitányi, 2003; Goodman et al., 2008), and by the finding that the difficulty of learning a Boolean concept tracks the length of its shortest logical expression (Feldman, 2000). In Section 2.2 we'll say precisely how the simplicity of a hypothesis is calculated, after we've said what hypotheses are made of.

The likelihood $P(d|h)$ expresses how probable the observed data would be if the hypothesis were true. In our setting a candidate hypothesis either explains the observed behaviour (i.e. it reproduces the trajectory exactly) or it does not. So the likelihood is all-or-nothing: the data have probability zero $P(d|h) = 0$ if they disagree with the hypothesis, and constant probability $P(d|h) = c$ otherwise (Goodman et al., 2008). The binary likelihood can only ever eliminate hypotheses, since those with likelihood $P(d|h) = 0$ yield posterior $P(h|d) = 0$. So once a set of hypotheses all fit the data perfectly (only those with likelihood $P(d|h) = c$ remain), the likelihood can't distinguish between the remaining hypotheses and their ranking is decided entirely by the prior. This division of labour is forced by the domain rather than chosen for convenience. Behaviour here is a trajectory through a discrete grid, and a candidate explanation either reproduces it cell for cell or it does not; there's no residual quantity for a graded likelihood to measure, so partial credit would have to be invented rather than observed (interpreter). Everything the data can do is therefore done by elimination, and every comparison among the survivors falls to the prior.⁷

Bayes' rule is a theorem of the probability calculus: given the basic commitment to representing degrees of belief as probabilities, it's the unique coherent way to revise them in light of evidence (Jaynes & Bretthorst, 2003; MacKay, n.d.). Having stated the computational problem the mind faces, Bayes' rule specifies the ideal solution. And the ideal solution generates predictions that can be checked against what people actually do. If human judgements line up with those predictions, we have grounds for modeling the mind as solving the Bayesian inference problem, and we have an explanation of *why* people behave as they do rather than merely a description of the fact that they do (Griffiths et al., 2024; Marr, 2010).⁸

⁷This gives up something Bayesian models of generalization normally rely on. Where hypotheses pick out sets of possible observations, a hypothesis that admits fewer observations assigns each of them more probability, so the likelihood favours the most specific hypothesis consistent with the data and does real graded work — the size principle (Tenenbaum & Griffiths, 2001). A deterministic program either emits the observed trajectory or doesn't, so there's no distribution over observations left for that principle to bite on.

⁸To be clear this claim is at the computational level, not at the algorithmic level. Nobody is saying that a child computes posterior probabilities, or represents the hypothesis space $\mathcal{H}$ explicitly, etc. The claim is that the child's behavior approximates the solution to an inference problem whose ideal form is described by Bayes' rule. It's not an algorithmic-level claim about the procedure by which the child does this, though we'll later introduce an algorithm for approximating this inference.

2.2 Programs as the hypothesis space

Bayes’ rule doesn’t specify what we should define our hypothesis space over, so we can choose the best medium for representing our hypotheses. Since we choose simplicity as our prior, hypotheses should have parts that can be counted so their relative size and complexity can be compared. And a hypothesis must be executable, since the indicator likelihood asks whether running a hypothesis reproduces the observed trajectory (Section 2.1). A finite learner must have unbounded and systematic capacity (Section 1.3), so the hypotheses must be generated by rules of combination (Fodor & Pylyshyn, n.d.). And also the hypothesis space must admit new terms, since we want to model how a learner acquires the terms that denote mental content (attitudes) even though they don’t appear in the data.

These requirements rule out connectionist representations (Section 1.4.2) where a hypothesis is a weight vector or a point in a high-dimensional parameter space (Tenenbaum et al., 2011). Though they’re efficient to update, they lack a natural notion of countable constituent parts so we can’t compare their relative sizes, and this is exactly the respect in which such models have been argued to fall short of human learning: they generalize from many examples rather than building explanatory structure from few (Lake et al., 2016). What we need is a structured symbolic representation like a grammar, predicate logic, relational schemas, or functional programs (Griffiths et al., 2024; Tenenbaum et al., 2011). What we need is a language of thought (Fodor, 2010), where mental representations are symbolic objects combined by syntactic rules and evaluated as a function of their parts (Bruner et al., 1986; Carey, 2011; Siskind, n.d.).

Programs are the natural medium for our hypothesis representation. They are compositional, since the value of a composite expression is a function of the values of its parts. And a program admits any expression of the right type in any argument position of that type. Programs can express any effectively computable relation between input and output (Church, 1936; Turing, 1937), and they are executable. And they encode procedural and declarative knowledge in the same format (Rule, n.d.). There’s a long tradition of treating thought this way — problem solving as symbol manipulation, the mind as a compact program that mirrors the structure of the world (Baum, 2004; Goodman et al., 2014; Newell et al., 1958). A symbolic medium also leaves the learner free to adopt whatever vocabulary turns out to be useful rather than whatever the modeller happened to supply (Gopnik & Wellman, 2012), which is the freedom Section 2.4 will exploit. And furthermore, if knowledge is programs then learning is program induction (Flener & Schmid, 2008; Gulwani et al., 2017; Schmid et al., n.d.) which we’ll soon make use of.

We adopt the λ -calculus as our program representation formalism, since it’s the standard choice for models of learning across domains (Piantadosi et al., n.d.; 2012; Schmidt, n.d.; Zettlemoyer & Collins, n.d.). Most importantly it gives us β -abstraction as the operation of taking a recurring piece of structure and giving it a name. So this way a learner can come to possess a term that names something that isn’t observable in the data, since the new term’s content is fixed entirely by what it does in combination the other terms.⁹

⁹The caveat of Section 2.1 applies here too: we don’t claim the mind implements a λ -calculus interpreter, only that its ToM competence is well described as inference over hypotheses that compose, execute, embed, and can be measured for size — all properties the λ -calculus representation supplies.

Since our programs are composed of terms of a grammar, a distribution over a grammar’s productions induces a distribution over expressions. Since the probability of an expression is the product of the probabilities of the choices made in generating it, longer expressions (involving more choices) get less probability mass. The cost of writing a message in a code optimal for that distribution is the negative log of its probability (Shannon, n.d.), so h ’s description length under the given grammar is $DL(h) = -\log P(h)$. So the prior is $P(h) \propto e^{-DL(h)}$. And we have an indicator likelihood for whether executing h reproduces the observed behavior. Substituting into Bayes’ rule, the posterior is computed as

$$P(h|d) \propto e^{-DL(h)} \cdot \mathbb{1}[h \text{ solves } d]$$

So maximizing the posterior is just minimizing the description length of correct programs. The learner should adopt the program which has the shortest description length of all programs that solve the data (that reproduce the trajectory). This is basically the theory theory (Section 1.4.3): the learner should prefer the simplest theory that accounts for the evidence. And the posterior is similar to the rational rules model, which has been shown to account for human rule-based concept learning (Chater & Vitányi, 2003; Goodman et al., 2008).

This equivalence between probability and code length is what the rest of the thesis measures things in, and it’s worth naming. Solomonoff’s theory of inductive inference is its ancestor: hypotheses are programs, and each gets prior weight falling exponentially in its length (Solomonoff, 1964). But that weighting is over programs for a universal machine, so the resulting prior is uncomputable and a learner cannot use it. What a learner can use is the same idea relativized to a particular language, which is the minimum description length principle: inductive inference as the search for the encoding that describes the data in the fewest bits (Grünwald, 2007; Rissanen, 1978). Because our likelihood is an indicator, the encoding of the data given the hypothesis costs nothing beyond the hypothesis itself, and MDL collapses into “shortest correct program”. It doesn’t stay collapsed. When Section 4.5.2 asks whether the learner should adopt a new primitive, the thing being encoded is no longer one trajectory but a whole corpus of solutions, and the code has two parts to pay for — the vocabulary and the corpus written in it — which is MDL in its usual two-part form. So the same principle stated here for a single hypothesis is what will later license changing the language. All quantities in what follows are reported in nats, i.e. description lengths under natural logarithms.

One qualification has to be entered straight away, since it shapes everything that follows. Description length is relative to the primitive vocabulary available, so the same hypothesis may be long under one grammar and short under another that happens to contain the right terms. Simplicity is therefore not a universal measure of a hypothesis’ naturalness: as stated so far it inherits whatever vocabulary the modeller chose, and a prior that can be tuned by choosing primitives is not much of a constraint. The classical reply is that this dependence is bounded — description lengths under any two universal languages differ by at most an additive constant, the length of an interpreter for one written in the other, so in the limit the choice of language washes out (Li & Vitányi, 2019). That reply is no comfort here. The invariance is asymptotic and the constant is arbitrary, whereas a learner with a finite search budget lives entirely inside the constant: as Section 2.3 will show, a term’s being a few nats cheaper is the difference between a hypothesis being found and never being reached at all. The vocabulary-dependence of description length is not a technicality to be

quotiented away, it’s the mechanism the thesis is about. Section 2.4 takes this up and argues that the vocabulary need not be stipulated either — that it can be found by the same inference, run one level up.

2.3 Program induction and the search problem

But it’s a specification of *what* the learner should end up with, not a procedure for getting there, and here we start to move from the computational level toward the algorithmic one (Marr, 2010; Tenenbaum et al., 2011). And as soon as we ask how h^* is to be found, the picture becomes much less comfortable.

The trouble is that this optimization is about as badly behaved as an optimization can be. The space is infinite, since a grammar with any recursion generates unboundedly many expressions, and the number of expressions of a given size grows exponentially in that size. There’s no gradient to follow, because programs are discrete: there’s no meaningful sense in which one is *slightly* perturbed into another. Worst of all, the binary likelihood gives no partial credit. A program that differs from a correct one by a single token receives exactly the same likelihood as a program picked at random, namely zero. So there’s no signal anywhere in the space to hill-climb on, and no way to tell that one is getting warm. The very feature that made the likelihood so clean in Section 2.1 is what makes the search blind.

What one can do, and what the algorithm this thesis uses does, is enumerate hypotheses in decreasing order of prior probability, checking each against the data until one fits. This is the right thing to do in the sense that it finds h^* , and it’s the only general strategy available given that nothing weaker than a full check tells us whether a candidate works. But the cost is the obvious one. Since $P(h) \propto e^{-DL(h)}$, enumerating every program of description length at most ℓ takes time on the order of e^ℓ . The time to find a solution is exponential in the description length of that solution under the current library.

This deserves emphasis, because it changes what the prior is doing. In Section 2.2 the description length was introduced as a way of *ranking* hypotheses: shorter means more probable, and among the programs that fit the data, the shortest wins. We can now see that it also determines which hypotheses are *reachable*. A hypothesis that is forty nats long under the learner’s current vocabulary is not merely improbable; it is not going to be found. The prior is simultaneously a measure of plausibility and a budget, and a concept’s being expressible in the language is no guarantee whatever that it’s learnable in the language. This is the move that resource-rational analysis makes general: a model of an agent with finite computation should treat the cost of thinking as part of the problem rather than as an implementation detail to be abstracted away (Lieder & Griffiths, 2020). Here we get that for free, because the quantity that prices a hypothesis and the quantity that prices the search for it are the same number.

That’s the pivot on which the rest of the thesis turns. If what a learner can acquire at a given moment is fixed by what’s short in its current vocabulary, and if the vocabulary changes as a result of what has already been acquired, then the ordering of acquisition is not something we have to stipulate. It’s a consequence. A concept becomes learnable at the point where earlier learning has made it short enough to be found, which is precisely the cumulative, staged picture that Section 1.3

demanding and that neither of the alternatives in Section 1.4 could deliver a mechanism for. The delay between goal attribution and false-belief attribution stops being a fact to be accommodated and becomes something the model can be asked to predict.

There are two ways to attack an exponential search of this kind. The first is to change the language so the target gets shorter: name a recurring piece of structure, add it to the stock of primitives, and every program using it becomes cheaper, so a program previously out of reach can fall within the budget. This is library learning (Section 2.4, Section 4.5.2). The second is to abandon enumeration in prior order for enumeration under a learned proposal distribution, trained to guess which primitives a task of this appearance is likely to need. This is amortized inference. The system this thesis builds uses only the first, for reasons given in Section 7.7.2; it is the first that does the work the argument needs, since it is the one that changes what is *reachable* rather than merely what is reached sooner. Neither makes the problem tractable in general, and neither is guaranteed to return h^* ; the claim is only that they make particular problems tractable that were not tractable before. Search over theories, treated in this way as a stochastic process rather than an exhaustive one, has been used to model theory change directly (Ullman et al., n.d.).

It's worth noticing that the nativist argument of Section 1.4.1 can be restated in these terms, and that doing so makes it stronger. The poverty-of-stimulus claim is usually put as a claim about data: there isn't enough of it to fix the target. But the sharper version is a claim about search: even granting the learner all the data it could want, and granting it the right hypothesis space, the target is exponentially far away, so something must be built in to put it within reach. We take this objection seriously, and the answer this thesis offers is not that the search is easy. It's that the distance is not fixed, because the language is not fixed. How a learner could come to change its own language is the subject of the next section.

2.4 Learning the language itself

Section 2.3 left us with a pragmatic reason to want the learner's language to change: the target is exponentially far away, and shortening it is the only way to bring it within reach. That's a reason of convenience, and taken by itself it's a little suspicious. If we're permitted to adjust the vocabulary whenever the search is going badly, then we can make anything learnable by adding a primitive that does the work, and the model explains nothing. What's needed is an account on which changing the language is not a device we apply from outside but something the learner does as part of the same inference it was already doing. This section gives that account, and the claim is that a prior can itself be learned, by exactly the machinery of Section 2.1 applied one level up.

2.4.1 The library as an overhypothesis

The move is to stop treating the prior as a fixed input. In the setup of Section 2.1 the learner has a hypothesis space $\mathcal{H}$ and a prior $P(h)$, and both are given. But a prior is a probability distribution, and a probability distribution can be parameterized, and a parameter can be given a distribution of its own and inferred from data like anything else. A hierarchical Bayesian model does this: it posits not a single level of hypotheses but several, with the hypotheses at each level constraining the space and the prior at the level below (Gelman, 1995). Bayesian inference is then run across all levels jointly. In Griffiths' phrase, we get "hypothesis spaces of hypothesis spaces, with priors on

priors” (Griffiths et al., 2024), and the effect is that the prior needed for a specific learning task can itself be learned, over a longer timescale, at the same time as it constrains the learning going on beneath it.

Knowledge at the upper level is *overhypothesis*, in Nelson Goodman’s sense (Goodman, 1983): it’s not a hypothesis about the data but a hypothesis about what the hypotheses about the data are like. A standard example shows the shape of it. In categorization, a learner can acquire the overhypothesis that categories of a certain ontological kind tend to be homogeneous in shape but variable in colour, and this abstract knowledge about feature variability then licenses generalizing a new category from a single example (Kemp et al., 2007; Kemp & Tenenbaum, 2008).

The upper level is there doing exactly what Section 2.3 showed a prior has to do: not merely ranking the lower-level hypotheses but cutting the space of them down to something a learner could search. That’s the reason hierarchy is not an optional refinement. For any cognitively realistic task it’s simply impossible to enumerate every logically possible hypothesis together with its prior and likelihood; the space is combinatorial (Tenenbaum et al., 2011). So a learner needs structural constraints that say what kind of thing an explanation could be before it can consider any particular explanation. Structure at the top makes search at the bottom feasible.

One reason to think the upper level is learnable at all is that it is fed by more evidence than the levels beneath it. A specific hypothesis is constrained only by the observations that bear on it directly, whereas an overhypothesis is constrained by every observation in every situation of the relevant kind; so the abstract level pools evidence across scenarios that are, from the point of view of the lower level, unrelated. A child who has watched many people pursue many different goals has seen very little evidence about what any one of them wants, but a great deal about the general shape of the relationship between wanting, seeing and doing. Where the upper level is a parameter with a handful of degrees of freedom, this pooling can be shown to make the abstract posterior concentrate faster than the specific one (Goodman et al., 2011; Kemp et al., 2007); where it is a library of λ -terms the space is discrete and unboundedly large, no such convergence result is available, and I claim none. What carries over is the asymmetry rather than the theorem, and it survives because the objective of Section 4.5.2 builds it in: an abstraction is worth adding only if it pays for itself across the whole corpus of solutions.

The example just given is an illustration of the framework rather than an instance of the model I’ll build: there the overhypothesis is a small parameter sitting above a lower level of ordinary hypotheses. Here the lower level is programs (Section 2.2), so the distribution to be learned is a distribution over the productions of a grammar, and the upper-level variable is that grammar itself: the stock of primitives, together with the weights on them. This specialization is not an analogy but a well-worked case of the framework, applied to syntactic rules (Perfors et al., 2011), to intuitive theories generated by a probabilistic grammar (Ullman et al., n.d.), and to program fragments transferred from solved problems to unsolved ones (Liang et al., n.d.). What DreamCoder adds is that the upper-level variable is a library of λ -terms, so that inferring the grammar means acquiring new *concepts* and not merely reweighting old ones (Ellis et al., 2020).

This also marks a shift in what kind of model we’re building. Early Bayesian models addressed the computational level only, characterizing cognition as approximately optimal statistical inference

in a fixed environment without saying how the computation was carried out. Hierarchical models sit between the computational and algorithmic levels, describing “cognition as approximately optimal inference in probabilistic models defined over a learner’s subjective and dynamically growing mental representations of the world’s structure, rather than some objective and fixed world statistics” (Tenenbaum et al., 2011). The representations grow. That’s the feature we need, and it’s what Section 4.5 will supply a concrete procedure for. It also blurs the tidy division of levels we started from: once the hypothesis space is itself something the learner builds and rebuilds, the question of what the learner is doing can no longer be cleanly separated from the question of what it is doing it *with*. That entanglement is not a defect of the account; it is the part of the account that does the developmental work.

2.4.2 Bootstrapping and minimal nativism

We can now state the position this thesis occupies. Learning gets off the ground by combining strong domain-general inferential and representational resources with minimal, skeletal domain-specific knowledge: “strong but domain-general inference and representational resources are aided by weaker, domain-specific perceptual input analyzers” (Goodman et al., 2011). This is minimal nativism, and it is a nativism — nobody is claiming the learner starts from nothing. The disagreement with Section 1.4.1 is over what has to be built in. On the modular view, what’s innate includes the attitude concepts themselves, a proprietary representational format for ascribing them, and the agent-directed inferential machinery that operates on it (Leslie, 1994). On the view taken here, what’s innate is a capacity to represent structured hypotheses, a preference for short ones, a procedure for searching, and enough perceptual apparatus to individuate objects and their motions. Everything specific to the mental is supposed to be constructed.

The construction is cumulative, which is what makes it bootstrapping in Carey’s sense (Carey, 2011; Piantadosi et al., 2012) rather than a single inference. An abstraction acquired at one stage becomes a term in the language in which the next stage’s hypotheses are written, and the cost of using it has already been paid. Applied to our domain, this predicts a particular relationship between the milestones of Section 1.2: theory of mind functions as the overhypothesis that structures particular attributions, and its components should become available in an order fixed by what each one is built out of. Goal-directedness is available early because it’s an abstraction over a great many observed pursuits. Belief is available late because it’s an abstraction over goal-directed pursuit together with perceptual access, and it cannot be assembled before its parts exist. The delay of Section 1.3, which Section 1.4.1 accommodates by maturation and Section 1.4.2 accommodates by frequency, here falls out of the compositional structure of what’s being learned. Whether it actually does so is an empirical question about a particular corpus and a particular library, and one this thesis puts to the test rather than assumes: Section 5.5 reports where in the run each capacity becomes available and in what order.

There is a second reply to Section 1.4.1 available here, independent of the delay. A learner acquiring abstract and specific knowledge simultaneously turns out to be almost as efficient as a learner endowed with a fixed and correct abstract theory from the start (Goodman et al., 2011). So the developmental evidence that abstract domain knowledge is in place early (Wellman & Gelman, 1992) does not by itself favour nativism, because that’s exactly the pattern a domain-general

hierarchical learner produces. Earliness is not evidence of innateness, and that inference is doing a great deal of work in the nativist argument.

What this section has not given is a procedure. Hierarchical Bayesian inference specifies what an ideal learner of abstract knowledge would conclude (Tenenbaum et al., 2006), and joint inference across levels is at least as intractable as the single-level search of Section 2.3 — worse, since the space of possible priors is larger than the space of hypotheses. So the situation at the end of this section mirrors the situation at the end of Section 2.1: we have a well-posed problem and a normative standard for its solution, and no way to reach it. What we need is an algorithm that approximates hierarchical Bayesian inference over the prior, in the specific case where the prior is a distribution over programs and the upper-level hypothesis is a library of primitives.

That algorithm is DreamCoder (Ellis et al., 2020), and I introduce it here only far enough to say what it is: the version this thesis runs is described in Section 4.5, and a reader who wants the full system will find it in (Ellis et al., 2020) rather than paraphrased twice here. It runs in rounds, alternating between the two levels of this section — solving a batch of tasks under a fixed language, then revising the language in light of what the solutions turn out to have in common. The alternation is not new with it: the same loop was proposed as a model of bootstrapped concept acquisition well before any neural component was added (Dechter et al., n.d.).

What DreamCoder contributes is a revision step principled enough to say when a new term is worth keeping, and that step is the answer to the objection this section raised against itself. The vocabulary is not adjusted by a modeller who finds the search going badly; it is adjusted by an objective, and the objective is the two-part code of Section 2.2 applied to the language rather than to a single hypothesis: the joint description length of the library together with the corpus written in it (Rissanen, 1978). Adding a term shortens every program that uses it and lengthens every program that does not, so a term is kept only if it pays for itself across the corpus as a whole. Section 4.5.2 states the objective and what it selects here.

Since I’ll lean on the word “approximates” for the rest of the thesis, it’s worth saying now in which respect the approximation falls short, because it isn’t only a matter of running out of time. Hierarchical Bayesian inference as described in this section maintains a posterior at every level, so an ideal learner holds a distribution over libraries and never commits to one. The algorithm does not do this. It maximizes rather than integrates, alternating between the two levels and carrying a single library forward from each round — a MAP point estimate of the upper-level variable — and it replaces the posterior over programs with a finite set of the best solutions found for each task. Nothing recovers a library discarded in an earlier round, so the procedure is a hill-climb through the space of languages and can settle on a local optimum that full inference would have avoided. This is the usual price of making hierarchical inference computable and I’ll not treat it as a defect, but it does bound the claims available later: when Section 5.2 reports that a given abstraction was found, the finding is that this search under this corpus arrived at it, not that it’s the posterior-optimal library.

What the loop buys, for the argument of this thesis, is the thing Section 1.3 asked for and Section 1.4 found missing: a mechanism by which a learner constructs new conceptual terms, adopts them on evidential grounds, and thereby brings within reach concepts that were unreachable before.

That is what the theory theory was accused of lacking in Section 1.4.3 — concepts constructed out of earlier concepts, by a procedure that decides what to construct on grounds of compression alone, with the order of construction scheduled by nobody but falling out of what is cheap at each stage. And because reachability is fixed by description length under the current library, while the library changes as a function of what has already been learned, the *order* in which concepts become available is a consequence of the mechanism rather than a stipulation added to it. The developmental ordering of Section 1.2 becomes something a model of this kind can be asked to predict rather than to accommodate (Palmarini et al., 2024) — which is what the rest of the thesis puts to the test.

2.5 What is claimed, and what is not

Fixing the level fixes the scope of the claim, and it is worth being explicit about what falls outside it. We are not studying how a child in fact comes to learn theory of mind. That is the province of developmental and cognitive psychology, and nothing here should be read as a proposal about neural implementation or about the moment-to-moment course of a particular child’s development.

Nor is the claim that children run the algorithm just named. ECD, the library-learning system described and implemented in Chapter 4, is offered as an algorithmic-level *realisation* of the computational-level claim: a demonstration that the problem, so stated, is solvable by a mechanism that begins with no mental vocabulary at all. Its role in the argument is one of sufficiency rather than fidelity. If a learner equipped only with domain-general primitives and a preference for short programs comes to posit belief-like structure, then positing such structure is something a learner *can* do rather than something it must be given — and the nativist’s inference from “no mechanism has been named” to “no mechanism exists” is blocked. Whether human learners use this mechanism, or some other one that solves the same problem, is a further question that this thesis does not settle.

Chapter 3

Bayesian Theory of Mind

In this chapter we first claim that Bayesian Theory of Mind (BToM) is the best available computational account of the mature ToM capacity described in Chapter 1. BToM formalises the intentional stance as inverse planning, and its posteriors quantitatively track human judgements. We then claim that BToM gets there by stipulation. The structure that makes belief attribution possible — an agent decomposed into a goal and a world model, a planner evaluated against a model that may diverge from the world — is written into the architecture rather than inferred from any scene. Section 3.4 draws up that inventory, and it is the inventory this thesis asks a learner to construct for itself.

3.1 Agent as planner

To adopt the intentional stance towards a system is to treat it as a rational agent. First attribute to it the beliefs and desires it ought to have (given its situation and history), then predict its behavior by working out what a rational agent would do (Dennett, 2002). People adopt the intentional stance even towards two triangles and a circle moving about a rectangle, seeing them as chasing, hiding, bullying, and escaping each other (Heider & Simmel, 1944). The intentional stance is a strategy that gets used because it regularly yields predictive success (CITE dennet again).

Bayesian Theory of Mind (BToM) gives a computational account of this strategy, modeling the agent as a planner. The agent occupies one of a set of states, has available in each state a set of actions, and the actions move it between states according to a transition function. The planner takes as input a utility function and a model of the environment, both defined over the agent’s state space, and returns the actions the agent should perform to maximize utility (Baker et al., 2009; Lake et al., 2016).

Research shows even twelve-month-olds encode an action as directed at a goal state, and expect an agent to use the most efficient means available to it. In one experiment, infants were habituated to a small circle jumping over a barrier to reach a large circle. When the barrier was removed and the circle kept jumping anyway rather than taking the newly available straight path, the infants were surprised (Gergely et al., 1995). Gergely and Csibra call this the teleological stance, which they distinguish from the intentional stance (Gergely & Csibra, 2003). It relates an action, a goal state, and the constraints of the situation but without attributing any representation to agents. Under the teleological stance, the reality constraints that the observer consults are their own, not the agent’s.

We return to this distinction in Section 4.6.5, when we defend our choice to hand our model learner a primitive planner `optimize`, which moves a value one step along the best available path toward improving some utility, even though the planner is algorithmically complex. From the developmental literature we know that efficient goal-directed interpretation is in place around a year

of age, three years before anything recognizable as false-belief attribution (Section 1.2). And on Gergely and Csibra’s analysis it isn’t yet ToM at all, it just says how a body moves given a goal and a layout.

3.2 Inverse planning

A planning agent computes in the forward direction, from a goal and a model of the environment to the behaviour. An observer needs to compute in the backward direction, from the visible behavior to the imputed goal and model (belief). The observer sees the trajectory (the agent’s sequence of states), and needs to compute the goal and belief that produced it. By Bayesian inverse planning, the observer can simulate what the agent would have done had it wanted this and believed that, and can then verify the hypothesis by comparing each simulation against what the agent in fact did. The process is analogous to modeling intuitive physics by running a physics engine forward under different settings and scoring the outcomes (Lake et al., 2016).

The learner compares hypotheses by Bayes’ rule (Section 2.1). Writing g for the agent’s goal, m for its model of the environment, and t for the observed trajectory, the posterior is calculated as

$$P(g, m \mid t) \propto P(t \mid g, m) \cdot P(g, m)$$

where the prior $P(g, m)$ encodes what the observer expects agents to want and think before seeing anything, and the likelihood $P(t \mid g, m)$ encodes the probability that an approximately rational agent with goal g and model m would move as observed (which is supplied by the planner) (Baker et al., 2009). It’s been shown experimentally that people’s predictions on behavioral stimuli are close to the model posteriors of this approximately rational inference mechanism (Baker et al., 2009).

What matters below is the second member of the pair, the agent’s model m , and the fact that the planner receives it as an input.¹⁰

3.3 Belief as an input

The posterior is joint over (g, m) , the agent’s goal g and its model of the environment m . But m doesn’t have to be an accurate model of the actual state of affairs (the observer’s world), it could be any world modeling. The observer can supply a model m' that diverges from the actual state of affairs, and then run the same posterior computation to see how the agent would behave if it were navigating in the counterfactual world m' . Then the same computation that yields sensible behavior from an accurate model would yield mistaken behavior from the inaccurate one.

This is belief attribution in essence, attributing a counterfactual world to an agent such that it acts according to the counterfactual model as its own private world. So we can ask: given an (irrational) trajectory, which model of the environment would’ve made those actions rational? Take the false-belief task (Section 1.2): Sally puts her marble in the basket and leaves, in her absence it’s moved to the box, and when she returns she searches the basket (Wimmer & Perner, n.d.). Evaluated against the actual (observer’s) world m , Sally’s search is irrational. But evaluated against her private model she had when she left m' , her behavior is optimal. An observer that can represent

¹⁰The inversion generalizes beyond (g, m) : assuming agents act to maximize reward, an observer can run it in whichever direction the situation leaves open, recovering an agent’s utilities or the costs its actions carried as readily as its goals (Jara-Ettinger, 2019; Jara-Ettinger et al., 2016). That extension is not the one this thesis follows up.

the divergence of worlds can predict that Sally will look in the wrong location, and even which wrong location she'll look in (Goodman et al., n.d.).

To handle divergent models, the observer needs to run the planning computation inside itself: once against its own model m , to know what is actually the case, and once against the model m' it attributes, to work out what the agent will do. This kind of nested computation is another reason that representing hypotheses as programs (Section 2.2) is especially fitting for the ToM domain (Stuhlmüller & Goodman, 2014). Regardless of whether a program takes m or m' as input, it's executed and evaluated in the same way. To attribute a mental state is to say something about the process by which an agent selects its actions, and a program is the natural way to write down that procedure.

3.4 What is stipulated in BToM

BToM is a model of the ToM capacity of a mature adult (Baker et al., 2009; 2017). In this thesis we're interested in how that capacity is acquired, specifically whether a system without a hardcoded BToM can develop something akin to it. So when we introduce our model we'll need to show that our learner doesn't have a built-in BToM. To argue that, we first have to be precise about which features aren't given by the data but are stipulated by the framework. There are five, and Section 4.6 answers them in this order.

The decomposition into agent, goal and belief. A scene depicting entities moving around doesn't inherently encode them as agents and goals, and it doesn't inherently encode their behavior as generated by two attitudes rather than one or five. But BToM decomposes the scene into an agent that has a goal and a belief, which the planner takes as separate arguments that vary and can be inferred independently of each other. The decomposition is in the architecture, not in the scene.

The separation of the believed world from the actual world. The planner is evaluated against a world model m' supplied as one of its inputs, and that input is permitted to diverge from the state of affairs the observer takes to obtain, $m \neq m'$. This is the stipulation that makes false belief representable at all, and it too is made in the architecture rather than inferred from any scene.

The space of admissible divergences. Given that m and m' may diverge, something has to specify which divergences are admissible. What's inferred under BToM is which particular world model m' the agent holds, but the range it's drawn from is fixed in advance, along with the scenarios that produce it — an occlusion, an absence while the world was rearranged. That a belief could be false, and how it came to be, is never itself in question.

The space of goals. Likewise on the other input. The posterior over goals is inferred, but the space it ranges over is given, and so is the prior on it. Baker et al. compare several goal priors precisely because the choice matters and is the modeller's to make (Baker et al., 2009). What inference does is locate a point inside a space someone else drew.

Rationality. That the agent plans approximately optimally toward its goal given its model. This is the one item on the list with independent developmental support, since the teleological stance is in place well before the first birthday (Section 3.1, Section 1.2), and it is correspondingly the one item this thesis also grants its learner.

The consequence is that inference in these models is parameter estimation within a fixed theory rather than construction of the theory's terms. That is exactly the distinction drawn in Section 1.3: the underdetermination problem, to which Bayesian inversion is the answer, presupposes a learner who already possesses the concept *believes* and has only to choose among ascriptions. A learner who lacks the concept cannot make an ascription at all, and no amount of inverting a planner will supply it, because the planner's shape is what encodes it.

The same fixity leaves the developmental ordering of Section 1.2 outside the model's reach. Since the structure is complete from the start, every one of its parameters is available for inference on day one. There is nothing in the framework that makes a goal attribution cheaper, or earlier, or prior to a belief attribution, so there is nothing in it that predicts why the first is reliable before the first birthday and the second not until around four. The delay has to be explained by something the model doesn't contain — maturation, or performance demands masking an underlying competence — which is the same appeal to auxiliary hypotheses that Section 1.4.1 was charged with in Section 1.4, and it's available here for the same reason: the structure that would have to develop is assumed rather than derived.

Written out as a list, in fact, the stipulations are close to an inventory of what the modularity-nativist claims is innate. Attitude concepts introduced as such, a proprietary format in which they are ascribed to agents, and inferential machinery that operates on agents in particular (Leslie, 1994) — the difference is that BToM says it in probability theory and says it precisely. So the nativist can read the framework's success as evidence *for* the position of Section 1.4.1 rather than against it: here is the endowment, written down, and here is how closely a system so endowed matches human judgement. The reading isn't perverse, and it's what the models license, given that they make no claim at all about where the endowment comes from. What they establish is that a learner who ends up with this structure would attribute mental states as people do. Where such a learner could get the structure is a question they leave open, and it's the question of the next section.

3.5 What this thesis asks

The five items of Section 3.4 have a shape we already have a name for. They are not hypotheses about any particular scene; they are hypotheses about what the hypotheses about scenes are like — that behaviour is generated by an agent, that the agent has a goal and a model, that the model may diverge from the world, that the divergence is drawn from such-and-such a range. That is an overhypothesis in the sense of Section 2.4, sitting one level above the attributions it structures. BToM installs it and infers underneath it. The question of this thesis is whether it can be inferred rather than installed: whether the upper-level variable, given to the framework by its modeller, is the kind of thing a learner could arrive at by the same inference it runs on everything else.

Putting it that way fixes what the model of the next chapter has to do. It cannot be handed any of the five. Its learner begins with grids, a planner, and domain-general combinators for building and consuming pairs — no agent, no goal, no attitude, no channel that is anybody's — and it has to assemble the structure out of them, in program space, under a description-length objective that knows nothing about which tasks are mental. This raises the bar past expressiveness. A language in which belief is merely *statable* proves nothing, since the stipulated architecture is *statable* in

any language rich enough to write it down. What has to be shown is that the structure is *selected*: that among the programs which reproduce the scenes, the one carrying the agency signature is the one compression keeps, and that it beats the non-mental rivals the very same library can express. Section 4.7 states that test in advance and Section 4.6 takes the five stipulations in the order given here.

Two things are worth conceding before the model is on the table. The first is that one item of the five is granted rather than earned. Rationality comes in as the **optimize** primitive, on the grounds Section 3.1 gave: the teleological stance is in place before the first birthday, three years before the capacity whose acquisition is being modelled, and on the best analysis of it it attributes no representation to anybody. Section 4.6.5 defends the choice at length and does not pretend it is free.

The second concession is about what is being asked for. BToM's m' is a belief in the full sense: a state the agent came to be in, that could have been otherwise, and that would be revised if the agent looked again. What the learner here is asked to find is thinner — a divergent world model, attributed to an agent and acted on by that agent's policy, but with no account of how the agent came to hold it and no machinery for updating it. Nothing in a single-frame transition function represents perceptual access, or a record of what was witnessed, or an update on being shown the truth. That is a real gap between the discovered structure and the thing it models, and it is the most important one; Section 6.4 takes it up as further work.

Chapter 4.

The Model

4.1. The learning problem

In our model the learner is an observer. It is shown scenes, which are visual episodes that unfold over discrete time steps on a small grid. For each set of associated scenes the learner must recover the underlying process that generated what it saw, by positing candidate hypotheses until it finds one that works. Hypotheses are programs, so a hypothesis is correct only when executing it reproduces the entire sequence of frames it was given. Concepts are primitives, so a concept is learned when the learner enriches its library of primitives with a new abstraction. We ask whether such a learner, shown scenes whose shortest explanation requires belief attribution, comes to learn the attribution structure as a new abstraction.

4.2. The task corpus

We created a corpus of sets of grid-based tasks. Most tasks are *trajectory tasks*. A trajectory is a sequence of 2d grids over the time dimension, that depicts some dynamic at play (e.g. a body that falls, an agent that approaches something, etc.). A task consists of k trajectories that all depict the same dynamic but vary in setup (where the entities start, how long the scene takes, what else is lying on the grid, etc). Given a task, the learner’s goal is to find a transition function that, for any trajectory in the task, returns the next grid for any grid in the sequence. So the transition function should be a generalized explanation of the underlying dynamic.

For example, the task below consists of four scenes that each depict the value 4 moving up. The grids of each timestep are collapsed into one, so each arrow indicates one timestep forward. A solution to the below task is a program that encodes “at each time step value 4 moves one step up”. Such a program solves the task because for each scene, given that scene’s starting positions, the program reproduces the rest of the trajectory.

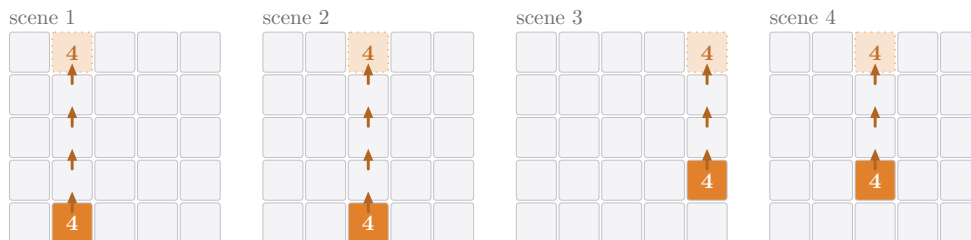


Figure 1: an instance of a directional movement task

The following task is solved by a program that encodes “at each time step, value 1 moves one step closer to value 2”.

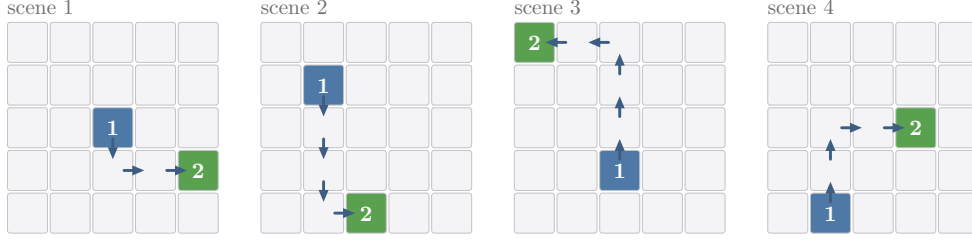


Figure 2: an instance of a goal-seeking task

Both of these, and most of the corpus, can be solved by programs that describe the scene extensionally via a purely behaviourist visual description. The task families we care about are those that can only be solved by attributing a belief to an agent.

Below is an instance of a *goal-displacement* task, inspired by the false-belief task (Wimmer & Perner, n.d.). The dark blue value (1) comes to rest on a cell a fixed step away from the green value (2), rather than on it (and the teal 7 is a witness value). A solution to this task is a program that encodes “at each time step value 1 moves one step closer to value 2, given that value 2 is two steps above its actual location”. So solving this task requires attributing to an agent a model that deviates from the actual one.

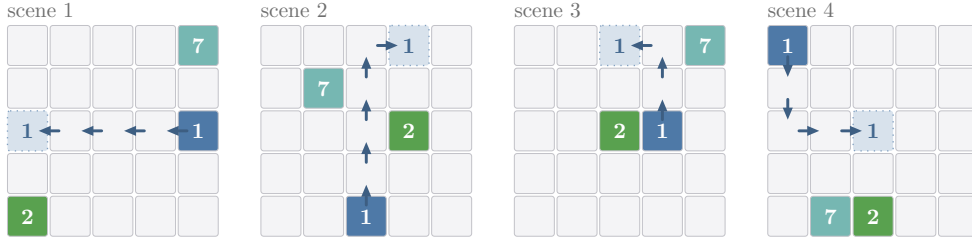
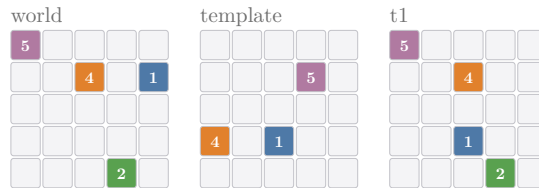


Figure 3: An instance of a goal displacement task

A second, smaller group are *template tasks*. Where a trajectory task’s input is a single initial grid, a template task’s input is a pair of grids: a *working* grid and a *template* grid, both $s \times s$. The task is to produce the working grid modified according to the pattern given in the template. This format is loosely inspired by the ARC-AGI-1 corpus [CITE].



We designed our corpus to exercise a variety of domains. See Table 10 in the appendix for the full list of task families in the corpus.

4.3. The hypothesis space

4.3.1. Programs as typed expression trees

A hypothesis is an expression tree over the library of primitives. Each node carries a type together with the types of the arguments it expects. A node with no arguments is a terminal whose value is itself, like **up** of type D or the literal **4** of type V ; a node with arguments is an operator, like **step** of type $V \times D \rightarrow (G \rightarrow G)$, applied to its filled-in arguments. The tree is directly executable, and evaluating it bottom-up yields the object its root type names. So **(step 4 up)** evaluates its two

leaves to the cell value 4 and the direction $(-1, 0)$ and applies **step** to obtain a transition function in $G \rightarrow G$.

A program’s syntactic size is its node count: one for a leaf, one plus its arguments for an operator. An abstraction used as a token adds just one, because the subtree it abbreviates lives inside the body and does not appear in the program that uses it. An abstraction converts many charged nodes into one. (See Section 7.4 for the representation that implements expression trees).

4.3.2. Type system

G is the type of 2d grids of cell values, the frames of a scene. Write G^* for stacks of grids, i.e. a full scene composed of a sequence of grids. So a transition function is then an element of $G \rightarrow G$. The cell values are members of the ground set $V = \{0, \dots, 9\}$, where each is an entity on the grid (e.g. the value 4 moving up each time step). The grid coordinates are $C = \{0, \dots, s-1\}$, a row or column index, from which a *position* in $C \times C$ is built. The remaining sets are D , the four movement directions, and $U \subseteq (C \times C \rightarrow \mathbb{R})$, the positional utilities that the **optimize** primitive (Section 4.4) scores.

Everything else the language names is an arrow between these. The arrows relevant to our argument are the ones into and out of the product $G \times G$, which serves as the (world, model) channel pair:

$G \rightarrow G \times G$	build a pair from a grid
$G \times G \rightarrow G \times G$	transform a pair
$G \times G \rightarrow G$	collapse a pair back to a grid
$G \rightarrow C \times C$	read a position off a grid
$G \times (C \times C) \rightarrow G$	impose a position on a grid

So there are four jobs to be done with a pair: building a pair, transforming a pair, collapsing a pair, and reading and writing the positions that give a collapse something to go off of. These operations are performed by the product primitives (Section 4.4).

Whether a program opens a private second channel is a matter of which of these arrows its nodes inhabit, so a (world, model) representation is a structural property of the program. The fixed product does carry the commitment that a learner holding a private representation holds exactly one (generalizing this is left as further work, see Section 6.3).

A program is checked by running it through an interpreter (Section 7.2). The solution to a trajectory task (Section 4.2) is a program of type $G \rightarrow G$, a transition function that given the task’s initial grid at step i should return the next grid at step $i + 1$. So the interpreter iterates the program $G \rightarrow G$ on the task’s initial grid G , yielding the stack of grids G^* which can be compared to the target scene. So the interpreter’s state at any step is just a single grid G , not a (world, model) pair $G \times G$, and certainly not a registry of agents, goals, or entities. No pair is handed from one step to the next: a program that opens a second channel (a model channel that may diverge from the world) must build it, use it, and collapse it again inside that step.

A template task starts from a pair in $G \times G$, a working grid and a constant external template, so its solution is a *commit* $G \times G \rightarrow G$ (which is one of the collapsing arrows above). The commit answers what each frame should do with the template. In a template task the pair is given rather

than built by the program, which is the one case in which a program is handed a second channel it did not open itself. Since a type-directed enumeration yields programs of one type only, trajectory tasks ($G \rightarrow G$ -rooted) and template tasks ($G \times G \rightarrow G$ -rooted) are searched separately (Section 4.5.1). Though compression pools the solutions of both (Section 4.5.2).

4.4. The primitive library

The learner starts with a library of primitive operations, listed in full in Section 7.3.

Some of the primitives are for assembling transition functions, e.g. **step**, of type $V \times D \rightarrow (G \rightarrow G)$, moves every cell of a given value one step in a given direction. **optimize**, of type $U \times V \rightarrow (G \rightarrow G)$, moves a value one step towards or away from a value according to the given utility. There are two utilities to give it, both of type $V \rightarrow U$: **neg_dist**, minus the distance to the nearest cell of a target value, and **distance**, plus that distance, so that maximising the first approaches a target and maximising the second flees it. Three primitives edit the grid outright: **wall_at** stamps a wall value (3) at a position, **clear_at** clears a single cell, both are of type $C \times C \rightarrow (G \rightarrow G)$. And **erase** is of type $V \rightarrow (G \rightarrow G)$, it removes every cell of a value. Finally **compose** runs one transition function then another, so it's of type $(G \rightarrow G) \rightarrow (G \rightarrow G) \rightarrow (G \rightarrow G)$. The terminals are the four directions, the cell values 0...9 of type V , and the coordinates **c0** ... **c4** of type C , a pair of which names a position.

Since we're interested in how transition functions $G \rightarrow G$ interact with (world, model) pairs $G \times G$, the rest of our primitives are the structure of the product. They are defined for grids $w, m, g \in G$, transition functions $f, f' \in G \rightarrow G$, values $v \in V$ and positions $p \in C \times C$. There are four kinds, one for each arrow of Section 4.3.2: those that build a pair out of a grid, those that transform a pair, those that collapse a pair back to a grid, and those that read and write a position.

We have two ways of *building* a pair, both of type $G \rightarrow G \times G$. The diagonal $\Delta(w) = (w, w)$, which the library calls **dup**, copies a grid into both channels. Its complement **pair_blank**, $w \mapsto (w, \mathbf{0})$, pairs the grid with an empty one instead, so the second channel starts as scratch rather than as a copy of the world.

We have four ways of *transforming* a pair, all of type $G \times G \rightarrow G \times G$. Three are the functorial actions, which lift transition functions to act on the channels: **mapsnd** acts on the second factor, $(\text{id} \times f)(w, m) = (w, f(m))$, **mapfst** on the first, $(f \times \text{id})(w, m) = (f(w), m)$, and **bimap** on both at once with one function for each, $(f \times f')(w, m) = (f(w), f'(m))$. The fourth is the twist $(w, m) \mapsto (m, w)$, **swap**, which exchanges the channels, so anything defined on one factor can be conjugated into a version acting on the other.

We have several ways of *collapsing* a pair to a single grid, of type $G \times G \rightarrow G$. The projections **fst** and **snd** return one channel and discard the other, $(w, m) \mapsto w$ and $(w, m) \mapsto m$. **overlay** and **underlay** union the two channels, differing only in which one wins where they disagree. **sync_all** moves every value the two channels share to the position it holds in the second, and **sync_except v** does the same for every shared value but v .

Finally we have the accessors on positions. **locate**, of type $V \rightarrow (G \rightarrow C \times C)$, gives $\text{loc}_v(g)$, the position of v in g ; **place**, of type $V \rightarrow (G \times (C \times C) \rightarrow G)$, gives $\text{put}_v(g, p)$, the grid g with v moved to p . A reader and a writer are joined by **register**, of type $(G \rightarrow C \times C) \times (G \times (C \times C$

$C) \rightarrow G) \rightarrow (G \times G \rightarrow G)$, which reads a position off the second channel and imposes it on the first: $\text{register}(\text{loc}, \text{put})(w, m) = \text{put}(w, \text{loc}(m))$. So `register (locate v) (place v)` is the collapse that moves the single value v to wherever the second channel puts it, and it is a collapse the learner has to assemble from three parts rather than one it is handed.

Each of these arrows needs its own composer, since `compose` joins two transition functions only, it's of type $(G \rightarrow G) \rightarrow (G \rightarrow G) \rightarrow (G \rightarrow G)$. So `compose_gp`, of type $(G \rightarrow G \times G) \times (G \times G \rightarrow G \times G) \rightarrow (G \rightarrow G \times G)$, extends a builder by a transform. `pipe_gpg`, of type $(G \rightarrow G \times G) \times (G \times G \rightarrow G \rightarrow G) \rightarrow (G \rightarrow G)$, follows a builder with a collapse, which is the composer that returns to $G \rightarrow G$. The interpreter only ever iterates a transition function, so a program that opens a second channel must close it again within the same time step. `compose_pg`, of type $(G \times G \rightarrow G \times G) \times (G \times G \rightarrow G) \rightarrow (G \times G \rightarrow G)$, follows a transform with a collapse, and is the only one that acts on a pair the program did not itself build, which is what a template task's given pair calls for.

The primitives are just the product structure, so nothing in them distinguishes a world from a model. The pair is symmetric: both factors are grids, every structure map is present in both directions, and no primitive marks either channel as anyone's representation. Which channel is the world and which is a belief is settled only by how a program assembles these pieces together, which is found by search.

4.4.1. The target compound

Chapter 3 formulates belief attribution as running the planner on a model that may diverge from the observer's. The observer runs the same planning computation twice, once against the world as it is and once against the model it attributes to the agent (Section 3.3). So in our setting, the observer needs to do three things: first *derive* a private grid that differs from the world in whatever the agent is wrong about. Then *run* the agent's movement policy in the private grid, so that its move is optimal with respect to its own belief rather than to the real world. Finally *commit* the result back to the world, publishing where the agent went and nothing else. The commit is necessary because the interpreter renders one grid per step, so motion on a private grid doesn't affect the rendered grid unless an agent's motion is explicitly committed from it.

Only the *run* step is done by a primitive, namely `optimize` which builds the agent's movement policy. Nothing in the library opens a private grid, and nothing publishes a single value's position back out of one. Both have to be assembled from the product structure above. So for *derive* and *commit*, we have two target structures:

$$\begin{aligned} \text{fork}(\text{derive}, \text{commit}) &= \text{commit} \circ (\text{id} \times \text{derive}) \circ \Delta \\ \text{sync}_v &= \text{put}_v \circ (\text{id} \times \text{loc}_v) \end{aligned}$$

`fork` is the derive-and-commit frame, taking a $\text{derive} \in G \rightarrow G$ and a $\text{commit} \in G \times G \rightarrow G$. It copies the world into a pair, applies the `derive` to the second channel to get the private grid, and hands the pair to the `commit`, which collapses it back to one grid; the private grid exists only for the duration of that call. sync_v is the commit belief needs: it reads where v sits in the private grid and moves the v in the world to that position, so exactly one value's position is published and everything else stands as the world had it.

Belief is the composition of fork and sync, with the derive doing the agent’s error and the commit doing the attribution. Taking a phantom wall as the error, it is $\text{fork}(\pi_a \circ \text{wall}_{r,c}, \text{sync}_a)$, which unfolds to the map

$$w \mapsto \text{put}_a(w, \text{loc}_a(\pi_a(\text{wall}_{r,c}(w))))$$

for an agent value a , a seek policy π_a , and a wall stamped at row r and column c , the three steps in order are: copy the world and put a wall on that copy, plan based on the copy of the world which has a wall in it, then finally write to the rendered world only where the agent ended up.

Note that the agent value a occupies three slots in that term: the movement policy π_a that plans over the privately walled grid, the read loc_a that finds where the agent ended up, and the write put_a that publishes it. So to find this compound, the enumerator needs to independently fill the argument slots of the three primitives with the same value placeholder.

4.5. Inference

Our procedure runs in alternating wake-sleep rounds, following the DreamCoder (Ellis et al., 2020). The waking phase enumerates programs for the tasks not yet solved, under the current library. The sleep phase revises the library to compress the programs found. We depart from (Ellis et al., 2020) in omitting the recognition model, the dreaming phase (see Section 7.7.2). Also, in our model the likelihood is evaluated against k observations at once rather than one (Section 4.2). And the abstraction search is delegated to Stitch rather than run bottom-up (Section 4.5.2). The two phases divide the inference of Chapter 2 between them: waking performs the per-task inference of Section 2.1, sleeping the revision of the prior that Section 2.4 placed one level above it.

4.5.1. Search

The waking phase solves the inverse problem of Section 2.1 in the only way Section 2.3 left available, by enumerating candidates in decreasing order of prior probability and checking each against the data until one fits. A hypothesis h , in the sense of Section 2.2, is here a typed expression tree over the current library (Section 4.3). Since enumeration in decreasing prior probability is enumeration in increasing description length, the search walks outward in expanding bands of cost — the concrete form taken by the $\sim e^\ell$ of Section 2.3 — and the first program it finds that reproduces the data is the shortest one that does, which is the maximum a posteriori hypothesis h^* exactly as Section 2.2 defined it. The approximation is not in what is returned but in when the search is abandoned: enumeration runs under a time budget, and a hypothesis whose cost band is not reached before the budget expires is simply not found.

The library fixes the prior. The $P(h) \propto e^{-\text{DL}(h)}$ of Section 2.2 is realised as a distribution over the library’s symbols, under which a program’s cost is the summed $-\log p$ of its nodes. Symbols are grouped by return type, and a symbol of type τ is priced at $-\ln$ of the number of symbols sharing τ , i.e. a type-uniform distribution. So the types give the branching factor at each argument slot, and the cost of each choice at that slot. Enumeration is type-directed in the same sense: an argument slot draws only from symbols whose type matches, so no ill-typed tree is ever built.

A worked example fixes the scale, since every cost quoted in Chapter 5 is in these units. Suppose the task depicts the value 4 rising in each successive grid, so the target is (step 4 up). There are

four symbols of type `dir`, so each gets $p = 1/4$ and $\text{cost}(\text{up}) = -\ln 1/4 = \ln 4 \approx 1.39$ nats. By the same reasoning `step` costs $\ln 8 \approx 2.1$ nats and `4` costs $\ln 10 \approx 2.3$. Summing gives ≈ 5.8 nats for the whole program.

One exemption to that pricing is worth stating. A cell value *visible* in the task’s own first frame costs nothing, since naming what is already on display is not a hypothesis about anything, while a position stays a latent the learner must pay for. The invisible phantom wall is thus something search has to buy, and the agent and goal identifiers, being in plain sight, are not.

The budget is what makes the double role of the prior, argued for abstractly at the end of Section 2.3, into something concrete. There the point was that a long hypothesis is not merely improbable but unreachable; here that is a fact about a clock. The two readings of description length cannot come apart, because in this procedure making a hypothesis more probable and making it cheaper to find are the same operation. And that points at the only remedy available: not to enlarge the budget, but to shorten the target, which is what Section 4.5.2 does.

Two notions of description length are in play, and it is worth keeping them apart. The *cost* of a program, in nats, is the summed $-\log$ probability of its nodes under the current prior; this is what orders enumeration, and it is what Section 2.2 identified with $-\log P(h)$. The *total* description length is the two-part quantity of Section 4.5.2: the length of the library itself plus the summed cost of every solved program written in it. The first prices one hypothesis against a language held fixed, and is what the waking phase minimises; the second prices a language together with everything solved in it, and is what the sleep phase minimises. Both are reported in nats, and Chapter 5 uses both — the margins between a belief compound and its rivals are costs in the first sense, the round-by-round curves of Section 5.4 are totals in the second.

Two details are specific to this setting. First, the success criterion, which is where the indicator likelihood $1[h \text{ solves } d]$ of Section 2.1 is cashed out as an execution: a candidate solves a task only if the interpreter, started from each scene’s own first frame, reproduces that scene’s every frame exactly, for all k scenes (Section 4.2). Second, each still-unsolved task is enumerated as its own cost-ordered stream rather than checked against one stream shared across the corpus, which is what lets the search be parallelized across tasks. The mechanics of cost-bounded search, including the pruning and the tie-break within a cost window, are given in Section 7.6.

4.5.2. Library revision

The sleep phase does the shortening, and it is here that the hierarchical model of Section 2.4 becomes an algorithm. Having solved some tasks, the learner looks for structure recurring across the solutions and gives that structure a name, adding it to the library of primitives from which future programs are built. The library is the overhypothesis of Section 2.4 made concrete: a set of named, reusable abstractions which, by fixing what symbols exist and how the probability mass is divided among them, is the learned prior under which the next round’s tasks are inferred. This is λ -abstraction used as Section 2.2 advertised it: taking a recurring piece of structure and giving it a name, so that the learner comes to possess a term whose content is fixed entirely by what it does in combination with the others.

The reason this is inference rather than convenience is the objection Section 2.4 raised against itself. If the modeller may adjust the vocabulary whenever the search is going badly, then anything can be made learnable by adding a primitive that does the work, and the model explains nothing. The answer is that the adjustment is made by an objective rather than by a modeller, and the objective is the two-part code of Section 2.2, now applied to the language rather than to a single hypothesis: the joint description length of the library together with the corpus written in it (Rissanen, 1978):

$$\min_{\text{library}} \left(\text{DL}(\text{library}) + \sum_{\text{tasks}} \text{DL}(\text{program} \mid \text{library}) \right)$$

Minimising this is the joint inference across levels of Section 2.4, in the plainest possible form: the vocabulary and the programs written in it are settled together, against a single criterion, rather than the vocabulary being fixed first and the programs then found within it.

Both terms matter, and the tension between them is what makes the choice non-trivial. Naming a recurring fragment collapses it, at each site where it is used, from a subtree to a single token, which shrinks the second term. But the library is now larger, and since a symbol’s probability falls as the number of symbols competing with it rises, every program written in the enlarged library gets a little more expensive, which grows the first. An abstraction is kept only when the tokens it saves across the whole corpus outweigh the price of carrying it. Structure that recurs by accident is not abstracted; structure that recurs because it is genuinely shared is. The constraint doing the work is conservation of probability mass, now operating on the language rather than on the data: a library that could express anything cheaply would be a library in which nothing is cheap.

The objective is indifferent to which tasks the shared structure came from. It sees a pooled corpus of solutions and asks what would compress it, so an abstraction serving one family of problems must win against every abstraction that would serve any other family. Two things about how it is optimised here matter to the argument.

The first is the search procedure. Where DreamCoder looks for abstractions bottom-up, enumerating refactorings of each found program and intersecting them, we delegate the search to the top-down Stitch (Bowers et al., 2023) library-learning system, which searches the space of abstractions directly and so never has to materialize every rewrite. This is a difference in tractability rather than in objective; the procedure and its utility bound are given in Section 7.7.

The second is the scope over which it runs. Search is divided by solution type (roots), but compression is **joint**: it pools the solutions of both types and revises one library over all of them, so an abstraction has to earn its place against the whole solved corpus rather than against its own share of it (Section 4.6.7).

What compression returns is a set of abstractions together with the corpus rewritten to use them. Each abstraction is registered as a new symbol, on the same footing as a primitive that was given — what was a subtree found in round N is an atom in round $N + 1$ — and the rewritten corpus becomes the starting point for the next round — in which programs a level deeper, belief compounds too expensive to enumerate from bare primitives, have become reachable because their shared core now costs a single token. This is the cumulative structure Section 2.4 called bootstrapping: each round changes what the next round can find.

4.6. The model does not encode a Bayesian theory of mind

Chapter 3 argued that BToM is the best available computational account of mature theory of mind, and then listed what it stipulates in order to get there. Since this thesis is asking whether that structure can be acquired, the obvious charge against the model is that we have re-encoded the same stipulations in a different notation and then congratulated the learner for finding them. The argument of Section 4.1 is only as strong as the answer to that charge. If any part of the apparatus — the interpreter, the type system, the primitive library, the corpus, the prior — already encodes the world/model distinction, or already carries a notion of agency, then a discovered belief abstraction shows nothing, because belief was present from the start in a thin disguise.

This section is that audit. The pattern each item takes is the same: something is given, and the thing adjacent to it — the thing that would turn it into an attribution — is withheld, so that the connection between them is left for the search to make. We take the charge item by item, in the order of Section 3.4, and then turn to two further places, the success criterion and the proposal distribution, where the result could have been smuggled in by the design of the experiment rather than by the design of the language.

4.6.1. The decomposition into agent, goal and belief

BToM decomposes a scene into an agent that has a goal and a belief, supplied to a planner as separate arguments that vary and are inferred independently. A scene of moving entities does not come so decomposed, and nothing in the data says that behaviour is generated by two attitudes rather than one or five: the decomposition is in the architecture.

In this model there is no such decomposition to inherit, because there is no planner with argument slots for it. The types are grids, cell values, coordinates and arrows between them; none denotes an agent, a goal, or an attitude. A “cell value” is an integer between 0 and 9, and the corpus rotates through eight different (agent, goal) pairs precisely so that no integer is privileged — an abstraction that specialised to “value 1 is the agent” would fail to recur and would not be kept. Nothing marks a cell as an agent, and role is entirely positional: the thing we call the agent is whatever value happens to occupy the policy slot and the commit slot at once.

The decomposition that BToM stipulates is, here, precisely the thing the learner has to construct. The structure

```
(fork <derive> <commit>)
```

with a value shared between a policy inside the derive and the commit outside it, is a two-argument shape in which one argument carries content and the other carries the attribution. That shape is not in the primitive set; it is a compound the searcher assembles and the compression step decides to keep. When Chapter 5 reports an invented abstraction whose body binds one hole to both the policy and the commit, what has been discovered is not the value of a parameter inside a given structure but the structure itself.

4.6.2. The separation of the believed world from the actual world

The complements are there because an unused corner of the vocabulary proves nothing. For each way belief’s composition has to be wired, the library carries the opposite corner of the same symmetry: channel direction (`mapsnd` against `mapfst`), commit scope (`register` on one value against `sync_all` and `sync_except`), z-order (`overlay` against `underlay`), projection (`fst` against `snd`), pairing (`dup` against `pair_blank`), the bifunctor axis (one factor against `bimap` on both), and, among the grid primitives, utility polarity (`neg_dist` against `distance`). Each is the natural tool for some non-mental family in Table 10, and the wiring of belief is one available corner among them rather than the only one the vocabulary affords (Section 4.6.2).

This is the stipulation that makes false belief representable at all, and it is where the model is most exposed. BToM evaluates the planner against a world model m' supplied as an input, and permits $m \neq m'$. Have we not simply built that in?

The honest answer has three parts.

The first concerns what is actually given. What the learner starts with is a product type and general combinators that inhabit it. The pair is symmetric and content-free: `dup` produces (w, w) , and there is nothing in it that says the second copy is a *model* or that anyone *holds* it. What makes one copy the world and the other a belief is entirely a matter of how the composition is wired: transform the second channel and not the first (`mapsnd`, not `mapfst`), commit from model to world and not the reverse (the direct commit, not its conjugate through `swap`), publish exactly the value whose policy was run and not every value that moved (`register (locate av) (place av)`, not `sync_all`). Each of those is a choice with an available opposite, each opposite is exercised by some non-mental family in Table 10, and every one of them is made in program space by the search. The medium is neutral; the asymmetry is the discovered part.

The second concerns the interpreter. In BToM the divergent model is an input to a planner that the observer runs *inside itself* — a nested evaluation, on the agent’s behalf, in a distinguished mode. Our interpreter has no such mode. Its state is one grid; it has no second channel to hand anybody, no notion of running something on someone’s behalf, and no way to distinguish evaluating a program from evaluating a program-as-an-agent-would. When a private grid exists at all it exists because a program built it, for the duration of one call, and the interpreter cannot tell that grid from any other.

The third concerns how far down the decomposition goes. A reader who found atomic `fork` and `sync_to_world` suspiciously like `believe` cut in half would be objecting to the control run of Section 5.2.2, and the objection is answered by not granting them: the library offhands the searcher `dup`, `mapsnd`, `compose_gp`, `pipe_gpg`, `locate`, `place` and `register` — product-category combinators, as domain-general as anything in the language, which no one would describe as mental. Belief is reconstructed from those, and the reconstruction is the chapter’s claim.

What we do not claim is that the regress ends. One can always ask for `dup` to be decomposed in turn, into allocation and reads and writes over a flat memory, so that “hold a second world model” becomes a discovered pattern of buffer use. That relocates the regress rather than ending it: “allocate a buffer” is then the primitive, and it is neither more nor less mental than “form a pair”, because

the capacity for plurality is presupposed by intensionality at every level. Intensionality just *is* the coexistence of a world and a model of it, so any substrate expressive enough to state “act on the world via a transformed model of it” must be able to hold two things at once. There is no substrate in which theory of mind is discovered from nothing; there are only substrates whose primitives are, or are not, individually non-mental and general. The claim we make is the second one, and the combinator library is how we make it as strongly as we know how.

4.6.3. The space of admissible divergences

Given that m and m' may diverge, BToM must say which divergences are admissible; the particular m' is inferred, but the range it is drawn from is fixed in advance, along with the scenarios that produce it — an occlusion, an absence while the world was rearranged. That a belief could be false, and how it came to be, is never itself in question.

Here there is no range to fix. The derive slot of a fork accepts *any* program of arrow $G \rightarrow G$: any grid endomorphism the language can express, including compounds the searcher builds. That is why the belief families differ from one another in what the belief is *about* rather than in some parameter of a common belief schema. A phantom wall is `wall_at r c`; a displaced goal is `step gv d`; being wrong about an obstacle and a goal at once is a sequence of both. The goal-displacement family varies its content across four shove shapes precisely so that an abstraction which specialised to one of them would not recur, forcing compression to keep the content subtree as a hole. This is the productivity and systematicity that Section 1.3 demanded of any adequate account: content composes, embeds, and is not drawn from a menu.

The concession to make here is a different one, and it is real. What the learner finds is a divergent world model, not a belief that is *formed* and could be *revised*. Nothing in the model represents how the agent came to be wrong — no perceptual access, no record of what was witnessed, no update when the agent looks again. In BToM the same lack is a stipulation about admissible divergences; here it is simply a limit on what the corpus depicts and what a single-frame transition function can express. It is the most important respect in which the discovered structure is thinner than the thing it is a model of, and Section 6.4 takes it up as further work.

4.6.4. The space of goals

BToM infers a posterior over goals, but the space it ranges over is given, and so is the prior on it; Baker et al. compare several goal priors precisely because the choice matters and is the modeller’s to make (Baker et al., 2009).

In this model a goal is not a variable ranging over a designated space. It is whichever cell value happens to fill the argument of `neg_dist`, drawn from the same ten integer terminals that supply agent identifiers and step magnitudes. There is no goal type, no goal prior, and no inference over goals as such — there is inference over programs, and a program that seeks something does so because a symbol landed in that slot. Even the polarity is a search choice rather than a stipulated preference: `distance` sits alongside `neg_dist` in the library, so maximising utility flees a value as readily as it approaches one, and the `flee` family exists to make sure that corner is genuinely inhabited. The only prior over any of it is the type-uniform distribution, which prices a goal value with exactly the same rule that prices up and `compose`.

4.6.5. Rationality, and the one thing we do grant

The remaining item on Chapter 3’s list is rationality: the assumption that the agent plans approximately optimally toward its goal given its model. This is the one stipulation of BToM that our learner also receives, in the form of the `optimize` primitive, and the case for granting it is the case Chapter 3 already made on its behalf — that it is the one item on the list with independent developmental support. We give the argument in four parts.

It is developmentally early, and early by a wide margin. Infants encode action as directed at a goal state and expect an agent to take the most efficient means available well before the first birthday (Section 3.1) (Gergely et al., 1995). That is roughly three years before anything recognizable as false-belief attribution (Section 1.2). A model of the acquisition of false belief that helps itself to efficient goal-directed interpretation is helping itself to something the developmental record says is already in place when the process it models begins.

It is not, on the best analysis of it, mentalistic. The teleological stance of Section 3.1 relates an action, a goal state and the constraints of the situation without attributing any representation to the agent (Gergely & Csibra, 2003). That is exactly the semantics of `optimize`. It says how a body moves given a target value and a layout. It does not say that anybody represents the layout.

Its signature makes the point formally. `optimize` has type $U \times V \rightarrow (G \rightarrow G)$: it takes a utility and a value and returns a function from a grid to a grid. It receives one grid and has no parameter identifying whose grid it is, no access to any other channel, and no way to acquire either. Whatever grid it is handed, it treats as the layout. It cannot represent a divergence, because it cannot see two things to diverge; it cannot attribute anything, because it has no argument for an attributee. Belief arises only when something *else* hands it a grid that is not the world and then publishes only part of the result — and that something else is the composition, which is what the learner has to find.

It is not reserved for the mental families. `optimize` is what `desire` uses to walk to a goal, what `flee` uses to run from a hazard, what `comet` uses to bend a trail, and what `obstacle` uses to detour around a real wall — a family whose scenes are, at the level of the rendered trajectory, extremely similar to the belief scenes and yet contain nothing mental at all. Availability is not use. That the same planner appears in mental and non-mental families is what makes the mental ones a discovery about wiring rather than about the planner.

We note the obvious residual cost: `optimize` packs a full breadth-first search into one token, and a more elementary treatment would decompose it into memory, conditionals, and interaction with the environment. That is further work (Section 6.5), and we do not pretend it is free. What we do claim is that granting a planner is not granting a theory of mind, on the same grounds Chapter 3 gave for treating the teleological stance as prior to and independent of the intentional one.

4.6.6. Two confounds that are ours rather than BToM’s

Two further items belong in the audit that answer to nobody on Chapter 3’s list. Neither is a stipulation of BToM; both are properties of how the experiment is run, and they are worth stating because they are the two knobs a sceptical reader would reach for first.

The success criterion admits no partial credit. A candidate program either reproduces all k scenes of a task cell for cell or it does not solve the task; there is no scoring rule under which a program that sends the agent somewhere close to the believed cell scores better than one that sends it somewhere else (Section 4.2, Section 2.1). This matters because a graded likelihood is exactly where a thumb could rest on the scale: under partial credit a mental program that is nearly right could be preferred to an extensional rival that is exactly right, and the comparison of Section 4.7 — which asks whether the mental program wins on description length *among programs that all reproduce the data* — would no longer be the comparison being made. Exact match keeps the likelihood out of the argument, so that the whole of the selection is done by the prior and the compression objective.

Nothing anywhere in the loop is conditioned on what a scene looks like. We omit DreamCoder’s recognition model, and Section 7.7.2 gives the reason at length: a proposal distribution trained on solved tasks is a component whose prior over the library is fit to the data, and belief is precisely the structure the corpus is meant to leave unfitted. Enumeration runs under the type-uniform prior over the current library and nothing else, so the only thing that can make a belief compound reachable is that earlier compression made it short — which is the mechanism the ordering claim of Section 4.7 depends on, and would be confounded by a proposal that had learned what belief scenes look like. The choice was settled by measurement rather than by design, and Section 5.11 reports what the loop gave when it was run both ways.

4.6.7. Two structural differences worth naming

Two further contrasts with BToM are not defensive but positive, and they bear on the explanandum of Section 1.3.

Inference here is not agent-directed. BToM’s machinery operates on agents in particular: the posterior is over (g, m) for an agent, and the inference has nowhere else to point. Our search is over programs for a scene, and it is the *same* search, under the same prior, with the same success criterion, that solves **denoise** and **relocation**. Nothing in it is specialised to the mental families, and nothing in it knows which families those are. Compression is undirected in the same way, and this is why it is pooled rather than run per solution type (Section 4.5.2): were it divided, the division of the search would itself bias abstraction toward belief, since the belief families make up a larger share of the trajectory tasks than of the corpus as a whole. Pooled, the budget that could have paid for belief’s structure is free to spend itself on the overlay, registration, obstacle and relocation families instead. That it goes to belief is a result about how much structure those solutions share, not an artefact of where we drew the boundaries of the search. Whatever is domain-specific in the outcome has to have come from the corpus and the objective, since there is nowhere else for it to have come from.

The ordering is a prediction rather than an accommodation. Because BToM’s structure is complete from the start, every one of its parameters is available for inference on day one, and nothing in the framework makes a goal attribution earlier or cheaper than a belief attribution; the delay of Section 1.2 has to be explained by something outside the model. Here the delay is the mechanism. A belief compound is a deep composition, its cost under the initial library puts it far outside any feasible budget window, and it becomes reachable only once earlier solutions have been compressed into tokens that shorten it. That is a claim that can fail — the compounds might never

come within budget, or they might come within budget in the wrong order — and Section 4.7 says what failure would look like.

4.7. What would count as success or failure

The measurements of Chapter 5 are tests rather than descriptions only if what they test is fixed in advance. This section fixes it.

What counts as having discovered belief. Not that the belief tasks were solved. A solved belief task with an extensional program is a failure of the corpus, not a success of the learner. The criterion is structural, and it is a condition on the invented term: an abstraction must enter the library whose body (i) opens a private channel, (ii) derives from it a world state that diverges from the actual one, (iii) evaluates an agent’s policy against that divergent state, and (iv) commits only the resulting action — not the divergent state — back to the world, with the agent value shared between the policy and the commit. All four conjuncts are checkable on the term, and the fourth is the one that distinguishes attribution from mere counterfactual computation.

It must be a compression win, not merely reachable. Reachability is cheap: given enough budget, many things can be enumerated. The discovered compound has to be *selected* by the joint objective of Section 4.5.2 over the non-mental rivals the same library can express, and the margin must be reported in nats against the shortest such rival — including the rivals that survive the k -scene filter for structural reasons, such as the transient-wall spelling and the witness-agent construction. A margin at or below zero is a failure even if the mental program was found first.

It must generalize behaviourally. A learned belief term should predict, on a scene held out of the run entirely, that the agent goes where it *believes* the goal to be rather than where the goal is. This is the false-belief test proper (Wimmer & Perner, n.d.), and it is the criterion that separates a compressive coincidence from a term that means what we say it means. It also has to be run on a family where the two readings come apart behaviourally, which is why the goal-displacement family carries it: in a wall-belief scene the transient-wall rival reproduces the whole trajectory, so only description length separates the readings there, whereas a displaced goal sends the two readings to different cells.

What would refute the hypothesis. Any one of the following:

1. no belief-structured abstraction, in the four-conjunct sense above, enters the library within budget;
2. one enters, but loses on description length to an extensional rival expressible in the same library;
3. a rival that never opens a private channel reproduces all k scenes of a mental family — which would mean the family was misclassified and the corpus does not test what it claims to;
4. the discovered term fails the held-out behavioural probe, sending the agent to the true location;
5. the discovered term is not applied selectively — if the library invents an agent constructor and then applies it to every mover in a scene, including one whose walk is fully explained by the bare world, then what was found is a habit of forking rather than an attribution.

Two things should be said now rather than left to the results chapter, because they bear on how the verdict should be read.

The first is that there is a depth past which the model’s expressive claim outruns its search claim. Two agents holding contradictory false beliefs, each detouring around its own phantom wall, is a scene the language states without difficulty, but it names six latent literals across two nested forks, and no amount of budget we could give it brought a single such task within reach. The selective-attribution criterion above is therefore carried by the cheaper two-observer case — two agents, one goal, one grid, where only one of them is attributed a belief — and the contradictory-belief case is left as a limit, not a result.

The second is that the criteria are stated per endowment, and a verdict is a verdict about one of them. They are the criteria the library of combinators must meet. The control run of Section 5.2.2 is held to the same list and reported against it, but meeting them there does not discharge them, since the granted `fork/sync_to_world` pair is exactly what an unsympathetic reader would call **believe** in two halves.

Chapter 5

Results

Chapter 3 wrote down the structure that a Bayesian theory of mind stipulates: a planner evaluated against a model of the environment supplied as an input, that input permitted to diverge from the state of affairs the observer takes to obtain, and the whole thing run on a particular agent’s behalf. Chapter 4 built a learner that has none of it — a single-grid interpreter, a monomorphic type discipline over grids and cell values, and a stock of domain-general primitives every one of which is exercised by some non-mental family — and stated in advance (Section 4.7) what would count as that learner constructing the structure anyway. This chapter reports what the learner did.

The claim the results are meant to support is a structural one, and it is worth stating precisely before any numbers. It is not that the belief tasks were solved; a solved belief task with an extensional program would be a failure of the corpus. It is that the joint description-length objective of Section 4.5.2 — the same objective, with no term keyed to the mental families — selects into the library a term whose body has the shape Chapter 3 stipulates, and that it does so from a starting point where that shape was neither a primitive nor a type. Section 5.1 fixes what was run. Section 5.2 reports the libraries the runs converged to and Section 5.3 lays the discovered term beside the BToM structure it is supposed to resemble. Section 5.4 through Section 5.8 then discharge the four criteria of Section 4.7 in turn: that the term is a compression win over the non-mental rivals, that its reachability explains the developmental ordering, that it generalises and is applied selectively, and that it passes the false-belief test on held-out scenes. Section 5.7 recovers, from the runs themselves, which shallower tasks it was assembled out of. Section 5.9 then tallies the result against the criteria, including what failed, and Section 5.10 takes the objections.

5.1 The runs

Everything reported here comes from two full runs on the Snellius cluster, one for each endowment of Section 4.4: the assembled combinator library of Table 12, and the control that grants `fork` and `sync_to_world` outright (Section 5.2.2). Each run enumerates a corpus of 300 tasks — 168 belief tasks across five families and 132 non-mental ones across sixteen — under a per-task budget of 3600 s (1200 s in round 1, which is a cap on the round that has no library yet), for up to six ECD rounds, with a joint `Stitch` call of six iterations at the end of each. A task is four scenes sharing one latent program (Section 4.2), so the 300 tasks are 1200 trajectories, and a candidate solves a task only if it reproduces every frame of all four.

Cell	Primitives	Solved	Belief solved	Constructor round
p2-nodream	combinator (Table 12)	264/300	132/168	2
p1-nodream	atomic control	287/300	155/168	1

Table 1: The two runs; the cell names are the run identifiers, p2 for the combinator endowment and p1 for the atomic one, with **nodream** recording that neither uses a recognition model (Section 7.7.2). “Constructor round” is the ECD round at whose compression step an abstraction with the agency signature of Section 5.2 first entered the library. The atomic cell was run on 2026-07-29 at commit **a6965cd**; the combinator cell on 2026-07-28 at commit **dec58d**, with its commit-form census re-derived under the later classifier (see Section 5.6).

Two facts about Table 1 should be recorded now rather than discovered later. The first is that the combinator cell reported here is a day older than the atomic one. A combinator rerun at the later commit was launched and did not finish: the deeper search needs roughly fourteen hours of wall-clock at these budgets and the job was given eight, so it died in the middle of its third round without writing a verdict. What changed between the two commits is a classifier and a check, not the search, and the combinator solutions have accordingly been re-classified under the later code without being re-researched. Where that re-classification changes a number I say so.

The second is that the figures and tables in this chapter are computed from **p2-nodream** — this is the run the argument of Section 4.4 rests on. Where the atomic control is informative the corresponding number from **p1-nodream** is given alongside, and every claim in Section 5.9 is checked against both cells.

Each run writes three artifacts: the programs it found (with their library-rewritten forms), a per-round trajectory with timings, and a verdict object recording every criterion of Section 4.7 as a machine-checked boolean. The figures and tables below are computed from those artifacts by separate analysis scripts, so that nothing reported here is produced by the code that produced the result.

5.2 What the loop finds

5.2.1 The combinator library: a two-stage bootstrap

The learner is handed the product combinators of Table 12 and must assemble the pair interface before it can use one. The result is not a slower version of the control run of Section 5.2.2; it is a two-stage bootstrap, and the two stages are worth separating because the first is the strongest evidence in the chapter that the machinery belief uses is not belief’s machinery.

Round 1 solves 132 tasks and **none** of them are belief tasks. Every non-mental family falls — physics, desire, the obstacle and relocation families, the overlay and underlay and comet families, all four registration variants — and the belief families are, at 1200 s, out of reach entirely. The compression step then runs over those 132 purely non-mental solutions and returns, among its six abstractions,

```
fn_1: (pipe_gpg (compose_gp dup (mapsnd #0)))      [fn, fn_p_g] -> fn
fn_5: (register (locate #0) (place #0))             [cellvalue] -> fn_p_g
```

which are, by the identities verified before the run began (Section 4.4.1), exactly `fork` and `sync_to_world`. The learner has reassembled the pair interface out of the categorical plumbing, and it has done so from evidence in which no agent holds a belief about anything: `fn_1` is induced by the overlay, underlay and comet families, `fn_5` by image registration.

Round 2 therefore starts where the control run of Section 5.2.2 starts, and 75 belief tasks fall. Its compression step returns the agency signature proper,

```
fn_7: (pipe_gpg (compose_gp dup (mapsnd (compose (wall_at $2 $1) (optimize (neg_dist $3) $0))))
      (register (locate $0) (place $0)))
```

in which `$0` — the agent — appears **three** times: once as the value `optimize` plans for, once in the `locate` that reads its position off the private model, and once in the `place` that writes that position into the world. Rounds 3 and 4 take the remaining 57, and the run converges at 132 of 168 belief tasks. The final library, expanded to base primitives, is Table 2.

Symbol	Body	Belief solves
fn_6	(pipe_gpg (compose_gp dup (mapsnd (compose \$2 (optimize (neg_dist \$1) \$0)))) \$3)	18
fn_7	(pipe_gpg (compose_gp dup (mapsnd (compose (wall_at \$2 \$1) (optimize (neg_dist \$3) \$0)))) (register (locate \$0) (place \$0)))	37
fn_8	(compose (optimize (neg_dist \$0) \$3) (pipe_gpg (compose_gp dup (mapsnd (compose (step \$0 \$1) (optimize (neg_dist \$0) \$2)))) (sync_except \$0)))	26
fn_9	(compose \$2 (optimize (neg_dist \$1) \$0))	46
fn_10	(pipe_gpg (compose_gp dup (mapsnd (compose (wall_at \$0 c2) (optimize (neg_dist \$1) \$2)))) sync_all)	32
fn_11	(pipe_gpg (compose_gp dup (mapsnd (compose (step \$0 \$1) (optimize (neg_dist \$0) \$2)))) (sync_except \$0))	22

Table 2: The final library (p2-nodream), bodies expanded to base primitives. `fn_7` is the agent constructor proper — one hole bound three ways. `fn_8`, `fn_10` and `fn_11` are the same shape spelled through a scope commit (Section 5.6.3); `fn_9` is the bare seek policy belief shares with the obstacle family, through which 47 obstacle solves are also rewritten; `fn_6` is the constructor with its commit left as a hole.

Four of the six carry the agency signature and a fifth, `fn_6`, is the constructor with its commit slot still open. The one term with no fork in it, `fn_9`, is the seek policy — and it is the only invented token that belief and a non-mental family use in common, which is the sharing Section 5.7 takes up. The three terms that commit through `sync_all` or `sync_except` are not a retreat from the single-value corner; on scenes whose only mover in the model is the agent those commits **denote** the single-value one, which Section 5.6.3 verifies by substitution and re-rendering.

This is the bootstrapping structure of Section 2.4 in its clearest form. An abstraction discovered in round N is an atom in round $N + 1$, and here the round-1 atoms are precisely the two capacities

— hold a second grid, transfer one value between two grids — that belief needs and that non-mental tasks independently pay for. Nothing schedules this order. It falls out of what is cheap at each stage.

5.2.2 The atomic control

The second endowment is a **control**, in which `fork` and `sync_to_world` are granted outright as single nodes rather than assembled from the product structure, so that belief is the two-token composition `(fork (compose (wall_at r c) (optimize (neg_dist gv) av)) (sync_to_world av))` with the agent value appearing twice instead of three times. This is not a second result, and it is not a weaker version of the one claimed above; the argument does not rest on it (Section 4.6.2). Its use is as an instrument. Because the two vocabularies are expressively identical — which is what the identities of Section 4.4.1 establish — a task the control solves and the combinator run does not is a task lost to the search budget rather than to the language, and that is a distinction no single run can draw. It also prices the same non-mental rivals at a second granularity, which is what shows the compression result is not an artefact of where the primitives were cut.

The control hands over at the start what the combinator run spent its first round building, so the whole of the search goes on the wiring rather than on the parts. It solves more — 287 of 300 tasks and 155 of 168 belief tasks against 264 and 132 — and its solve set strictly contains the combinator cell's, which is the fact Section 5.11 leans on: 23 of the 36 belief tasks the combinator run missed are solved here, in a vocabulary that expresses nothing the other does not.

Round 1 begins with no library, a uniform prior over 37 primitives, and a 1200 s budget per task. It solves every non-mental family and 55 of the 168 belief tasks by brute cheapest-first enumeration. The obstacle families fall almost immediately,

```
[ 859] caught (compose (wall_at c1 c1) (optimize (neg_dist 5) 4))
```

and the belief tasks built on the same wall content fall two and a half orders of magnitude later in the cost walk:

```
[297342] caught (fork (compose (wall_at c1 c2) (optimize (neg_dist 2) 1)) (sync_to_world 1))
```

The bracketed number is the position in the enumeration, and the ratio between the two is the concrete form of the point Section 2.3 made abstractly: a belief compound is not a slightly less probable hypothesis than an obstacle compound but a hypothesis some three hundred times deeper into an exponentially growing space. The comparison is not rigged by the choice of content: the cheapest belief task of any family in round 1, one whose private derive is a bare `optimize` rather than a wall, still lands at position 21528.

The compression step at the end of round 1 returns six abstractions. Two matter. The first is the wall-and-navigate policy that the obstacle family shares with the belief tasks' derive:

```
fn_1: (compose (wall_at #3 #2) (optimize (neg_dist #1) #0)) [cellvalue, cellvalue, coord, coord] -> fn
```

and the second is built on top of the seek fragment, and is the thing this thesis is about:

```
fn_0: (fork (compose #2 (optimize (neg_dist #1) #0)) (sync_to_world #0)) [cellvalue, cellvalue, fn] -> fn
```

`fn_0` takes a goal value `#1`, an agent value `#0`, and an arbitrary `fn` in the hole `#2`. It runs `#2` on a private copy of the grid, plans `#0`'s move on the result, and publishes `#0`'s move — and only `#0`'s move — back to the world. The content of the false belief is the hole; the agent is the parameter that appears twice.

In round 2 the budget is the same but the library is not, and the remaining 100 belief tasks fall. What had been a search hundreds of thousands of steps deep is now a fill-in-the-blanks — here a witness task, an agent misled about where its goal went alongside a second agent who is not, caught by composing a plain `optimize` for the witness with `fn_2`, which is `fn_0` with a `step` in its hole:

```
[ 8668] caught (compose (optimize (neg_dist 2) 6) (fn_2 up 2 1))
```

Round 3 adds nothing, at which point the loop has converged. The final library, expanded to base primitives, is Table 3.

Symbol	Body	Belief solves
<code>fn_6</code>	<code>(fork (compose \$2 (optimize (neg_dist \$1) \$0)) (sync_to_world \$0))</code>	37
<code>fn_7</code>	<code>(compose \$2 (optimize (neg_dist \$1) \$0))</code>	18
<code>fn_8</code>	<code>(compose (optimize (neg_dist \$1) \$3) (fork (compose (step \$1 \$0) (optimize (neg_dist \$1) \$2)) (sync_to_world \$2)))</code>	24
<code>fn_9</code>	<code>(fork (compose (fork (compose (optimize (neg_dist \$1) \$0) (optimize (neg_dist 0) \$0)) (sync_to_world \$0)) (optimize (neg_dist \$0) \$1)) (sync_to_world \$1))</code>	14
<code>fn_10</code>	<code>(compose (fork (compose (wall_at c2 \$0) (optimize (neg_dist \$1) \$2)) (sync_to_world \$2)) (optimize (neg_dist \$3) \$4))</code>	29
<code>fn_11</code>	<code>(fork (compose (wall_at \$1 \$0) (optimize (neg_dist \$2) \$3)) (sync_to_world \$3))</code>	52

Table 3: The final library under the atomic control (`p1-nodream`), bodies expanded to base primitives. Five of the six carry the agency signature; `fn_7` is the bare seek policy, which belief shares with the obstacle family (47 obstacle solves are rewritten through it).

Five of the six invented terms are agent constructors: a fork whose derive ends in a policy for some value and whose commit publishes that same value. They differ in what the private derive does — an arbitrary hole in `fn_6`, a shove of the goal in `fn_8`, a phantom wall at a free coordinate in `fn_11` and at a fixed column in `fn_10` — and in what else is composed around them. The most-used of them is `fn_11`, the phantom-wall constructor, through which 52 of the 155 belief solutions are rewritten. `fn_9` is worth a remark: its derive contains another fork, so it is an agent whose private model is itself derived by forking. Fourteen belief solutions are rewritten through it. That is not second-order mentalizing in the sense of Section 1.2, since the inner fork is part of the outer agent's model rather than a model of another mind, but it does establish that the discovered shape embeds, which is one of the properties Section 1.3 asked for.

5.2.3 The agency signature

Under either endowment the invented term has the same shape, and the shape is a coincidence in the grammar that the objective declined to treat as one. Written in the abbreviations of Section 4.4.1, the constructor is

```
(fork (compose <derive> (optimize (neg_dist gv) av))
      (sync_to_world av))
```

`optimize` takes a `cellvalue` saying whose move to compute. The `commit` takes a `cellvalue` saying whose position to publish. These are different argument slots on different primitives, filled independently by the type-directed enumerator, and there are ten cell-value terminals to fill each of them with. Nothing in the grammar, the type system or the interpreter requires them to agree. In the invented abstraction they are one hole — and when the `commit` is itself assembled, as it is in `fn_7`, the same cell value has to land in the `locate` and the `place` as well, so the coincidence is threefold rather than twofold.

That collapse is what makes the term an attribution rather than a computation. A program that derives a counterfactual grid, plans on it, and publishes the result of planning for **some other** value is expressible, is one token cheaper to describe in no respect, and does not occur. What compression kept is the version in which the value whose behaviour is published is the value whose reasoning was simulated — which is to say, the version in which the private grid is **somebody's**.

It should be said that nothing in this identifies the term as **belief** in any thicker sense, and on the account of intentionality this thesis has been working with, nothing needs to. To adopt the intentional stance towards a system is to treat it as having the beliefs it ought to have and to predict its behaviour accordingly (Dennett, 2002); what makes a state a belief is the role it plays in that predictive scheme, not an intrinsic mark it carries. The invented term is a state that is derived privately, is planned against, is attributed to one value rather than to the scene, and predicts where that value will go. That is the role. Asking whether the thing filling it is **really** a belief is asking a question the framework declines.

5.3 The discovered term against BToM

Section 3.4 listed what BToM assumes in order to model mature performance. The list is now a list of things to look for in the invented term, and Table 4 is the correspondence.

BToM (Chapter 3)	In the discovered term	Status here
A planner mapping (utility, environment model) to action	<code>optimize (neg_dist gv) av</code> , applied to whatever grid it is handed	Granted as a primitive (Section 4.6.5)
The environment model m' supplied as an <i>input</i> to the planner	The second channel of the pair, the only one the derive writes	Discovered: <code>mapsnd</code> , not <code>mapfst</code> ; the world channel is untouched
$m \neq m'$ permitted — the divergence that makes false belief expressible	The derive is an arbitrary <code>fn</code> applied to the private copy only	Discovered: which channel is transformed is a search choice with an exercised opposite
The goal g , ranging over a given space under a given prior	Whichever <code>cellvalue</code> lands in <code>neg_dist</code> 's slot, priced like every other symbol	Not a space: no goal type, no goal prior (Section 4.6)
The attribution: m' is <i>this agent's</i> model, on whose behalf the planner runs	The shared hole — <code>optimize</code> 's actor <i>is</i> <code>sync_to_world</code> 's committed value	Discovered: the conjunct the compression step kept
The nested evaluation: the observer runs the planner inside itself	<code>fork</code> 's private grid lives for one call and is discarded, never rendered	Structural, in program space; the interpreter has no such mode (interpreter)
Inverse planning: infer (g, m) from the trajectory by Bayes	Cheapest-first enumeration under the library prior, indicator likelihood	The inference of Section 2.1, unchanged and not agent-directed
Belief as a state that is <i>formed</i> and could be <i>revised</i>	Nothing: the model exists for one transition and has no history	Absent — the honest gap (Section 5.11)

Table 4: What BToM stipulates, and where it turns up in the term the compression step invented.

Read down the middle column and the invented abstraction is a Bayesian theory of mind's agent, written in a language that had no word for one. Read down the right and the difference from Chapter 3 is where each row came from. BToM's rows are architecture, fixed before any scene is seen, and its inference locates a point inside them. Here, three of the rows — which channel diverges, whose model it is, and what the divergence consists in — are decisions made in program space by a search that also solves image registration and denoising, and kept by an objective that also had the option of spending its budget on the overlay and relocation families.

Two rows are worth dwelling on because they are the ones an unsympathetic reader will press.

The first is the planner. We grant it, and Chapter 3 gave the reason: on Gergely and Csibra's analysis the teleological stance relates action, goal and situational constraint without attributing any representation to anybody, and it is in place by twelve months, three years ahead of false-belief attribution (Gergely et al., 1995; Gergely & Csibra, 2003). The results bear out the formal version of that argument. `optimize` appears in the desire family (16 solves), the flee family, the comet family and all 48 obstacle solves — scenes which are, at the level of the rendered trajectory, nearly

indistinguishable from the wall-belief scenes and contain nothing mental at all. What separates the mental families is not that they use the planner but what is wired around it.

The second is the pair. **dup** produces (w, w) and says nothing about which copy is a model or who holds it; the asymmetry is entirely in the composition. Section 5.6 shows that the symmetric alternatives are not merely available but **used** — by other families, in the same runs — which is what turns “the learner could have gone the other way” from an assertion into a measurement.

5.4 The objective selected it

The guiding thought behind the whole project is that an intensional theory — one that attributes a belief to an agent — is the most compressed available description of that agent’s behaviour, and that this is not a coincidence. The agent navigates by its own representation of the world, so a theory that posits that representation is tracking the process that actually generated the data. One can of course describe the same trajectory without attributing anything, but the description has to account for each step of the walk separately, and it will not transfer to the next scene. The prediction, then, is that the description-length objective should prefer the intensional reading, and prefer it more the more experience the learner has had.

Section 4.7 demanded that the compound be a compression win rather than merely reachable. There are three measurements here: what the library does to the description length of the whole corpus, what it does to belief programs specifically, and how the belief program compares to the non-mental rivals that the same library can express.

5.4.1 The corpus description length

The quantity the compression step minimises (Section 4.5.2) is the joint description length of the library together with the corpus written in it. Fixing the corpus and repricing it under each round’s library gives, for round r ,

$$\text{DL}(X|r) = \sum_{x \in X} \text{DL}(p_x|L_r) + \text{DL}(L_r)$$

where X is the corpus, p_x the program the run found for task x , and L_r the library after round r ’s compression. Figure 4 plots this, split by family.

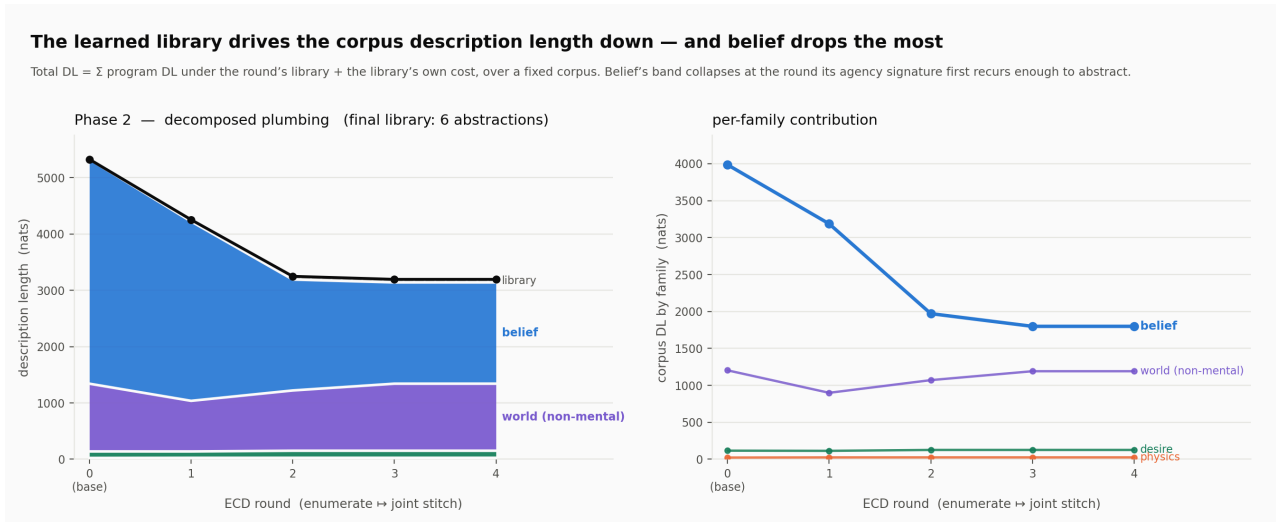


Figure 4: Corpus description length by round (p2-nodream). Left: the stacked total, with the library’s own cost as the black line. Right: the per-family contribution. The belief band collapses at round 2, the round after the one that rebuilt `fork` and `sync_to_world` from non-mental evidence (Section 5.2.1).

Under the base primitives the 264 solved programs cost 5328 nats. After four rounds they cost 3195, including the 54 nats the library charges for itself. Of the 2133 nats saved, 2187 come from the belief family alone — 3986 nats down to 1799 — and the largest single step is the 1216 nats that go at round 2, the round whose compression step invents the constructor. The non-mental families do not move. `world` (the sixteen non-mental families pooled) goes from 1203 to 1191, `desire` from 116 to 126, `physics` from 23 to 25: the last two get slightly **worse**, which is the library tax of Section 4.5.2 showing up exactly where it should — a family that gains no abstraction still pays for the enlarged symbol set.

That asymmetry is the argument. If the initial primitives had encoded anything mental, belief would have been cheap from the start and the belief band would look like the desire band. It is because belief is expensive under the base library and cheap under the learned one that we can say the mental content is in what was learned rather than in what was given.

The atomic control gives the same picture a round earlier and from a higher starting point: 6017 nats down to 3687, belief carrying 4723 to 2345, with its largest drop — 2025 nats — at round 1, since there the constructor can be invented as soon as anything has been solved.

5.4.2 What the library buys each family

Per task, the same result in nats. Table 5 prices each family’s found program under the base primitives and under the final library.

Family	Solves priced	Base (nats)	Final library	Saved
belief	25	34.87	15.95	18.92
obstacle	48	16.21	12.95	3.26
desire	16	7.86	7.86	0.00
comet	4	16.09	16.09	0.00
relocate	16	14.13	14.13	0.00
registration	4	6.68	6.68	0.00
perception	4	4.16	4.16	0.00
flee	4	7.86	7.86	0.00
deletion	4	5.78	5.78	0.00
denoise	4	4.87	4.87	0.00
multi_reg / reg_except / inpaint / readout	4 each	2.08–4.38	unchanged	0.00

Table 5: Median description length per task (p2-nodream), before and after the learned library. Belief saves 18.92 nats per task and 472 nats over the priced solves; every family but obstacle saves nothing. Belief is the only family the library substantially compresses, and obstacle is the only other family it touches at all — through `fn_9`, the seek fragment the two share, which is exactly the donor relation Section 5.7 recovers independently. The atomic control gives the same shape with a smaller belief saving, 33.71 down to 13.38 nats, since less of the compound is left to abstract once the pair interface is given.

One caveat about the belief row. This census prices only solutions whose library-rewritten form re-parses under the final library, which here is 25 of the 132 belief solves — all of them from the witness family — so the 18.92 is a median over a fifth of the family rather than over all of it. Figure 4 is the unrestricted version of the same measurement: it prices every solved program in every round, and gives the belief band’s 3986 to 1799 nats directly.

5.4.3 No non-mental rival reproduces the scenes

The third measurement is the one Section 4.7 cared most about: whether a non-mental program is available that is as short and as accurate. Each belief task carries a battery of hand-written rival spellings — the transient-wall schedule that stamps a wall, lets the agent walk, and erases it before the frame is rendered; the version that shoves the goal in the world rather than in a model; pure desire; a physics reading with no wall at all; and, for the witness family, the four orderings of stamp, act, witness and erase. Each rival is rendered over every scene of every belief task and compared frame for frame.

Rival spelling	Tasks	Reproduces all 4 scenes	Reproduces the agents	Median margin (library, nats)
transient wall (stamp / act / erase)	24	0	24	+6.27
transient wall (erase-3 variant)	24	0	24	+5.35
transient shove (shove / act / restore)	51	0	51	+18.59
witness then stamp / act / erase	48	0	0	+14.17
stamp / act / erase then witness	48	0	0	+10.91
stamp, witness, act, erase	48	0	0	+10.91
stamp, act, witness, erase	48	0	0	+10.91
transient phantom + shoved goal	9	0	0	+19.73
shove goal in the world	60	0	0	+4.87
both agents seek the goal	24	0	0	+4.17
transient, witness ignored	48	0	0	+3.74
pure desire (no belief)	27	0	0	−0.69
no wall (physics)	72	0	0	−2.53
no belief (seek past the real wall)	9	0	0	−2.53

Table 6: The non-mental rivals (p2-nodream), over the 132 solved belief tasks. “Margin” is $DL(\text{rival}) - DL(\text{found})$ under the final library; positive means the mental reading is shorter. Not one rival reproduces all four scenes of any task.

The headline of Table 6 is the third column. Across 132 belief tasks and fourteen rival spellings, **no non-mental rival reproduces all four scenes of any task**. This is a change from earlier versions of the corpus and it is a change in kind. It used to be that the transient-wall rival reproduced the wall-belief scenes exactly and had to be beaten on description length; the honest form of the argument was then a margin. Two corpus changes closed it. An inert bystander object now sits on the phantom-wall cell for the whole trajectory, so stamping a real wall there overwrites something the frame shows and nothing in the language can put it back; and the k -scene format (Section 4.2) requires a rival to survive four independent placements of the same latent, which a single wall coordinate chasing a believed goal cannot do. The fourth column records how close the rivals get: the transient-wall spelling still reproduces the **agents’** trajectories on all 24 wall-belief tasks, and the transient shove on 51 goal tasks. They get the walk right and the world wrong.

This is exclusion by reproduction rather than by price, and it is the stronger form of the argument — a rival that cannot render the data is not a rival at all, whatever it costs. It is also the form that keeps the argument honest in the other direction: 111 of the 540 rival pricings in Table 6 come out **shorter** than the mental program, almost all of them the bottom three rows, and every one of them is a program that simply does not depict the scene — an agent walking to a goal it is not wrong about. Cheapness without accuracy is worth nothing, and it is the likelihood of Section 2.1 rather than the prior that disposes of them. But the price is still informative, and the last column is where the compression story reappears. Take the closest rival, the transient wall. Under the base primitives it costs 0.12 nats more than the mental program — to three decimal places a tie, which

is what one would expect, since neither reading has any structure to reuse. Under the final library it costs 6.27 nats more — a factor of about $e^6 \approx 500$ in prior probability. Nothing about the rival changed. What changed is that the mental reading collapsed into a token the library had a reason to invent and the transient-wall schedule did not, because a bespoke stamp-act-erase sequence recurs nowhere else in the corpus.

The atomic control gives the same result from a slightly looser starting tie: +1.28 nats under base primitives, widening to +10.91 under its final library. That the effect survives both carvings of the primitive set is the point of running the control — the compression win is not an artefact of how coarsely the parts were cut.

5.5 Reachability, and the order in which things arrive

Section 2.3 argued that description length in this setting is simultaneously a measure of plausibility and a budget: a hypothesis that is long under the current library is not merely improbable but unreachable. Section 4.7 turned that into a prediction about ordering. Figure 5 is the prediction met.

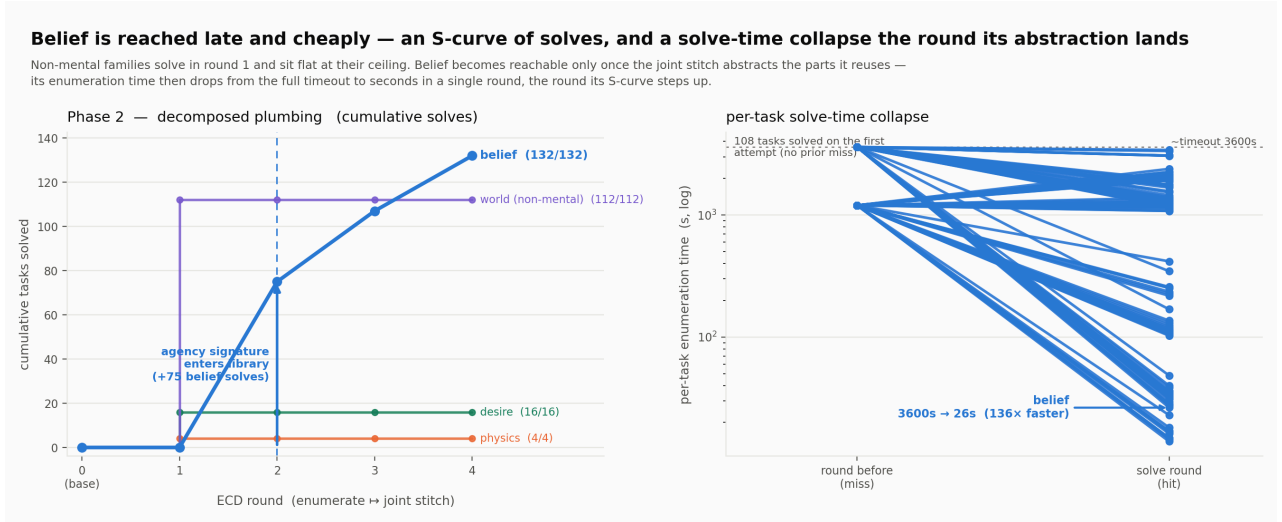


Figure 5: Solve dynamics (p2-nodream). Left: cumulative solves by family, by round. Right: for each belief task solved after at least one miss, its enumeration time in the round it missed against the round it was solved.

The non-mental families are flat lines at their ceiling from round 1: they need no abstraction and get none. Belief is an S-curve — 0, then 75, 107, 132 — whose step is the round after the constructor enters the library. And the first point of that curve is a zero rather than a small number: with the pair interface given only in parts, not one belief task is solved until the compression step has reassembled `fork` and `sync_to_world` out of non-mental evidence. Belief here is not merely slow in round 1; it is out of reach.

The right-hand panel is the same fact per task, and it is more equivocal. All 132 belief solves were missed at least once, and the worst case goes from the full 3600 s timeout to 26.4 s, a factor of 136 — but the **median** moves only from a 1200 s round-1 timeout to a 1164 s solve. What the library buys in this cell is therefore reachability rather than speed: it brings the compound inside the budget window at all, and most tasks then spend most of that window. The atomic control moves the median further — of 100 tasks missed at least once, it goes from a 1200 s round-1 timeout to

a 691 s solve, and the steepest collapse is from 1200 s to 2.1 s, a factor of 562 — which is what one expects of a cell that starts a rung higher; but there too the typical task still spends more than half its budget. In neither case did the tasks or the budget change; what changed is the description length of their solutions, and with it the position of those solutions in the cost-ordered walk.

It is worth being clear about what this does and does not establish. It does not show that a child’s four-year delay has this cause. What it shows is that in a learner of this kind the delay is not something that has to be put in. Section 3.4’s complaint against BToM was that its structure is complete from the start, so every parameter is available for inference on day one and nothing in the framework makes goal attribution earlier or cheaper than belief attribution; the ordering has to be imported from outside as maturation or as a performance limitation. Here the ordering is the mechanism. Desire is solved in round 1 because `(optimize (neg_dist 2) 1)` is a three-symbol program; belief is not solved until round 2 because its compound is out of budget until earlier learning has shortened it. That is the cumulative, staged picture Section 1.3 demanded, produced rather than accommodated: the bootstrapping of Section 2.4, measured rather than assumed.

5.6 Generality and selectivity

An abstraction that is a compression win might still be a bad theory in two ways: it might have baked in the particular values it was trained on, and it might be applied to everything that moves. Section 4.7’s fifth refutation condition was precisely the second of these — a habit of forking rather than an attribution. Three measurements answer them.

5.6.1 The constructor spans the corpus

The corpus rotates through eight distinct (goal value, agent value) pairs so that no integer is privileged (Section 4.6). Figure 6 counts, for each combination and each invented token, how many solved belief programs are rewritten through it.

One discovered abstraction is the agent — it tiles every (gv, av) belief task

Rows are the (goal-value, agent-value) combinations the curriculum poses; columns are the invented library abstractions. A cell counts solved belief programs whose rewritten form uses that abstraction — the agent-constructor column is filled for every combination.

Phase 2 — decomposed plumbing (agent constructor: fn_9)

(gv, av)	fn_9	fn_6	fn_7	fn_8	fn_10	fn_11	variants
gv1·av7	5	3	4	4	2	3	▲●■◆
gv2·av1	5	3	7	4	3	2	★▲●■◆
gv4·av9	6	2	4	3	4	4	★▲●■◆
gv5·av4	5	2	5	3	6	3	★▲●■◆
gv6·av2	6	3	6	3	2	2	★▲●■◆
gv7·av6	6	1	2	3	6	3	▲●■◆
gv8·av5	6	1	5	3	4	2	★▲●■◆
gv9·av8	7	3	4	3	5	3	★▲●■◆

invented library abstraction →

agent constructor tiles every (gv,av)

■ wall
 ◆ witness
 ▲ goal
 ● two observers
 ★ false-obstacle

Figure 6: Constructor coverage (p2-nodream). Rows are the eight (goal, agent) value combinations; columns are the six invented abstractions. Every column is occupied in every row.

The matrix is fully occupied: each of the six invented tokens is used in all eight combinations. The agent constructor `fn_7` carries 37 of the 132 belief solves and appears in every row; the most-used token is `fn_9`, the bare seek policy, at 46. So the constructor is not a memorised (goal, agent) pair with a wrapper — its cell-value slots are genuine holes, exercised across every pair the corpus rotates through. The instructive comparison is the obstacle family, whose policy did **not** generalise into a single token in this run: because obstacle scenes vary their wall coordinate more than their (goal, agent) pair, no single wall-bearing policy recurred often enough to be abstracted with holes. Belief’s derive is therefore not a single cheap token even after compression, which is to say that the belief compound is not shallow even with the constructor in hand. The atomic control gives the same picture slightly less evenly: its constructor `fn_11` spans all eight combinations, by 52 of 155 solves, but one of its six tokens — the bare seek policy `fn_7` — is missing from two rows.

5.6.2 The symmetric complements go elsewhere

Table 12 lists, beside each primitive belief uses, the opposite corner of an independent symmetry axis: channel direction, commit scope, z-order, projection, pairing, the bifunctor axis, and utility polarity. The point of including them was that an unused complement proves nothing. Table 7 reports which family claimed which corner.

Corner	Family that selected it
<code>register</code> (<code>locate v</code>) (<code>place v</code>) (<code>read model</code> , <code>write world</code> , one value)	belief (37 solves), registration
<code>via_swap</code> (the same commit through the swapped pair)	perception
<code>sync_all</code> (every shared value)	multi-registration, perception
<code>sync_except</code> (all but one)	registration-except
<code>dup</code> / <code>mapsnd</code> (diagonal; act on the second factor)	belief (81), overlay, underlay, comet
<code>overlay</code> / <code>underlay</code> (z-order)	overlay, comet / underlay, inpainting
<code>snd_gg</code> (project the model channel)	readout
<code>neg_dist</code> / <code>distance</code> (utility polarity)	belief, desire, obstacle / flee
<code>wall_at</code> / <code>clear_at</code> / <code>erase</code> (grid edits)	belief, obstacle, relocation / deletion, relocation / denoise
<code>mapfst</code> , <code>bimap</code> , <code>pair_blank</code> , <code>swap</code>	no family, belief included

Table 7: The cube census (**p2-nodream**). Belief selects the one asymmetric corner; every complement that any family claims is claimed by a non-mental one. The atomic control gives the same census with `sync_to_world` in place of the `register` triple and `sync_to_model` in place of `via_swap`.

Every complement the run’s census tracks — the channel direction, the two scope corners, the reversed z-order, the model-side projection, the repulsive utility, the removal edits — is claimed by some family, and belief claims none of them. Availability is not use: the search had all of them in hand at the same prior cost, and it took the corner that corresponds to acting on the world through a private model of it.

The last row is the honest exception. Four of the combinators offered — acting on the first factor, acting on both, pairing with a blank, and the bare twist — are used by **no** family in this run. Their being available still does the work the design asked of them, since a corner the search could have taken and did not is what makes taking the belief corner a choice; but they are not, in these runs, independently exercised the way the other complements are, and a corpus with a family that needed them would make the point more cleanly.

5.6.3 Attribution is selective

The two-observer family puts two agents in one scene with one goal and one grid, where only one of them is wrong about anything and the other’s walk is plain desire. If what the learner acquired were a habit of forking, it would fork for both. In **p2-nodream**, 24 of 24 solved scenes commit for the believer and not for the bystander; in **p1-nodream**, 24 of 24. There is no over-attribution anywhere in the 48 solved two-observer scenes.

This is also the place to explain a number that looks alarming in the combinator census and is not. Of the 132 belief solutions in that cell, 37 commit through the literal `register` (`locate av`) (`place av`) and 95 commit through a **scope complement** — `sync_all`, or `sync_except gv`. Read as a token, that looks like belief giving up its corner. Read as a computation, it is not: on a scene whose only moving value in the model is the agent, “move everything to its model position” and “move

everything but the goal” both denote “move the agent”, and the classifier verifies this by swapping the commit for the single-value one and re-rendering all four scenes. Every one of the 95 passes. What differs is the spelling — the agency hole expressed on the goal rather than on the actor — and it is cheaper by a token, which is why compression prefers it wherever the scene permits.

The family that does not permit it is the false-obstacle family, which is constructed so that no scope complement reproduces its scenes: a real wall and a stationary goal are present alongside the phantom one, so a wholesale commit would move them. Every solved false-obstacle task in either run — 16 of 24 in `p1-nodream`, 9 of 24 in `p2-nodream` — commits through the literal single-value `sync_to_world`. Where the degeneracy is excluded by construction, the agency commit is forced, and it is found.

Under the classifier at the later commit, which reads the commit form off the world-level commit position rather than off the solution string, the census over both cells reports **zero** non-mental commits: every belief solution in either run publishes exactly what some model sought.

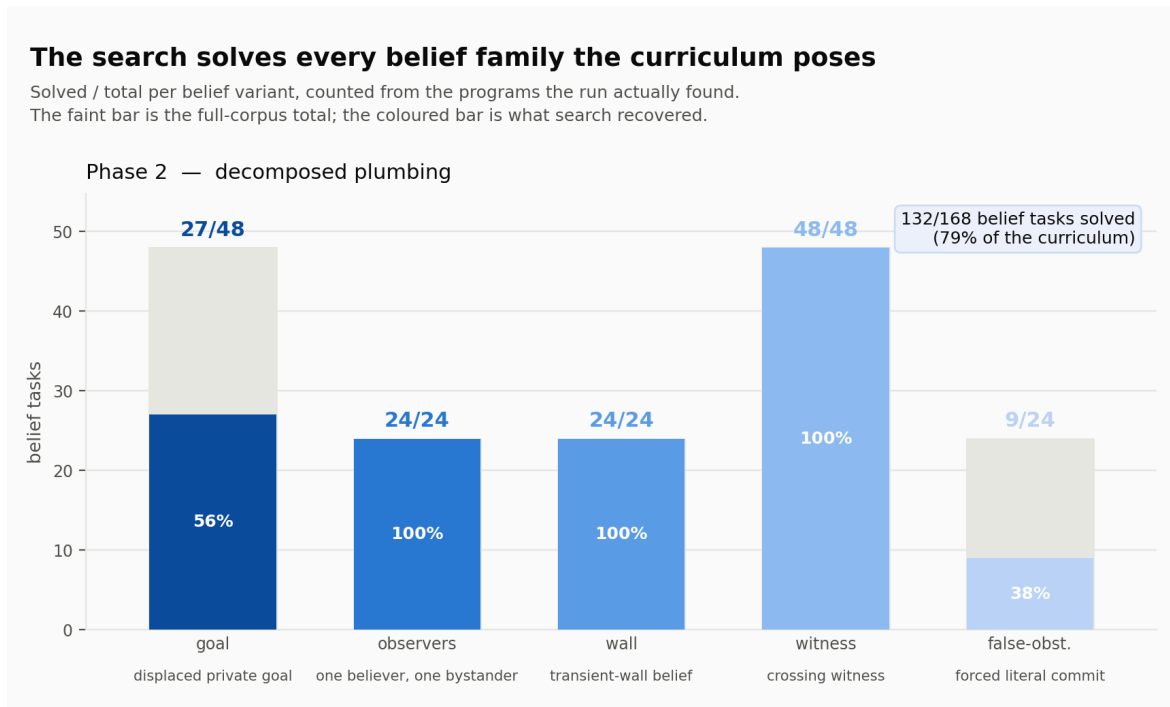


Figure 7: Belief coverage by family (`p2-nodream`). The faint bar is the corpus total; the coloured bar is what search recovered.

Figure 7 gives the coverage that all of this is computed over, and it is the weakest number in the chapter. Three families are complete — wall-belief 24 of 24, witness 48 of 48, two-observer 24 of 24 — but goal-displacement is 27 of 48 and the false-obstacle family, the deepest compound in the corpus, naming a real wall, a phantom wall, a displaced goal and an agent, is 9 of 24. The atomic control reaches 155 of 168 on the same corpus: complete on the first three, 43 of 48 on goal-displacement and 16 of 24 on false-obstacle. Since its vocabulary expresses nothing this one does not, the 23 tasks in the gap are tasks this run ran out of budget on rather than tasks it could not state — Section 5.11 is careful about how much that argument is worth.

5.7 Scaffolding

Section 1.2 records that theory of mind arrives in a fixed order, and Section 1.4.3’s account of why is that later concepts are built out of earlier ones. Nothing in the corpus asserts such an order: a task carries only its own kind, and no family is marked as scaffolding for another . Whether the order is there is therefore a question to be put to the runs, and it can be put in a way that does not presuppose the answer. Take each solved belief program, abstract its concrete values and coordinates to typed holes, and ask which **other** families’ solved programs occur inside it as subterms.

Donor family	Belief tasks it feeds	The fragment, and the role it plays
desire	132	(optimize (neg_dist #v) #v) — the action policy
obstacle	84	(compose (wall_at #c #c) (optimize (neg_dist #v) #v)) — the derive
registration-except	51	(sync_except #v) — the commit, in its scope spelling
physics	48	(step #v up) — the displacement inside a displaced-goal derive
registration	37	(register (locate #v) (place #v)) — the agency commit
false-obstacle belief	28	the whole constructor, feeding witness and wall
goal belief	26	a displaced-goal constructor
wall belief	23	a phantom-wall constructor, feeding the witness family
witness belief	3	a composed pair of agent blocks
flee	2	(optimize (distance #v) #v)
two-observer belief	2	a seek composed with an agent block

Table 8: Structural scaffolding recovered from `p2-nodream`: which shallower solved programs occur as subterms of belief solutions. Nothing in the corpus labels any of these a scaffold; the relation is read off the programs.

Every one of the 132 belief solutions contains a desire solution. That is the strongest and least surprising row: belief’s policy is the desire family’s program, and that family has nothing mental about it — it is one object approaching another. The commit is the same story split across two spellings: 37 solutions contain the registration family’s `register (locate v) (place v)` outright, and 51 contain the registration-except family’s scope commit, which on these scenes denotes it (Section 5.6.3). The obstacle family supplies the derive for 84. And the belief families feed each other in the order the corpus’s difficulty predicts: a wall-belief constructor sits inside witness-belief, which is a wall-belief with a second, unforked agent walking across the phantom cell.

The tool prints the same fact as an assembly. A witness-belief solution, annotated with the family that certifies each maximal fragment:

```
(compose
  (optimize (neg_dist 4) 5)
  <- desire [action policy])
```

```
(pipe_gpg (compose_gp dup
  (mapsnd (compose (wall_at c3 c1)
    (optimize (neg_dist 2) 1))))
  (register (locate 1) (place 1))) <- belief [agent constructor]
```

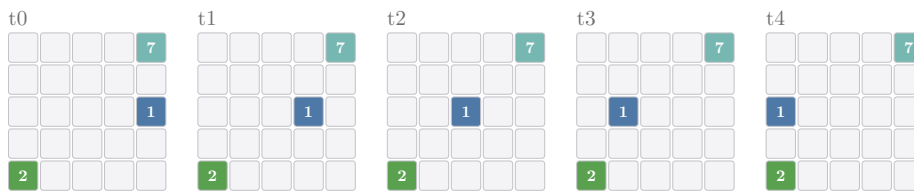
The second mechanism is compression rather than containment, and it is the one the objective actually paid for: when a belief program and a non-belief program are rewritten through the **same** invented token, that token is shared reuse. Here there is exactly one such token, `fn_9`, the seek fragment shared between belief and the obstacle family, alongside the two round-1 tokens that reassembled `fork` and `sync_to_world` out of the overlay and registration families. The atomic control has only the seek fragment, since there is no pair interface left for it to rebuild. So the sharing is real but narrow, and that is the correct result: what the non-mental families donate is the **parts**, and what is belief-specific is the **wiring**. An abstraction that were shared wholesale with a non-mental family would not be an agent constructor.

Two caveats. This is scaffolding in the sense of compositional dependence recovered from a converged run, not a longitudinal claim about a learner acquiring one rung and then the next; the ordering evidence for that is Section 5.5. And the curriculum is ours: the obstacle and relocation families are deliberately over-weighted precisely so that the wall-and-navigate policy would recur often enough to be abstracted. That is a modelling choice about what the learner sees, not a thumb on the scale of what it concludes, but it is a choice, and Section 5.11 returns to it.

5.8 The behavioural probe

Everything so far is description length. Section 4.7’s third condition was behavioural: on a scene held out of the run entirely, does the learned term predict that the agent goes where it **believes** the goal to be?

The probe uses the goal-displacement family, because it is the one where the two readings come apart in behaviour. In a wall-belief scene the transient-wall rival walks the agent along the same path, so only description length separates the readings there — that was Section 5.4.3. In a goal-displacement scene the agent walks to where it believes the goal is, one cell from the true goal, which never moves, so a program that does not represent the belief cannot land on the right cell.



The scenes are drawn from a seed that never entered the run and are checked to be absent from the training corpus. The library is rebuilt by re-stitching the run’s own found programs, so the terms priced are the terms the run converged to. We then instantiate the discovered belief compound on the novel scene and, as its foil, the shortest program the same library can express without it.

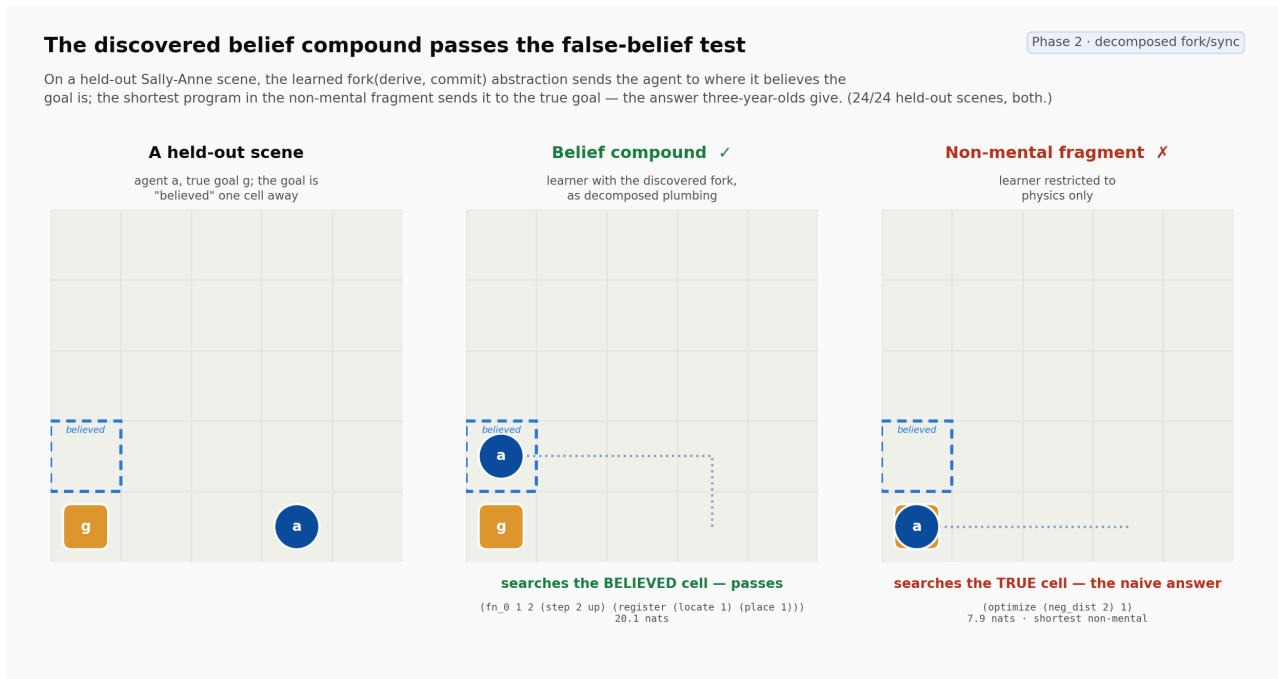


Figure 8: The false-belief test on a held-out Sally-Anne scene (p2-nodream). The learned belief compound sends the agent to the believed cell; the shortest non-mental program under the same library sends it to the true goal.

Through the library, the belief compound is written

```
(fn_0 1 2 (step 2 up) (register (locate 1) (place 1)))
```

which expands to

```
(pipe_gpg (compose_gp dup (mapsnd (compose (step 2 up)
                                              (optimize (neg_dist 2) 1))))
  (register (locate 1) (place 1)))
```

— the discovered constructor applied to a derive that privately displaces the goal, committing only the agent’s own move. Its final frame puts the agent on the believed cell, one step short of the true goal. The learner restricted to the non-mental fragment writes the shortest thing that fragment can say, plain desire `(optimize (neg_dist 2) 1)`, and walks straight to where the goal really is.

Across all 24 held-out scenes the belief compound lands on the believed cell every time. The non-mental program lands on the true goal cell in 18 of 24 and, in the remaining six, one step short of it on the true-goal side — it is walking toward the real goal and runs out of frames. It never once lands on the believed cell. The same probe run against the atomic control’s library gives the same 24 of 24, as it must: the two endowments differ in how the compound is **spelled**, not in what it computes.

The nats are the point of the exercise. Here the non-mental program is **shorter** — 7.9 against 20.1 here, and 7.9 against 13.4 under the atomic control — which is the opposite of Section 5.4.3, and that is exactly why both measurements are needed. Where a non-mental rival reproduces the scene, it does so only at greater length, so description length rules it out. Where a non-mental rival

is cheaper, it is cheaper only because it predicts the wrong behaviour, so the data rule it out. There is no program in the non-mental fragment that is both as short as the belief compound and as accurate. A learner that has compressed its experience into that compound passes the false-belief task; a learner confined to the fragment fails it, and fails it in the specific way the developmental literature documents — by answering with the true location (Wimmer & Perner, n.d.).

5.9 The verdict

Section 4.7 fixed in advance what would count as success and what would refute the hypothesis. Table 9 is the tally.

Criterion (Section 4.7)	Combinator	Atomic
An abstraction enters the library that opens a private channel, derives a divergent state, evaluates a policy against it, and commits only the action, with the agent value shared between policy and commit	Met, round 2 (fn_7, one shared hole used three times)	Met, round 1 (fn_6, one shared hole used twice)
Selected by the joint objective over the non-mental rivals the same library expresses	Met: +6.27 nats over the nearest rival under the library, from +0.12 under base primitives	Met: +10.91 nats, from +1.28
No rival that never opens a private channel reproduces all k scenes of a mental family	Met: 0 of 14 rival spellings over 132 tasks	Met: 0 of 14 over 155 tasks
The term generalises behaviourally on a held-out scene	Met: 24/24 to the believed cell	Met: 24/24
The term is applied selectively, not to every mover	Met: 24/24	Met: 24/24
Belief coverage	132/168 (79%)	155/168 (92%)

Table 9: The criteria of Section 4.7 against the runs.

Two things did not come out clean, and they belong here rather than in a footnote.

Both runs print ok: false, and for two different reasons, neither of which is a row of Table 9. I report them because a run that prints **ok: false** should not be quoted as if it printed **ok: true**.

The combinator cell fails **no_belief_complement**. That check was written before the commit-form classifier learned to tell a scope commit that **denotes** the single-value one from a genuinely non-mental commit, so it fires on all 95 of the **sync_all** and **sync_except** spellings discussed in Section 5.6.3. Under the later classifier those 95 are verified by substitution and re-rendering, the

census splits into 37 literal and 95 degenerate, and the count of non-mental commits is zero. The failing boolean is the old check, not a finding.

The atomic cell fails `obstacle_belief_misrepresents_obstacle`, which demands that **every** solved task in the two families whose agent is wrong about an obstacle — plain wall-belief and false-obstacle — have a private model in which the obstacle is misplaced. Plain wall-belief passes 24 of 24, so the failures are in the false-obstacle family, and the diagnosis recorded for that run is that one or two of its tasks were solved by a program representing the agent as wrong about the **goal** rather than about the wall — a nested-fork displaced-goal model, which reproduces the scenes and is a belief attribution, but not the one the family was built to force. On that diagnosis this too is a mis-specified check rather than a non-mental rival: the criterion asks about the wall because that is what the family names, but the family’s scenes admit a second mental explanation the check does not recognise.

Coverage is incomplete, and unevenly so. The combinator cell misses 21 of 48 goal-displacement tasks and 15 of 24 false-obstacle tasks. The case that these are budget misses rather than expressive ones is the atomic control: its solve set strictly **contains** the combinator cell’s — 23 belief tasks solved there and not here, none the other way — in a vocabulary that expresses nothing this one cannot, and the combinator solve-time distribution (Figure 5) has a large mass pressed against the 3600 s ceiling. That is an argument from a shorter spelling of the same programs, not from a longer run, and a longer run is what would settle it; this thesis does not have one. The `belief_dual` family, removed from the corpus for the reasons Section 4.7 gives, is the version of this that could not be papered over at all.

Against that, what did come out clean is the load-bearing part. In both cells the compression step invented a term with the shared-hole signature; in both the belief families used it and no non-mental family did; in both the cube census found belief on the one asymmetric corner with every complement claimed by somebody else; and in both, under the later classifier, not one belief solution committed a value that no model had sought.

5.10 Objections

5.10.1 “fork and sync_to_world are just believe cut in half”

The sharpest objection is that the atomic primitive set of the control run already contains the answer. By the definition this thesis took from the literature, mentalizing is attributing to an agent an intensional state — a representation of the world that may diverge from it. `fork` builds a divergent representation and `sync_to_world` reads an agent’s position off it. Putting them together is not discovery; it is reassembly of something that was handed over in two pieces.

Four things answer this, in increasing order of force.

First, the parts have non-mental uses and take them. `fork` is used by the overlay, underlay and comet families; `sync_to_world` by image registration. These are not token gestures: they are what those families’ **ground-truth** programs are made of, and the census in Section 5.6 shows the searcher reaching for them there. A primitive that only ever appeared inside belief compounds would be a **believe** in disguise; one that image registration uses is a general-purpose tool.

Second, the field is symmetric and the complements are live. Table 7 is the measurement: the opposite channel direction, the two wider commit scopes, the reversed z-order, the other projection and the reversed utility polarity are all inhabited by other families in the same run, at the same prior cost, and belief takes none of them. It is one thing to observe that a learner did not go the other way; it is another to observe that other tasks in the same run **did**.

Third, the composition is where the content is, and the composition is what was searched for. The pair is symmetric and content-free — **dup** makes (w, w) , and nothing in it says which copy is a model or who holds it. Three independent decisions make one copy a belief: transform the second and not the first, commit from model to world and not the reverse, and publish exactly the value whose policy was run. Each has an available opposite, each opposite is exercised elsewhere, and all three are made in program space.

Fourth, and most directly, the objection is to a variant the thesis does not rest on. Remove the two primitives, as the combinator library of combinators does, and the structure comes back. The searcher is handed **dup**, **mapsnd**, **compose_gp**, **pipe_gpg**, **locate**, **place** and **register** — product-category combinators nobody would call mental — along with the wrong-channel complements **swap**, **mapfst**, **bimap** and **pair_blank**. What happens is Section 5.2.1: the non-mental families reassemble **fork** and **sync_to_world** in round 1 with no belief task solved, and the agency signature is built on top of them in round 2 with the agent value now shared three ways instead of two. If **fork** were **believe** in disguise, that run would have had to discover **believe** from scratch, and it did.

5.10.2 “But **fork** still smuggles in a representational capacity”

A weaker version of the objection concedes all of that and locates the smuggling one level down: the ability to make a copy of the world that differs from the world just **is** the representational capacity that theory of mind consists in.

The reply is that it is not, and the distinction matters. Building a world-state that differs from the actual one is counterfactual reasoning — the capacity to consider what would be the case if things were otherwise — and it is thoroughly domain-general, with a developmental record of its own well before false-belief attribution. Theory of mind is not that capacity; it is an abstraction that **uses** it, by attributing the counterfactual world to a particular agent, such that the divergent world is **that agent’s** world. That attribution is precisely the conjunct that the shared hole encodes, and it is precisely the conjunct that nothing in **fork** supplies: **fork** will happily derive a private grid, compute over it, and publish an unrelated value, and the language will let you write that. What it will not do is tell you not to.

5.10.3 “The regress does not end”

Nor do we claim it does. One can always ask for **dup** to be decomposed in turn — into allocation, reads and writes over a flat memory, so that “hold a second world model” becomes a discovered pattern of buffer use. That relocates the regress rather than ending it. “Allocate a buffer” is then the primitive, and it is neither more nor less mental than “form a pair”, because the capacity for plurality is presupposed by intensionality at every level. Intensionality just **is** the coexistence of a world and a model of it, so any substrate expressive enough to state “act on the world via a transformed model of it” must be able to hold two things at once. The pair is to theory of mind

roughly what the integers are to arithmetic: not a disguised form of the thing to be learned, but the medium without which it cannot be stated. There is no substrate in which theory of mind is discovered from nothing. There are only substrates whose primitives are, or are not, individually non-mental and general, and the argument of this section is that these ones are.

5.10.4 “The corpus was built to produce this result”

The last objection is about the curriculum rather than the language, and it has the most substance. The obstacle family is deliberately the densest corner of the corpus, at six tasks per (goal, agent) combination, because the wall-and-navigate policy has to recur often enough for compression to abstract it; belief’s derive is short only because that abstraction exists. Change the mix and the result moves.

Three things are worth saying. The first is that this is a claim about what the learner is **shown**, not about what it concludes, and it is exactly the claim Section 1.4.3 makes on the theory theory’s behalf: a concept becomes available when earlier learning has made it cheap, so the composition of earlier experience is a determinant of what is learnable and when. A model in which the curriculum did not matter would be a model in which the ordering result of Section 5.5 was vacuous. The second is that the corpus is not stratified: it is one undifferentiated bag with no mental / non-mental label anywhere in the search or the compression step, and the joint stitch of Section 4.5.2 pools both root types, so the budget that paid for belief’s structure was free to be spent on overlay, registration, obstacle or relocation instead. The third is that the dose-response experiment that would quantify this — vary the non-mental fraction, measure whether the constructor is still invented — is not in this thesis. It is the single most informative experiment left undone, and Section 5.11 says so.

5.11 What the results do not show

Seven limits, in roughly decreasing order of how much they cost the argument.

The belief has no history. What the learner finds is a divergent world model, not a belief that is formed and could be revised. Nothing in the discovered term represents how the agent came to be wrong: no perceptual access, no record of what was witnessed, no update when the agent looks again. This is a limit of the corpus and of a single-frame transition function, and it is the largest respect in which the discovered structure is thinner than the thing it is a model of. Chapter 3’s BToM has an analogous gap — the space of admissible divergences is stipulated rather than derived — but BToM’s belief is at least a state that could in principle be revised, and this one is not. Section 6.4 takes it up.

The curriculum mix is untested. As Section 5.10 concedes, the corpus-mix dose-response has not been run. Nor have the two others that would sharpen the result: a silo-versus-joint stitch, which would show whether the constructor survives being compressed against belief alone, and a seed-robustness sweep reporting every verdict metric as k out of five corpus seeds. Each of the two cells reported here is a single seed.

Omitting the recognition model was decided on the evidence, not in advance. Section 7.7.2 gives the reason for leaving DreamCoder’s third phase out, and the reason is a good one, but it is not the order in which the work happened: the loop was run both ways first, and the numbers are what settled it. Under the atomic control the dreamed proposal was a small win — 291

tasks and 159 belief tasks against 287 and 155 — concentrated in the two hardest families. Under the combinator library it was a substantial loss: 240 against 264, with the damage concentrated in the witness family, which fell from 48 of 48 to 26 of 48. The mechanism is worth recording because it is not the obvious one. The dreamed prior was gated to families with at least one solved instance, so it cannot be what delayed the first belief solve, and indeed both combinator cells made theirs in round 2. What differed was the **library**: the two cells solved slightly different sets of tasks in round 1, the joint stitch therefore saw a different corpus, and the abstractions it returned differed accordingly — the dreamed cell converging on a library through which the witness compound is longer. The deficit was transmitted by the compression step rather than by the proposal distribution, which is to say that the two interact: a proposal that changes **which** tasks are solved changes what the next library can be. That interaction is a real property of the architecture, and characterising it is left undone here. What can be said is that the constructor was invented in the same round, with the same shape, in all four cells, so the structural result does not turn on the choice either way.

Coverage is not complete, and the misses are not proven to be budget misses. Section 5.9 gives the numbers. The claim that a longer run would close them is supported by the comparison between the two endowments and by the shape of Figure 5, and it is not supported by a run. The experiment that would settle it is a budget dose-response — the same corpus at 600, 1200 and 2400 s per task, with the solve rate per belief family plotted against the budget — which would separate a family that is out of **reach** from one that is merely out of **time**.

The one-model frame is fixed by the type system, not discovered. Both endowments give the learner the product $G \times G$, which says that a learner holding a private representation holds exactly one — and a one-model frame is precisely the kind of thing a theory of mind is supposed to have. Nothing reported here shows that compression would **select** a single private channel if more were expressible, because in these runs no more are. The experiment that would settle it replaces the fixed pair with a recursive grid-stack, so that channel arity becomes a free parameter, and measures which arity each family selects; it is not in this thesis, and Section 6.3 sets it out as further work.

The planner is a single token. `optimize` packs a full breadth-first search into one symbol. Section 4.6.5 defends granting a planner, on grounds the developmental record supports, but it does not defend granting it in this form, and a more elementary treatment would decompose it into memory, conditionals and interaction (Section 6.5).

None of this is a claim about children. The model is a sufficiency result at the computational level: it shows that a learner with domain-general representational resources and a preference for short programs **can** arrive at belief-structured abstractions, which blocks the inference from “no mechanism has been named” to “no mechanism exists”. Whether human learners use this mechanism, or another one that solves the same problem, is not settled by anything in this chapter.

Chapter 6

Further work

6.1 Comparison to LLMs

I’m doing search in program-dynamics space, not token space. Synthesizing programs is logical and semantic discovery, unlike token prediction. ECD enumerates typed program trees by probability and runs them on grids; there is no surface-syntax statistics anywhere.

6.2 Polymorphism

We use a simply-typed monomorphic type system. For our purposes, types are really used only to prune enumeration. They constrain which compositions are legal, not what the program must prove. DreamCoder uses polymorphic types, so unification can rule out ill-typed branches early. But since our DSL has only ~ 30 primitives our bottleneck isn’t branching factor, so monomorphic dict lookup is sufficient.

With monomorphic types, a stitch abstraction discovered at type `grid` can only be reused at `grid`. With polymorphism, an abstraction abstracted over $\forall a$ can be reused across instantiations of any type a . The grid-stack of Section 6.3 would have to hand-engineer this, with a `cons/nil` constructor and a depth-polymorphic lift written out for each pair operation; if we had polymorphism it would just fall out of the type discipline.

To do something like that, we’d replace bare string types with a small ADT:

- `ground ('con', 'grid', [])`
- `variable ('var', 'a')`
- `applied ('con', 'pair', [t1, t2])`
- `arrows (args, ret)`

We’d implement Robinson unification

6.3 Channel arity as a free parameter

The product $G \times G$ of Section 4.3.2 carries a commitment we should not be making on the learner’s behalf. It says that a learner holding a private representation holds *exactly one*, and a one-model frame is precisely the kind of thing a theory of mind is supposed to have. If the answer to “why one world and one model, and not three?” is “because the type system says so”, then part of the structure at issue has been fixed by the modeller rather than found by the learner. Section 5.11 lists this as a limit of the results; this section says what removing it would take.

The design is a further weakening of the endowment of Section 4.4 — strictly less given, on the same corpus, with the same objective. The fixed pair is replaced by a recursive grid-stack,

```
gstack ::= () | (grid, *gstack)
```

with `nil` and `cons` as constructors, and every pair operation is replaced by a depth-polymorphic lift of itself that acts on the top of the stack. The type system of Section 4.3.2 gains the arrows $G \rightarrow G^*$, $G^* \rightarrow G^*$ and $G^* \rightarrow G$ to push, transform and project, and loses the arrows into and out of $G \times G$ that fixed the arity at two. At depth 1 the lifts reproduce `fork` and `sync_to_world` exactly, so nothing expressible under either endowment of Section 4.4 is lost; but the number of private channels a program opens is now something search selects rather than something the type system fixes.

The same monomorphism is also why the language needs a separate composer at each arrow of Section 4.3.2. The pieces belief is assembled from Δ , $\text{id} \times f$ and $\circ$ — are defined at any objects, not just at G , so the four composers of Section 4.4 are four copies of one operation the type system cannot state once. Nothing in the argument turns on this, but a language with polymorphic plumbing would collapse them into a single symbol, and the wiring belief needs would be that much less of the endowment.

The measurement is then per family: for each solved task, read off the maximum stack depth its program reaches, and compare that against the depth the family’s ground-truth program uses. The corpus would need at least one non-mental family whose ground truth genuinely requires depth 2 — a cross-channel blur, say — so that arity 2 is known to be both expressible and selectable, and the physics and desire families should sit at arity 0, holding no private channel at all. The prediction the thesis’s argument makes is that no family, mental or not, selects more than one private channel except the one whose ground truth demands two; and the quantity that would carry it is not the observation but the margin, the description length of the arity-1 encoding of a belief scene against the arity-2 encoding of the same scene, under base primitives and under the learned library.

If that came out, belief’s single-model frame would be a discovery rather than a stipulation, and the last structural commitment in Section 4.3.2 would be gone. If it did not — if compression started stacking channels wherever the grammar allowed it — what the runs of Chapter 5 found would still be a belief abstraction, but the claim that the learner also found its *frame* would have to be withdrawn.

6.4 persistent models and interaction

currently an attributed grid state, a belief, exists only for that one frame, since we’re just working with grid to grid transition functions. so do something where the agent actually has a persistent memory of its modified grid

and something where the agent can actually react to its environment, can write to the private model and read from it, can navigate based on obstacles, etc

6.5 breaking down optimize

this primitive packs in a lot, it’s a full bfs search. so try to rediscover that. this requires a persistent model (to keep track of visited cells) and a and interactions with the environemtn, conditionals (if wall, move away)

6.6 probabilistic primitives

an agent that sometimes does this and sometimes that, etc

Go from a LoT to a PLoT

6.7 vs. real world

obviously this isn't how an infant learns theory of mind

I'm missing the *self*. I'm focusing on learning to distinguish between different entities in the domain, but not an infant learning to distinguish others from itself. For example, if I bite myself it hurts, but if I bite someone else it doesn't. maybe pain occurs only if I bite this but not that (extensional). or we have different states, different experiences, and sometimes someone else can be experiencing pain even when I'm not.

I'm missing the scale. I'm focusing on a narrow timescale. in real infants this would unfold over a long time scale.

6.8 richer belief content

6.9 recursive mindreading

6.10 POMDP

observation function

Chapter 7.

Appendix

This appendix collects the implementation of the model outlined in Chapter 3.

- full list of tasks
- full list of primitives

7.1. Full Corpus

Family	Ground-truth program	What it probes
physics	step v d	the bare interpreter, no private model
desire	optimize (neg_dist gv) av	goal-directed motion without a model
obstacle	compose (wall_at r c) (optimize ...)	a detour round a <i>real</i> wall
flee	optimize (distance hv) av	the repulsion corner of the utility axis
deletion / denoise	clear_at r c / erase v	the grid-edit removal corners
relocation	compose (clear_at ...) (wall_at ...)	the grid-edit “move” corner
overlay / underlay	fork (step v d) overlay	fork with a <i>non-mental</i> commit; z-order
comet	fork (optimize ...) overlay	fork’s derive slot taking a seek policy
wipe	pipe_gpg pair_blank snd	the pairing complement: a fresh blank channel, not a copy
belief_wall	fork (compose (wall_at r c) (optimize ...)) (sync_to_world av)	detour round an <i>invisible</i> wall
belief_witness	the above, composed with a second bare seeker	a witness who crosses the phantom wall
belief_goal	fork (seq (step gv d) (optimize ...)) (sync_to_world av)	Sally-Anne: belief whose <i>content</i> varies
belief_observers	a bare seeker composed with a forked one	selective attribution: one agent forks, one does not
belief_false_obstacle	fork (seq (wall_at Wb) (step gv d) (optimize ...)) (sync_to_world av)	wrong about obstacle <i>and</i> goal, with a real wall present
registration / perception	sync_to_world v / sync_to_model v	the sync commits doing <i>non-mental</i> alignment
multi_registration / registration_except	sync_all / sync_except v	the scope complements
inpainting / readout	underlay / snd	the z-order and projection complements
composite	compose_pg (bimap (step u d) (erase w)) overlay	the full bifunctor: two layers, one map each
drift_reg	compose_pg (mapfst (step u d)) (register ...)	the world-channel map: register while the image scrolls
map_update	compose_pg swap sync_all	the twist: the model adopts the world wholesale

Table 10: The task families, by the program behind them. The first block are trajectory tasks, solved by transition functions; the last are template tasks, solved by commits (roots). Programs are written throughout in the abbreviated spelling of Section 4.4.1: fork and sync_to_world are not primitives of the learner’s library but compounds it must assemble . The last three families exist only under the combinator library; nothing in the atomic control maps both channels, maps the world channel

7.2. The interpreter

The first is compositionality. “Iteratively apply f , starting from x_0 , for n time steps” is structure common to every solution in the corpus. Were it part of the program, it would be baked into every abstraction the learner discovers: a corpus of ten falling-object scenes would yield not the reusable $\lambda 0. (\text{step down } \#0)$ but an overspecified *apply (step down #0) from x_0 for #1 steps*, whose output type is an entire rendered scene. Such an abstraction cannot be combined with another, so it could never serve as a component of the deeper compounds that belief requires, and library learning would be defeated at the first round. Keeping the recurrence out of program space keeps every abstraction an arrow $G \rightarrow G$ and therefore composable.

The second is that the recurrence carries no information the learner could be credited with discovering. Since it is shared by every solution, what search is actually looking for is only f ; the rest is what is needed to render a scene from f in order to check it against the observation. So we move the rendering machinery out of program space and into the evaluation framework. The interpreter is given in full in Section 7.2.

The interpreter is discussed at the model level in Section 7.2. What follows is the design argument in the form it was originally worked out, followed by the two interpreters themselves.

Say we have a candidate program ρ , so the enumerator needs to check for each task x in the corpus whether ρ solves task x . A program ρ solves task x iff running ρ on initial frame x_0 outputs the entire history x . So every solution should be a program that takes as input a $d \times d$ frame and returns a $d \times d \times n$ stack of frames, a program of the form: “iteratively apply transition function f , starting with initial frame x_0 , for n timesteps”. So every solution program has the same outer structure, with only f varying between programs.

This presents a problem: this shared structure, present in every program, will be baked-in to every abstraction. For example say ten tasks in a corpus depict some object falling down, so stitch should identify *(step down #0)* as a shared structure and add this “gravity” primitive to the library. But because “apply f , starting with x_0 , for n timesteps” is a structure common to all programs, stitch would add an overspecified abstraction $f_1 := \lambda 0. 1. \text{iteratively apply (step down \#0), starting with } x_0, \text{ for \#1 timesteps}$ rather than the more general $f_2 := \lambda 0. (\text{step down } \#0)$.

The problem is that f_1 doesn’t compose, which defeats the purpose of library learning. Say we abstracted f_1 as “gravity”, and we now want to use it to solve a task involving two objects both dropping to the ground. We can’t do that, because f_1 ’s output is of type `matrix`, an entire 3d scene depicting just one falling object. But if we’ve got an f_2 abstraction with output type `fn`, we can create a program like *(compose (fn_2 4) (fn_2 5))* that reuses the abstraction for each component of a scene within that same scene (where `compose` is of type `fn -> fn -> fn`).

Since “iteratively apply transition function f , starting with initial frame x_0 , for n timesteps” is common to every program, what the enumerator is looking for is really just that transition function f . The rest is just what’s needed to render a scene given a transition function, to check whether f correctly produces a given task scene. So we simply move the rendering machinery from program space to the evaluation framework.

We implement an interpreter that, given a candidate program (transition function) f , a task's initial frame x_0 , and the task's duration (z -dimension) n , generates a scene x' by applying f iteratively (starting with x_0) for n timesteps.

```
def unfold(g: grid, T: int, f: fn) -> mat:
    frames = [g.copy()]
    for _ in range(T - 1):
        g = f(g)
        frames.append(g.copy())
    return np.stack(frames)
```

`unfold` is the anamorphism that at each timestep applies f to the previous grid to generate the new grid, and appends the new grid to the list of grids. So `unfold`'s state is just a single grid: it doesn't encode a world/model grid pair, nor a registry of agents and goals. A transition function f is a grid endomorphism $\text{fn} :: \text{grid} \rightarrow \text{grid}$.

For some of the tasks, we use an interpreter for programs of root $\text{fn_p_g} :: (\text{grid}, \text{grid}) \rightarrow \text{grid}$ since those tasks don't involve just one input grid but a pair. That uses `unfold_with_template(g, template, T, c)`

```
def unfold_with_template(g, template, T, c):
    """grid, grid, int, fn_p_g -> mat: thread g; each frame pair it with a
    *constant external* template and apply commit c. Unlike fork, the second
    channel is a given input, not a privately derived model – so a program that
    uses sync_to_world here is doing registration, not holding a belief.
    """
    frames = [g.copy()]
    for _ in range(T - 1):
        g = c((g, template))
        frames.append(g.copy())
    return np.stack(frames)
```

7.3. The primitive catalogue

The primitive sets handed to the searcher are summarised at the model level in Section 4.4. The exhaustive catalogue — every symbol's repr, type signature and semantics, under each endowment — belongs here.

Symbol	Type	Semantics
compose	$(G \rightarrow G)^2 \rightarrow (G \rightarrow G)$	run one transition then the other
step	$V \times D \rightarrow (G \rightarrow G)$	move every cell of the given value one step in the direction
optimize	$U \times V \rightarrow (G \rightarrow G)$	move the given value one greedy (BFS-optimal) step to improve the utility
neg_dist	$V \rightarrow U$	minus the distance to the nearest cell of the target value (attract)
distance	$V \rightarrow U$	plus that distance, so maximising it flees the target
wall_at	$C \times C \rightarrow (G \rightarrow G)$	stamp a wall at a position
clear_at	$C \times C \rightarrow (G \rightarrow G)$	clear one cell
erase	$V \rightarrow (G \rightarrow G)$	remove every cell of a value
right left up down	D	the four movement directions
c0 ... c4	C	coordinate terminals, a pair naming a position (the invisible latent pool)
0 ... 9	V	cell-value terminals (content-priced when visible)

Table 11: The grid core: the primitives that move cell values about a single grid, shared by both endowments.

Symbol	Type	Semantics
<code>dup</code>	$G \rightarrow G \times G$	Δ : pair a grid with a copy of itself
<code>mapsnd</code>	$(G \rightarrow G) \rightarrow (G \times G \rightarrow G \times G)$	$\text{id} \times f$: act on the second channel
<code>compose_gp</code>	$(G \rightarrow G \times G) \times (G \times G \rightarrow G \times G) \rightarrow (G \rightarrow G \times G)$	compose a pair producer with a pair endomorphism
<code>pipe_gpg</code>	$(G \rightarrow G \times G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$	produce a pair, then consume it
<code>locate</code>	$V \rightarrow (G \rightarrow C \times C)$	loc_v : where v is in a grid
<code>place</code>	$V \rightarrow (G \times (C \times C) \rightarrow G)$	put_v : move v to a position
<code>register</code>	$(G \rightarrow C \times C) \times (G \times (C \times C) \rightarrow G) \rightarrow (G \times G \rightarrow G)$	$(w, m) \mapsto \text{plc}(w, \text{loc}(m))$: read a position off one channel, impose it on the other
<code>overlay</code>	$G \times G \rightarrow G$	union the channels, model wins ties — a <i>non-mental</i> commit
<code>underlay</code>	$G \times G \rightarrow G$	union the channels, world wins ties
<code>sync_all</code>	$G \times G \rightarrow G$	move <i>every</i> shared value to its model position
<code>sync_except</code>	$V \rightarrow (G \times G \rightarrow G)$	every shared value but one
<code>fst / snd</code>	$G \times G \rightarrow G$	project the world channel / the model channel
<code>swap</code>	$G \times G \rightarrow G \times G$	the twist: exchange the channels
<code>mapfst</code>	$(G \rightarrow G) \rightarrow (G \times G \rightarrow G \times G)$	$f \times \text{id}$: act on the <i>first</i> channel
<code>bimap</code>	$(G \rightarrow G)^2 \rightarrow (G \times G \rightarrow G \times G)$	$f \times g$: act on both
<code>pair_blank</code>	$G \rightarrow G \times G$	pair a grid with a fresh blank rather than a copy
<code>compose_pg</code>	$(G \times G \rightarrow G \times G) \times (G \times G \rightarrow G) \rightarrow (G \times G \rightarrow G)$	transform a pair, then consume it — the third composition, and the only one that acts on a pair the program did not build

Table 12: The product combinators: the plumbing, the commits that consume a pair, and the symmetric complements. The lower rows are the complements discussed in Section 4.4.

TODO: the atomic variant’s pair interface, transcribed from `prims.py`.

7.4. Program representation

The hypothesis space is described at the model level in `!types`. This is the data structure that represents it.

Every program, primitive, and invented abstraction is a `Delta`, i.e. an object that can serve as a node in an expression tree. A `Delta` carries a `head` (the Python value or callable it denotes), a `type` (the type its subtree returns), a list of `tailtypes` (the types of the arguments it expects), and a list of `tails` (the actual argument subtrees, once they are filled in). So a `Delta` with no `tailtypes` is a *terminal*, a leaf whose value is just its head, for example `up :: dir` or the literal `4 :: cellvalue`. A

Delta with **tailtypes** is an *operator*: an interior node like `step :: cellvalue, dir -> fn`, whose head is applied to its filled-in tails. Then a node’s **arrow** is the signature that the enumerator matches against when it fills argument slots. For a terminal, its arrow is just its **type**. For an operator, it’s the pair (**tailtypes**, **type**), the arrow from its input types to its output types.

The expression tree is directly executable. Calling a **Delta** evaluates the program it represents, so a terminal returns its head while an operator recursively evaluates each of its tails and applies its head to the results. So the program (`step 4 up`) is a **step**-headed node with tails `4` and `up`; calling it evaluates the two leaves to the cell value `4` and the direction `(-1,0)`, then applies **step** to obtain a transition function `fn :: grid -> grid`.

An invented abstraction is represented as a **Delta** as well, but it’s distinguished by a **hiddentail** which is the body of the abstraction. The **hiddentail** is itself a **Delta** tree over lower-level primitives, with numbered argument holes `$0`, `$1`, ... standing in for the abstraction’s parameters. When such a node is called, its body is copied and each hole `$i` is replaced by the corresponding tail before the body is evaluated. This is what makes an abstraction cost a single token. The whole subtree it abbreviates is hidden inside the **hiddentail** and does not appear in the program that uses it.

The description length of a program is its node count, computed by **length**. a leaf counts one, and an operator counts one plus the lengths of its tails. So an invented abstraction used as a token adds just one, since its body lives in the un-charged **hiddentail**. This node count is the minimum-description-length quantity the enumerator uses to tiebreak between programs that solve a task within the same budget window (see `!enumeration`).

7.4.1. The library object

The library of primitives is a **Deltas** object. It is instantiated with a list of **core** primitives and grows a parallel list of **invented** abstractions as compression discovers them. When the enumerator draws primitives, it draws from the concatenation of **core** and **invented** which is the full library. Whenever a primitive is added, **Deltas.infer** rebuilds the indices the search depends on — most importantly **bytype**, which groups the alphabet’s symbols by their return type. This grouping is what the type-uniform prior is computed from: a symbol of type τ is priced at $-\ln$ of the number of symbols sharing type τ , so **bytype** gives both the branching factor at each argument slot and the cost of each choice (`!enumeration`).

7.5. Task generators and necessity filters

The identifiability condition on the mental families is stated in identifiability beyond the k scenes, two conditions on the scene itself and a set of exemptions that no scene can settle. This section states them and says how the generators enforce them. Both conditions are sharpest in the false-wall family, where the agent detours around a wall that is not there.

The falsely believed wall must matter. Placed nowhere near the agent’s route, it leaves the agent navigating to its goal exactly as it would have without any false belief; positing the belief is then neither necessary nor helpful, and the scene shows nothing. So the phantom wall lies on the shortest path the agent would take were its belief true, and costs it extra steps over that path. A merely monotone “detour” is just another shortest path, which plain desire reproduces under different tie-breaking. Scenes failing this are rejected by the generator.

A world-level stamp must be detectable. This one is structural. On a bare grid nothing distinguishes a wall the agent *believes* in from a real wall stamped into the world and taken back down before the frame is rendered: a rival that stamps, acts, then erases reproduces every scene honestly, at every k . The scenes must therefore make the stamp visible, and they do so by leaving an inert object sitting on the phantom-wall cell for the whole trajectory — the cell the agent bends around in Stamping a wall there overwrites that object, and nothing in the language can put it back, so a world-level stamp shows up in the rendered frame. The witness family makes the same argument with a moving object in place of a static one: an agent who walks across the phantom cell would be clobbered by any real wall stamped there.

Some rivals are identities, and are exempted by argument. A last few rivals cannot be excluded by the scenes at all, because what they are is an *identity* rather than a coincidence, and no number of scenes separates two programs that denote the same function on the scenes in question. On a two-value scene, “move every shared value except the goal to its model position” simply *is* “move the agent to its model position”. These are exempted from the rival census of Section 5.4.3 by argument, and the full set is:

7.6. Cost-bounded enumeration

The search is described at the model level in !enumeration. These are its mechanics.

While a single cost-ordered stream can in principle be checked against every task in the corpus at once, in the experiments each still-unsolved task is instead enumerated as its own cost-ordered stream, parallelized across tasks. The prior Q each stream runs under is the same one: the type-uniform prior over the current library, with the content-literal boost of !enumeration. Nothing in the loop conditions Q on the individual scene (!no-amortization).

Program search proceeds by budget window to ensure that cheaper programs are enumerated first. For (**step 4 up**), the first two windows are skipped by pruning. We start enumerating in window $[0, 2)$, but since **step**’s cost is 2.1 nats it exceeds the budget, so it cannot be chosen as the root node of the program tree and we proceed to budget window $[2, 4)$. After paying 2.1 nats for **step**, only 1.9 nats remain to fill the two holes in the partial program tree. **step**’s tail types (arguments) are **cellvalue** and **step**. But any primitive of type **cellvalue** costs 2.3 nats, so **step**’s first argument can’t be placed. So trees with a **step** node are pruned, since they would never complete within the second budget window. Search is cheapest-first, since more complex programs only become affordable once the window is wide enough.

At budget window $[4, 6)$ the ceiling is finally sufficiently wide to admit all three symbols. The tree is assembled incrementally: each enumerated subtree is appended to a copy of its parent node, and a callback fills the remaining argument slots by the same recursion (so programs of any arity or nesting depth are handled by the same procedure). Since enumeration is type-directed, each argument slot draws candidates only from symbols of the matching type, so no ill-typed partial tree is ever constructed. (**step 4 up**) completes at its total cost of ≈ 5.8 nats.

When (**step 4 up**) is found the tree rooted by **step** has no more available holes, so the callback fires to evaluate the found program ρ . For each task x in the corpus, the interpreter generates the output matrix given the input grid of the task by calling **unfold** on ρ , initial grid x_0 , and x ’s

third dimension n . Then the resulting output matrix x' is hashed and compared against task x 's hash. So for a program to solve a task, it must produce a grid that exactly matches that task's entire trajectory, not just the endpoint of the trajectory. This ensures that programs capture the full behavior rather than just the end result.

If (step 4 up) is the first solution found for the task, it's stored as the task's solution. Because the windows are tried in increasing order of cost, the first solution the search returns is in principle the shortest program that solves the task. But enumeration is parallelized so search unfolds simultaneously over program trees, so we need to tiebreak when multiple programs solve a task within the same budget window. To tiebreak we first run `Delta.simplify` on the programs, a semantics-preserving rewrite that collapses spurious nesting which sometimes occurs during enumeration. We then run `Delta.length` to get the node count of the expression trees we need to compare. Every operator and leaf adds to the count, and an invented abstraction used as a token adds one to the count too. The body of the abstraction isn't charged, since the components now count for just application. The program with the lower node count wins since it has the lower minimum description length (since by symbol count it's shorter).

Also, the search space contains lots of junk programs that crash or return nothing even though they type-check. So the enumerator callback wraps both the interpreter and the evaluator in `try/except` blocks and skips any degenerate or non-callable program.

7.7. Compression with Stitch

The compression objective and the tradeoff it makes are given in `!compression`. This is the search that optimizes it.

In DreamCoder (Ellis et al., 2020) the compression step is bottom-up: for each found program it enumerates the exponential set of program refactorings (stored as version spaces) and intersects them to grow abstractions up from reoccurring subexpressions. Rather than this bottom-up approach, we delegate the search for abstractions to the top-down Stitch (Bowers et al., 2023) library-learning system. Instead of enumerating refactorings of every program, stitch searches the space of abstractions directly. So stitch doesn't have to materialize every rewrite.

To create an abstraction, stitch starts with a tree that consists of just a root node `??`, an unexpanded hole that the search still needs to refine. At the root `??`, the abstraction *matches* every subexpression, since it's the most general abstraction possible: any subexpression can be trivially "rewritten"

. As the tree is built up and the search is refined, the set of possible match locations shrinks.

A node of the stitch abstraction search tree is a partial abstraction. It consists of some combination of primitive operations (like `step`) and argument holes `#i`, and at least one unexpanded hole `??` that the search still needs to refine.

At each step it commits part of the abstraction's body to a concrete operator or splits a subtree, branching downward toward more specific abstractions.

upper bound for a partial abstraction $A_{??}$ (Bowers et al., 2023)

$$U_{\text{upperbound}}(A_{??}) = \sum_{e \in \text{matches}(A_{??})} \text{size}(e)$$

goal of compression is to minimize the size (cost) of program corpus after rewrite. Compressive utility function:

Each compression phase consists of a fixed number of stitch iterations, determined by the `iterations` parameter. Each iteration searches for the single most compressive abstraction, rewrites the corpus in terms of it, and repeats, so later abstractions may be built out of earlier ones. The `max_arity` parameter bounds how many holes (`#0`, `#1`, ...) an abstraction may carry.

7.7.1. Normalization before compression

Before passing the found solutions to stitch for abstraction, we first need to fully expand any existing primitives in those solutions. For this we implement `normalize`, the complementary operation to abstraction. It fully inlines every invented `fn_k` back to the bare primitive alphabet, recursively substituting each abstraction’s argument holes with its actual arguments until nothing but core primitives remain.

We need to normalize because stitch’s top-down compression can only abstract over structure it can see, and an opaque `fn_k` token has no visible interior. So any abstraction that would need to reach inside `fn_k`, or span its boundary and surrounding context, is simply invisible to the search.

the compression step must be handed programs in a common vocabulary — feeding it already-compressed programs, in which a prior round’s `fn_0` appears as an opaque token, would let those tokens collide with the fresh abstractions the compressor invents under the same names. Normalizing before compression guarantees the compressor sees only primitives and the structure it discovers is genuinely its own. Renaming it to `inv_0` avoids the lexical clash but leaves it just as opaque; stitch still can’t see the `(compose (wall_at ...) (optimize ...))` sitting inside it. Inlining flattens the token back to that primitive structure, and only then can the compressor find abstractions that cut across the old boundaries.

The best syntactic fragment to abstract from round N ’s solution may carve the primitives differently than round $N + 1$ ’s. For example,

So we can refactor abstractions that were previously ideal but are no longer ideal

7.7.2. No recognition model

DreamCoder’s third phase trains a neural recognition model that reads a task and emits a task-conditional prior $Q(\cdot | x)$, so that enumeration proceeds under a Q biased toward the primitives. We don’t implement it.

The reason is a difference of scale between DreamCoder’s setting and this one. Amortization earns its place when the branching factor is large and the loop is long: DreamCoder works with libraries of hundreds of symbols over dozens of iterations, and there the cost of one forward pass per task is repaid many times over by the cost bands it lets the enumerator skip. Ours is a small language run for a few rounds. The learner starts with 37 primitives under the atomic control and 45 under the combinator library, and ends with 43 and 51 once compression has added its six; and the abstraction the whole argument turns on enters at the first compression step under the one endowment and the

second under the other, with the loop converging a round or two later (Section 5.5). There is no long tail of iterations across which a learned proposal could amortize anything. What such a proposal would buy is reaching sooner what is already reachable, and reachability, not speed, is the quantity the argument of Section 2.3 depends on.

Omitting the phase also removes it as a place where the result could be smuggled in. A recognition model trained on solved tasks is a component whose prior over the library is fit to data, and belief is precisely the structure the corpus is meant to leave unfitted. With no such component, no distribution anywhere in the loop is conditioned on what a scene looks like, and the audit of Section 4.6 has one fewer item to defend.

This is a departure from DreamCoder, and it is one we arrived at by measurement rather than by design: the loop was run both ways before it was settled, and Section 5.11 reports what each way gave.

Bibliography

- Apperly, I. A., & Butterfill, S. A. (2009). Do humans have two systems to track beliefs and belief-like states?. *Psychological Review*, 116(4), 953–970. <https://doi.org/10.1037/a0016923>
- Baker, C. L., Jara-Ettinger, J., Saxe, R., & Tenenbaum, J. B. (2017). Rational quantitative attribution of beliefs, desires and percepts in human mentalizing. *Nature Human Behaviour*, 1(4), 64. <https://doi.org/10.1038/s41562-017-0064>
- Baker, C. L., Saxe, R., & Tenenbaum, J. B. (2009). Action understanding as inverse planning. *Cognition*, 113(3), 329–349. <https://doi.org/10.1016/j.cognition.2009.07.005>
- Baron-Cohen, S., Leslie, A. M., & Frith, U. (1985). Does the autistic child have a “theory of mind” ?. *Cognition*, 21(1), 37–46. [https://doi.org/10.1016/0010-0277\(85\)90022-8](https://doi.org/10.1016/0010-0277(85)90022-8)
- Barone, P., Corradi, G., & Gomila, A. (2019). Infants' performance in spontaneous-response false belief tasks: A review and meta-analysis. *Infant Behavior and Development*, 57, 101350. <https://doi.org/10.1016/j.infbeh.2019.101350>
- Baum, E. B. (2004). *What is thought?*. MIT Press.
- Beaudoin, C., Leblanc, É., Gagner, C., & Beauchamp, M. H. (2020). Systematic Review and Inventory of Theory of Mind Measures for Young Children. *Frontiers in Psychology*, 10, 2905. <https://doi.org/10.3389/fpsyg.2019.02905>
- Berthiaume, V. G., Shultz, T. R., & Onishi, K. H. (2013). A constructivist connectionist model of transitions on false-belief tasks. *Cognition*, 126(3), 441–458. <https://doi.org/10.1016/j.cognition.2012.11.005>
- Berwick, R. C., Pietroski, P., Yankama, B., & Chomsky, N. (2011). Poverty of the Stimulus Revisited. *Cognitive Science*, 35(7), 1207–1242. <https://doi.org/10.1111/j.1551-6709.2011.01189.x>
- Bowers, M., Olausson, T. X., Wong, L., Grand, G., Tenenbaum, J. B., Ellis, K., & Solar-Lezama, A. (2023). Top-Down Synthesis for Library Learning. *Proceedings of the ACM on Programming Languages*, 7(POPL), 1182–1213. <https://doi.org/10.1145/3571234>
- Bruner, J. S., Goodnow, J. J., & Austin, G. A. (1986). *A study of thinking*. Transaction Books.
- Buttelmann, D., Carpenter, M., & Tomasello, M. (2009). Eighteen-month-old infants show false belief understanding in an active helping paradigm. *Cognition*, 112(2), 337–342. <https://doi.org/10.1016/j.cognition.2009.05.006>
- Butterfill, S. A., & Apperly, I. A. (2013). How to Construct a Minimal Theory of Mind. *Mind & Language*, 28(5), 606–637. <https://doi.org/10.1111/mila.12036>
- Carey, S. (2011). *The Origin of Concepts*. Oxford University Press USA - OSO.
- Chater, N., & Vitányi, P. (2003). Simplicity: a unifying principle in cognitive science?. *Trends in Cognitive Sciences*, 7(1), 19–22. [https://doi.org/10.1016/S1364-6613\(02\)00005-0](https://doi.org/10.1016/S1364-6613(02)00005-0)
- Chomsky, N. (1975). *Reflections on Language*. Pantheon Books. <https://books.google.nl/books?id=R78kAQAAMAAJ>
- Chomsky, N. (2014). *Aspects of the Theory of Syntax* (50th ed). MIT Press.

- Church, A. (1936). An Unsolvable Problem of Elementary Number Theory. *American Journal of Mathematics*, 58(2), 345. <https://doi.org/10.2307/2371045>
- Churchland, P. M. (1981). Eliminative Materialism and the Propositional Attitudes. *The Journal of Philosophy*, 78(2), 67. <https://doi.org/10.2307/2025900>
- Coşkun, K., Kara, C., & Coskun, M. (2020). New Scaling of Theory of Mind Tasks: Where Can the Intentional Aspect Be Scaled?. *Activitas Nervosa Superior*, 62(3), 78–87. <https://doi.org/10.1007/s41470-020-00072-2>
- Crane, T. (n.d.). *The Mechanical Mind: A Philosophical Introduction to Minds, Machines and Mental Representation, Second Edition*.
- Csibra, G., & Gergely, G. (2007). ‘Obsessed with goals’: Functions and mechanisms of teleological interpretation of actions in humans. *Acta Psychologica*, 124(1), 60–78. <https://doi.org/10.1016/j.actpsy.2006.09.007>
- Dechter, E., Malmaud, J., Adams, R. P., & Tenenbaum, J. B. (n.d.). *Bootstrap learning via modular concept discovery*.
- Dennett, D. C. (2002). *The Intentional Stance* (8. pr.). MIT Press.
- Downes, S. M. (1999). Can Scientific Development and Children's Cognitive Development Be the Same Process?. *Philosophy of Science*, 66(4), 565–578. <https://doi.org/10.1086/392755>
- Dörrenberg, S., Rakoczy, H., & Liszkowski, U. (2018). How (not) to measure infant Theory of Mind: Testing the replicability and validity of four non-verbal measures. *Cognitive Development*, 46, 12–30. <https://doi.org/10.1016/j.cogdev.2018.01.001>
- Ellis, K., Wong, C., Nye, M., Sable-Meyer, M., Cary, L., Morales, L., Hewitt, L., Solar-Lezama, A., & Tenenbaum, J. B. (2020, June). *DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning*. arXiv. <https://doi.org/10.48550/arXiv.2006.08381>
- Feldman, J. (2000). Minimization of Boolean complexity in human concept learning. *Nature*, 407(6804), 630–633. <https://doi.org/10.1038/35036586>
- Flener, P., & Schmid, U. (2008). An introduction to inductive programming. *Artificial Intelligence Review*, 29(1), 45–62. <https://doi.org/10.1007/s10462-009-9108-7>
- Fodor, J. (1992). A theory of the child's theory of mind. *Cognition*, 44(3), 283–296. [https://doi.org/10.1016/0010-0277\(92\)90004-2](https://doi.org/10.1016/0010-0277(92)90004-2)
- Fodor, J. A. (1998). *Concepts: Where Cognitive Science Went Wrong* (1st ed.). Oxford University PressOxford. <https://doi.org/10.1093/0198236360.001.0001>
- Fodor, J. A. (2010). *The Language of Thought* (Digit. repr). Harvard Univ. Pr.
- Fodor, J. A., & Pylyshyn, Z. W. (n.d.). *Connectionism and cognitive architecture: A critical analysis*.
- Frank, M. C., Goodman, N. D., & Tenenbaum, J. B. (2009). Using Speakers' Referential Intentions to Model Early Cross-Situational Word Learning. *Psychological Science*, 20(5), 578–585. <https://doi.org/10.1111/j.1467-9280.2009.02335.x>
- Gelman, A. (1995). *Bayesian Data Analysis* (First edition). Chapman, Hall/CRC.

- Gergely, G., & Csibra, G. (2003). Teleological reasoning in infancy: the naïve theory of rational action. *Trends in Cognitive Sciences*, 7(7), 287–292. [https://doi.org/10.1016/S1364-6613\(03\)00128-1](https://doi.org/10.1016/S1364-6613(03)00128-1)
- Gergely, G., Nádasdy, Z., Csibra, G., & Bíró, S. (1995). Taking the intentional stance at 12 months of age. *Cognition*, 56(2), 165–193. [https://doi.org/10.1016/0010-0277\(95\)00661-H](https://doi.org/10.1016/0010-0277(95)00661-H)
- Goldman, A. I. (2006). *Simulating minds: the philosophy, psychology, and neuroscience of mindreading*. Oxford University press.
- Goodman, N. (1983). *Fact, fiction, and forecast* (4. ed). Harvard Univ. Pr.
- Goodman, N. D., Baker, C. L., Bonawitz, E. B., Mansinghka, V. K., Gopnik, A., Wellman, H., Schulz, L., & Tenenbaum, J. B. (n.d.). *Intuitive Theories of Mind: A Rational Approach to False Belief*.
- Goodman, N. D., Tenenbaum, J. B., & Gerstenberg, T. (2014). *Concepts in a Probabilistic Language of Thought*.
- Goodman, N. D., Tenenbaum, J. B., Feldman, J., & Griffiths, T. L. (2008). A Rational Analysis of Rule-Based Concept Learning. *Cognitive Science*, 32(1), 108–154. <https://doi.org/10.1080/03640210701802071>
- Goodman, N. D., Ullman, T. D., & Tenenbaum, J. B. (2011). Learning a theory of causality. *Psychological Review*, 118(1), 110–119. <https://doi.org/10.1037/a0021336>
- Gopnik, A. (1997). *Words, thoughts, and theories*. MIT Press.
- Gopnik, A. (2012). Scientific Thinking in Young Children: Theoretical Advances, Empirical Research, and Policy Implications. *Science*, 337(6102), 1623–1627. <https://doi.org/10.1126/science.1223416>
- Gopnik, A., & Tenenbaum, J. B. (2007). Bayesian networks, Bayesian learning and cognitive development. *Developmental Science*, 10(3), 281–287. <https://doi.org/10.1111/j.1467-7687.2007.00584.x>
- Gopnik, A., & Wellman, H. M. (1992). Why the Child's Theory of Mind Really Is a Theory. *Mind & Language*, 7(1–2), 145–171. <https://doi.org/10.1111/j.1468-0017.1992.tb00202.x>
- Gopnik, A., & Wellman, H. M. (2012). Reconstructing constructivism: Causal models, Bayesian learning mechanisms, and the theory theory. *Psychological Bulletin*, 138(6), 1085–1108. <https://doi.org/10.1037/a0028044>
- Gordon, R. M. (1986). Folk Psychology as Simulation. *Mind & Language*, 1(2), 158–171. <https://doi.org/10.1111/j.1468-0017.1986.tb00324.x>
- Griffiths, T. L., Chater, N., & Tenenbaum, J. B. (2024). *Bayesian models of cognition: reverse engineering the mind*. The MIT Press.
- Griffiths, T. L., Chater, N., Kemp, C., Perfors, A., & Tenenbaum, J. B. (2010). Probabilistic models of cognition: exploring representations and inductive biases. *Trends in Cognitive Sciences*, 14(8), 357–364. <https://doi.org/10.1016/j.tics.2010.05.004>

- Griffiths, T. L., Sobel, D. M., Tenenbaum, J. B., & Gopnik, A. (2011). Bayes and Blickets: Effects of Knowledge on Causal Induction in Children and Adults: Cognitive Science. *Cognitive Science*, 35(8), 1407–1455. <https://doi.org/10.1111/j.1551-6709.2011.01203.x>
- Grünwald, P. D. (2007). *The Minimum Description Length Principle*. MIT Press.
- Gulwani, S., Polozov, O., & Singh, R. (2017). *Program synthesis* (Issue 4.2017, 1-2). Now Publishers.
- Heider, F., & Simmel, M. (1944). An Experimental Study of Apparent Behavior. *The American Journal of Psychology*, 57(2), 243. <https://doi.org/10.2307/1416950>
- Heyes, C. M. (2018). *Cognitive gadgets: the cultural evolution of thinking*. Harvard University press.
- Heyes, C. M., & Frith, C. D. (2014). The cultural evolution of mind reading. *Science*, 344(6190), 1243091. <https://doi.org/10.1126/science.1243091>
- Jara-Ettinger, J. (2019). Theory of mind as inverse reinforcement learning. *Current Opinion in Behavioral Sciences*, 29, 105–110. <https://doi.org/10.1016/j.cobeha.2019.04.010>
- Jara-Ettinger, J., Gweon, H., Schulz, L. E., & Tenenbaum, J. B. (2016). The Naïve Utility Calculus: Computational Principles Underlying Commonsense Psychology. *Trends in Cognitive Sciences*, 20(8), 589–604. <https://doi.org/10.1016/j.tics.2016.05.011>
- Jaynes, E. T., & Bretthorst, G. L. (2003). *Probability theory: the logic of science*. Cambridge university press.
- Kemp, C., & Tenenbaum, J. B. (2008). The discovery of structural form. *Proceedings of the National Academy of Sciences*, 105(31), 10687–10692. <https://doi.org/10.1073/pnas.0802631105>
- Kemp, C., Perfors, A., & Tenenbaum, J. B. (2007). Learning overhypotheses with hierarchical Bayesian models. *Developmental Science*, 10(3), 307–321. <https://doi.org/10.1111/j.1467-7687.2007.00585.x>
- Kemp, C., Tenenbaum, J. B., Niyogi, S., & Griffiths, T. L. (2010). A probabilistic model of theory formation. *Cognition*, 114(2), 165–196. <https://doi.org/10.1016/j.cognition.2009.09.003>
- Kulke, L., Von Duhn, B., Schneider, D., & Rakoczy, H. (2018). Is Implicit Theory of Mind a Real and Robust Phenomenon? Results From a Systematic Replication Study. *Psychological Science*, 29(6), 888–900. <https://doi.org/10.1177/0956797617747090>
- Lake, B. M., Ullman, T. D., Tenenbaum, J. B., & Gershman, S. J. (2016, November). *Building Machines That Learn and Think Like People*. arXiv. <https://doi.org/10.48550/arXiv.1604.00289>
- Leslie, A. M. (n.d.). *Pretense and Representation: The Origins of "Theory of Mind"*.
- Leslie, A. M. (1994). Pretending and believing: issues in the theory of ToMM. *Cognition*, 50(1–3), 211–238. [https://doi.org/10.1016/0010-0277\(94\)90029-9](https://doi.org/10.1016/0010-0277(94)90029-9)
- Leslie, A. M. (2005). Developmental parallels in understanding minds and bodies. *Trends in Cognitive Sciences*, 9(10), 459–462. <https://doi.org/10.1016/j.tics.2005.08.002>
- Li, M., & Vitányi, P. (2019). *An Introduction to Kolmogorov Complexity and Its Applications*. Springer International Publishing. <https://doi.org/10.1007/978-3-030-11298-1>
- Liang, P., Jordan, M. I., & Klein, D. (n.d.). *Learning Programs: A Hierarchical Bayesian Approach*.

- Lieder, F., & Griffiths, T. L. (2020). Resource-rational analysis: Understanding human cognition as the optimal use of limited computational resources. *Behavioral and Brain Sciences*, 43, e1. <https://doi.org/10.1017/S0140525X1900061X>
- Luo, Y., & Baillargeon, R. (2010). Toward a Mentalistic Account of Early Psychological Reasoning. *Current Directions in Psychological Science*, 19(5), 301–307. <https://doi.org/10.1177/0963721410386679>
- MacKay, D. J. C. (n.d.). *Information Theory, Inference, and Learning Algorithms*.
- Margolis, E., Samuels, R., & Stich, S. P. (Eds.). (2012). *The Oxford handbook of philosophy of cognitive science*. Oxford University Press. <https://doi.org/10.1093/oxfordhb/9780195309799.001.0001>
- Marr, D. (2010). *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. The MIT Press. <https://doi.org/10.7551/mitpress/9780262514620.001.0001>
- McClelland, J. L. (2010). Emergence in Cognitive Science. *Topics in Cognitive Science*, 2(4), 751–770. <https://doi.org/10.1111/j.1756-8765.2010.01116.x>
- Milligan, K., Astington, J. W., & Dack, L. A. (2007). Language and Theory of Mind: Meta-Analysis of the Relation Between Language Ability and False-belief Understanding. *Child Development*, 78(2), 622–646. <https://doi.org/10.1111/j.1467-8624.2007.01018.x>
- Newcombe, N. S. (2011). What Is Neoconstructivism?. *Child Development Perspectives*, 5(3), 157–160. <https://doi.org/10.1111/j.1750-8606.2011.00180.x>
- Newell, A., Shaw, J. C., & Simon, H. A. (1958). Elements of a theory of human problem solving. *Psychological Review*, 65(3), 151–166. <https://doi.org/10.1037/h0048495>
- Onishi, K. H., & Baillargeon, R. (2005). Do 15-Month-Old Infants Understand False Beliefs?. *Science*, 308(5719), 255–258. <https://doi.org/10.1126/science.1107621>
- Palmarini, A. B., Lucas, C. G., & Siddharth, N. (2024, May). *Bayesian Program Learning by Decompiling Amortized Knowledge*. arXiv. <https://doi.org/10.48550/arXiv.2306.07856>
- Perfors, A., Tenenbaum, J. B., & Regier, T. (2011). The learnability of abstract syntactic principles. *Cognition*, 118(3), 306–338. <https://doi.org/10.1016/j.cognition.2010.11.001>
- Perfors, A., Tenenbaum, J. B., & Wonnacott, E. (2010). Variability, negative evidence, and the acquisition of verb argument constructions. *Journal of Child Language*, 37(3), 607–642. <https://doi.org/10.1017/S0305000910000012>
- Perner, J. (1991). *Understanding the Representational Mind*. MIT Press.
- Perner, J., & Wimmer, H. (1985). “John thinks that Mary thinks that...” attribution of second-order beliefs by 5- to 10-year-old children. *Journal of Experimental Child Psychology*, 39(3), 437–471. [https://doi.org/10.1016/0022-0965\(85\)90051-7](https://doi.org/10.1016/0022-0965(85)90051-7)
- Piantadosi, S. T., Goodman, N. D., Ellis, B. A., & Tenenbaum, J. B. (n.d.). *A Bayesian Model of the Acquisition of Compositional Semantics*.

- Piantadosi, S. T., Tenenbaum, J. B., & Goodman, N. D. (2012). Bootstrapping in a language of thought: A formal model of numerical concept learning. *Cognition*, 123(2), 199–217. <https://doi.org/10.1016/j.cognition.2011.11.005>
- Premack, D., & Woodruff, G. (1978). Does the chimpanzee have a theory of mind?. *Behavioral and Brain Sciences*, 1(4), 515–526. <https://doi.org/10.1017/S0140525X00076512>
- Quine, W. V. (2013). *Word and object* (New ed). MIT Press.
- Rakoczy, H. (2022). Foundations of theory of mind and its development in early childhood. *Nature Reviews Psychology*, 1(4), 223–235. <https://doi.org/10.1038/s44159-022-00037-z>
- Rissanen, J. (1978). Modeling by shortest data description. *Automatica*, 14(5), 465–471. [https://doi.org/10.1016/0005-1098\(78\)90005-5](https://doi.org/10.1016/0005-1098(78)90005-5)
- Ruffman, T. (2014). To belief or not belief: Children’s theory of mind. *Developmental Review*, 34(3), 265–293. <https://doi.org/10.1016/j.dr.2014.04.001>
- Rule, J. S. (n.d.). *Pre ‘cis of The child as hacker: Building more human-like models of learning*.
- Rumelhart, D. E. (1986). *Parallel Distributed Processing: Explorations in the Microstructure of Cognition: Foundations* (Issue v.1). MIT Press.
- Schmid, U., Kitzelmann, E., & Plasmeijer, R. (n.d.). *AAIP 2009 Proceedings of the ACM SIGPLAN Workshop*.
- Schmidt, L. A. (n.d.). *Meaning and compositionality as statistical induction of categories and constraints*.
- Scholl, B. J., & Leslie, A. M. (2001). Minds, Modules, and Meta-Analysis. *Child Development*, 72(3), 696–701. <https://doi.org/10.1111/1467-8624.00308>
- Schulz, L. (2012). The origins of inquiry: inductive inference and exploration in early childhood. *Trends in Cognitive Sciences*, 16(7), 382–389. <https://doi.org/10.1016/j.tics.2012.06.004>
- Sellars, W. (1956). *EMPIRICISM AND THE PHILOSOPHY OF MIND*.
- Shannon, C. E. (n.d.). *A Mathematical Theory of Communication*.
- Siskind, J. M. (n.d.). *A computational study of cross-situational techniques for learning word-to-meaning mappings*.
- Solomonoff, R. (1964). A formal theory of inductive inference. Part I. *Information and Control*, 7(1), 1–22. [https://doi.org/10.1016/S0019-9958\(64\)90223-2](https://doi.org/10.1016/S0019-9958(64)90223-2)
- Southgate, V., Senju, A., & Csibra, G. (2007). Action Anticipation Through Attribution of False Belief by 2-Year-Olds. *Psychological Science*, 18(7), 587–592. <https://doi.org/10.1111/j.1467-9280.2007.01944.x>
- Spelke, E. S. (2022). *What babies know: core knowledge and composition, volume 1* (First Edition). Oxford University Press. <https://doi.org/10.1093/oso/9780190618247.001.0001>
- Spelke, E. S., & Kinzler, K. D. (2007). Core knowledge. *Developmental Science*, 10(1), 89–96. <https://doi.org/10.1111/j.1467-7687.2007.00569.x>

- Stuhlmüller, A., & Goodman, N. (2014). Reasoning about reasoning by nested conditioning: Modeling theory of mind with probabilistic programs. *Cognitive Systems Research*, 28, 80–99. <https://doi.org/10.1016/j.cogsys.2013.07.003>
- Tenenbaum, J. B., & Griffiths, T. L. (2001). Generalization, similarity, and Bayesian inference. *Behavioral and Brain Sciences*, 24(4), 629–640. <https://doi.org/10.1017/S0140525X01000061>
- Tenenbaum, J. B., Griffiths, T. L., & Kemp, C. (2006). Theory-based Bayesian models of inductive learning and reasoning. *Trends in Cognitive Sciences*, 10(7), 309–318. <https://doi.org/10.1016/j.tics.2006.05.009>
- Tenenbaum, J. B., Kemp, C., Griffiths, T. L., & Goodman, N. D. (2011). How to Grow a Mind: Statistics, Structure, and Abstraction. *Science*, 331(6022), 1279–1285. <https://doi.org/10.1126/science.1192788>
- Turing, A. M. (1937). On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 1, 230–265. <https://doi.org/10.1112/plms/s2-42.1.230>
- Ullman, T. D., & Tenenbaum, J. B. (n.d.). *Bayesian Models of Conceptual Development: Learning as Building Models of the World*.
- Ullman, T. D., Goodman, N. D., & Tenenbaum, J. B. (n.d.). *Theory Acquisition as Stochastic Search*.
- Villiers, J. G. de, & College, S. (n.d.). *Can Language Acquisition give Children a Point-of-View?*.
- Wellman, H. M. (2018). Theory of mind: The state of the art. *European Journal of Developmental Psychology*, 15(6), 728–755. <https://doi.org/10.1080/17405629.2018.1435413>
- Wellman, H. M., & Gelman, S. A. (1992). Cognitive Development: Foundational Theories of Core Domains. In *Annual review of psychology. Volume 43, 1992: Annual review of psychology. Volume 43, 1992*. Annual Reviews Inc.
- Wellman, H. M., & Liu, D. (2004). Scaling of Theory-of-Mind Tasks. *Child Development*, 75(2), 523–541. <https://doi.org/10.1111/j.1467-8624.2004.00691.x>
- Wellman, H. M., Carey, S., Gleitman, L., Newport, E. L., & Spelke, E. S. (1990). *The Child's Theory of Mind*. The MIT Press. <https://doi.org/10.7551/mitpress/1811.001.0001>
- Wellman, H. M., Cross, D., & Watson, J. (2001). Meta-Analysis of Theory-of-Mind Development: The Truth about False Belief. *Child Development*, 72(3), 655–684. <https://doi.org/10.1111/1467-8624.00304>
- Wellman, H. M., Kushnir, T., Xu, F., & Brink, K. A. (2016). Infants Use Statistical Sampling to Understand the Psychological World. *Infancy*, 21(5), 668–676. <https://doi.org/10.1111/inf.12131>
- Wimmer, H., & Perner, J. (n.d.). *Beliefs about beliefs: Representation and constraining function of wrong beliefs in young children's understanding of deception*.
- Woodward, A. (1998). Infants selectively encode the goal object of an actor's reach. *Cognition*, 69(1), 1–34. [https://doi.org/10.1016/S0010-0277\(98\)00058-4](https://doi.org/10.1016/S0010-0277(98)00058-4)

- Xu, F. (2019). Towards a rational constructivist theory of cognitive development. *Psychological Review*, 126(6), 841–864. <https://doi.org/10.1037/rev0000153>
- Xu, F., & Kushnir, T. (2013). Infants Are Rational Constructivist Learners. *Current Directions in Psychological Science*, 22(1), 28–32. <https://doi.org/10.1177/0963721412469396>
- Xu, F., & Tenenbaum, J. B. (2007). Word learning as Bayesian inference. *Psychological Review*, 114(2), 245–272. <https://doi.org/10.1037/0033-295X.114.2.245>
- Xu, F., Kushnir, T., & Benson, J. B. (2012). *Rational constructivism in cognitive development* (Issue 43). Academic Press.
- Zettlemoyer, L. S., & Collins, M. (n.d.). *Learning to Map Sentences to Logical Form: Structured Classification with Probabilistic Categorical Grammars*.